# NEW DESIGNS OF POST-QUANTUM CRYPTOGRAPHIC CANDIDATES

## LATTICE BASED DIGITAL SIGNATURE SCHEME


REPORT BY

ARUP MAZUMDER

PRINCIPAL INVESTIGATOR

SHASHANK SINGH

*Indian Institute of Science Education and Reseach Bhopal*


PROJECT START DATE: NOVEMBER 30, 2022



2025

IISER BHOPAL

# Abstract

In this report, we represent the design, analysis, and implementations of a new lattice-based digital signature scheme which is intended to be a quantum secure. This report begins with the essential mathematical foundations such as lattice theory, discrete Gaussians, algebraic number theory, the shortest vector problem, the closest vector problem, standard lattice hard problems, the Shortest Integer Solutions (SIS), and the Learning With Errors (LWE) problems and its ring/module version. Moreover, we surveyed the two main paradigms for lattice-based signatures: the Fiat-Shamir with Aborts framework (specifically CRYSTALS-Dilithium) and the GPV hash-and-sign framework over the NTRU lattice (specifically Falcon and its successors Mitaka, Antrag, Zalcon and Solmae). After considering the trade-offs of each framework, our particular concerns were floating-point arithmetic, side-channel leakage, and implementation difficulties in GPV-style schemes; the report adopts the Fiat-Shamir with Aborts paradigm with the mathematical hard problems module-LWE and module-SIS.

Depending on these foundations, a slightly new digital signature scheme is proposed that closely follows Dilithium but introduces a modified key-generation and rejection sampling strategy by aiming to reduce the expected number of signing repetitions while preserving public key size. The scheme is formally secure based on the hardness of module-LWE (key recovery) and module-SIS/SelfTarget module-SIS (forgery), with concrete security estimated using lattice-reduction (BKZ) and Core-SVP hardness analysis. The proposal specifies two parameter sets: one reuses Dilithium's prime, and the other uses an alternative NTT-friendly prime, across NIST security levels 2, 3, and 5. Finally, two independent Python implementations are provided: one modeled on the Dilithium reference code and the other a direct FIPS-204-style implementation. Both the implementations are validated against known-answer test vectors and benchmarked for repetition counts and signing time, confirming the theoretical efficiency gains in practice.

# Contents

# Milestone 1

# Prerequisites and important lattice-based schemes for DSAs

The public key cryptographic schemes base their security on the hardness of very complicated mathematical problems intractable by both classical computers and future special/general-purpose quantum computers. The most important mathematical problems currently used are the factorisation problem and the discrete logarithm problem. Both of them are vulnerable to quantum computers, thanks to Shor [85] algorithm. The hard lattice problems, namely the Shortest Vector Problem (SVP), the Closest Vector Problem (CVP), and their variants, are assumed to be quantum resistant , as an efficient algorithm to solve them efficiently using quantum computers does not exist. They are already known to be computationally hard for classical computation. This makes them suitable to be used in designing cryptographic protocols.

## 1.1 Digital Signature Scheme

The digital signature of a digital file (say message) is another digital file (say signature), created using the secret key of the signer, which, when valid, guarantees the authenticity/approval of the message. Anyone with the corresponding public key can check the validity of the digital signature. The digital signature is a tiny file often embedded into the original file itself.

**DEFINITION 1.1.1: Digital Signature**

A (digital) signature scheme consists of three probabilistic polynomial times (PPT) algorithms $(\texttt{GEN}, \texttt{SIGN}, \texttt{VRFY})$ such that:

- **GEN:** $(pk, sk) \leftarrow \texttt{GEN}(1^n)$. We assume that $pk$ and $sk$ each has length at least $n$, and that $n$ can be determined from $pk$ or $sk$.
- **SIGN:** It outputs the signature $\sigma \leftarrow \texttt{SIGN}_{sk}(\mu)$, where $sk$ is a secret key and $\mu$ is a message.
- **VRFY:** It takes as input a $pk$, a message $\mu$, and a signature $\sigma$ and outputs a bit $b$, with $b = 1$ meaning valid and $b = 0$ meaning invalid. $b := \texttt{VRFY}_{pk}(\mu, \sigma)$.

It is required that except with negligible probability over $(pk, sk)$ output by $\texttt{GEN}(1^n)$, it holds that $\texttt{VRFY}_{pk}(\mu, \texttt{SIGN}_{sk}(\mu)) = 1$ for every (legal) message $\mu$.

Let $\Pi = (\texttt{GEN}, \texttt{SIGN}, \texttt{VRFY})$ be a signature scheme, and consider the following experiment

for an adversary $\mathcal{A}$ and parameter $n$.

**DEFINITION 1.1.2: The signature experiment**

$\text{SigForge}_{\mathcal{A},\Pi}(n)$:

- $(pk, sk) \leftarrow \texttt{GEN}(1^n)$.
- $\mathcal{A}$ is given $pk$ and access to an oracle $\texttt{SIGN}_{sk}(\cdot)$. $\mathcal{A}$ then outputs $(\mu, \sigma)$. Let $\mathcal{Q}$ denote the set of all oracle queries that $\mathcal{A}$ has made.
- $\mathcal{A}$ succeeds if and only if $\texttt{VRFY}_{pk}(\mu, \sigma) = 1$ and $\mu \notin \mathcal{Q}$. In this case, the experiment's output is defined as 1.

**DEFINITION 1.1.3**

A signature scheme $\Pi = (\texttt{GEN}, \texttt{SIGN}, \texttt{VRFY})$ is existentially unforgeable under an adaptive chosen-message attack (UFCMA), or just secure, if for all PPT adversaries $\mathcal{A}$, there is a negligible function $\varepsilon$ such that:

$$\Pr\left[\text{SigForge}_{\mathcal{A},\Pi}(n) = 1\right] \leq \varepsilon(n).$$

## 1.2 Introduction to lattice

**DEFINITION 1.2.4: Lattice**

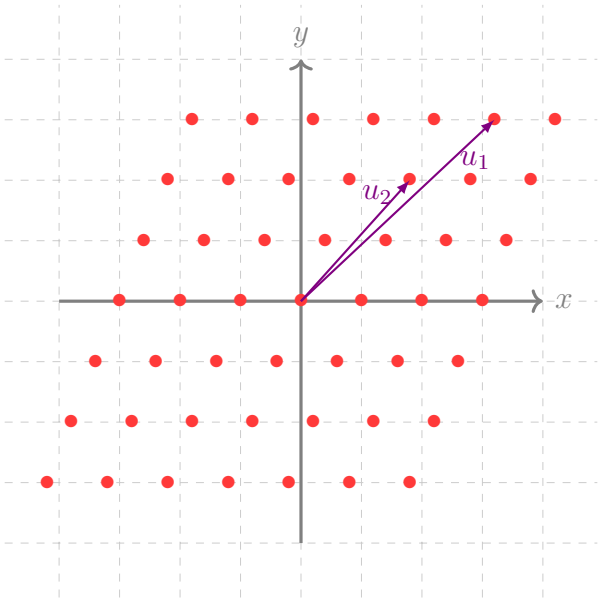
Let $\mathbb{R}^n$ be a real vector space of dimension $n$ and $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_m\} \subset \mathbb{R}^n$ be a set containing $m$ linearly dependent vectors of $\mathbb{R}^n$. A lattice $\mathcal{L}(\mathbf{B})$, generated by $\mathbf{B}$, is the set containing all the integer linear combinations of these vectors. The dimension of the lattice is $n$, and the rank is $m$. The set $\mathbf{B}$ is called the basis of the lattice. When $n = m$, the lattice is said to be of full rank.

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^{m} x_i \mathbf{b}_i : x_i \in \mathbb{Z} \right\} \tag{1.1}$$

In computer programming, a basis $\mathbf{B}$ of the lattice is treated as a two-dimensional matrix where the rows are the basis vectors. In the matrix notation, a lattice could be written as

$$\Lambda = \mathcal{L}(\mathbf{B}) = \{\mathbf{xB} : \mathbf{x} \in \mathbb{Z}^m\} \tag{1.2}$$

A two-dimensional lattice is pictorially shown below. It can be visualized as a regular arrangement of points in the Euclidean Space.




In the rest of the report, we consider full rank (i.e., $m = n$) lattices only and almost always choose the coordinates of lattice bases from the set of integers to facilitate the coding (computer implementation) aspect of the lattice. Several bases are generating the same lattice. The two such lattice bases $\mathbf{B}$ and $\mathbf{B}'$, such that $\mathcal{L}(\mathbf{B}) = \mathcal{L}(\mathbf{B}')$, are related by $\mathbf{B} = \mathbf{B}'\mathbf{U}$, where $\mathbf{U}$ is a unimodular lattice i.e. $\det(\mathbf{U}) = \pm 1$. A lattice basis is also a basis for the vector space $\mathbb{R}^n$, but a basis for $\mathbb{R}^n$ may not be a basis for the lattice.

$$\mathbb{R}^n = \text{span}(\mathbf{B}) = \{\mathbf{xB} : \mathbf{x} \in \mathbb{R}^m\} \tag{1.3}$$

**DEFINITION 1.2.5: Fundamental Parallelopiped / Determinant for a lattice**

For a lattice $L = \mathcal{L}(\mathbf{B})$, we define the fundamental parallelopiped as the set

$$\mathcal{F}(B) = \left\{ \sum_{i=1}^{m} x_i \mathbf{b}_i : 0 \leq x_i < 1 \right\} \subset \mathbb{R}^n \tag{1.4}$$

The volume of $\mathcal{F}(L)$ is invariant of the lattice bases and is called the determinant of the lattice $L$. For a full-rank lattice, it is given by $\mathcal{F}(L) = \det(B)$.

A basis $\mathbf{B}$ for a vector space $\mathbb{R}^n$ can be transformed using the well-known Gram-Schmidt orthogonalization method to another basis for $\mathbb{R}^n$, whose vectors are mutually orthogonal i.e., $\langle \mathbf{b}_i, \mathbf{b}_j \rangle = 0$ where $\langle \mathbf{b}_i, \mathbf{b}_j \rangle$ represents the dot (inner) product of two vectors $\mathbf{b}_i$ and $\mathbf{b}_j$.

**DEFINITION 1.2.6: Gram-Schmidt orthogonalization method (GSO)**

Let $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$. We iteratively define a new set $\tilde{\mathbf{B}} = \{\tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}_2, \ldots, \tilde{\mathbf{b}}_n\}$ as follows.

$$\tilde{\mathbf{b}}_1 = \mathbf{b}_1$$

$$\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \tilde{\mathbf{b}}_j \text{ where } \mu_{i,j} = \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle}$$

In the above, we basically removed the components of $\mathbf{b}_i$ along $\tilde{\mathbf{b}}_j$ from $\mathbf{b}_i$ for all $j = 1, 2, \ldots (i-1)$ and hence the set $\tilde{\mathbf{B}}$ is again a basis of $\mathbb{R}^n$ with the mutually orthogonal basis vectors.

Note that if the $\mathbf{B}$ is a lattice basis, $\tilde{\mathbf{B}}$ may not be a basis of the same lattice. However, it is possible to transform a lattice basis into another basis for the same lattice whose vectors are nearly orthogonal.

**Algorithm 1:** Lenstra-Lenstra-Lovász (LLL) Reduction

**Input:** A lattice basis $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$, $\mathbf{b}_i \in \mathbb{Z}^n$ and $\delta \in [3/4, 1)$.
**Output:** Another lattice basis of $\mathcal{L}(\mathbf{B})$.

Start: Compute $\tilde{\mathbf{B}} = \{\tilde{\mathbf{b}}_1, \tilde{\mathbf{b}}, \ldots, \tilde{\mathbf{b}}_n\}$
Reduction Step:
**for** $i = 2$ **to** $n$ **do**
  **for** $j = i - 1$ **to** $1$ **do**
    $\mathbf{b}_i = \mathbf{b}_i - c_{i,j}\mathbf{b}_j$, where $c_{i,j} = \lfloor \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle} \rceil$  `// this does not affect GSO`
Swap Step:
**if** $\exists i$ *such that* $\delta \|\tilde{\boldsymbol{b}}_i\|^2 > \|\mu_{i+1,i}\tilde{\boldsymbol{b}}_i + \tilde{\boldsymbol{b}}_{i+1}\|^2$ **then**
  swap($\mathbf{b}_i, \mathbf{b}_{i+1}$); **go to** the step 1.

Note that in the Algorithm 1, we have taken the coordinates of basis vectors as integers just for efficiency and ease of implementation. The Algorithm 1 transforms a basis into another lattice basis whose vectors are nearly orthogonal. The Algorithm 1 runs in the polynomial time in $n$ (for proof we refer to the book [66]) and a basis returned by the algorithm is called a $\delta$-LLL reduced basis of $\mathcal{L}(\mathbf{B})$.

**DEFINITION 1.2.7: $\delta$-LLL Reduced Basis**

A basis $\mathbf{B}$ of the lattice is called a $\delta$-LLL reduced basis if it satisfies the following properties:

1. $|\mu_{i,j}| \leq 1/2$ for $j = 1, 2, \ldots, i-1$, where $\mu_{i,j} = \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle}$.
2. $\delta \|\tilde{\mathbf{b}}_i\|^2 > \|\mu_{i+1,i}\, \tilde{\mathbf{b}}_i + \tilde{\mathbf{b}}_{i+1}\|^2$ for all $i$.

We further note that any set of $n$ linearly independent vectors does not form a basis for the lattice.

**THEOREM 1.2.8**

A set $S$ of $n$ linearly independent vectors from $\mathcal{L}(\mathbf{B})$ forms a basis of the lattice iff fundamental parallelopiped $\mathcal{F}(S)$ does not contain lattice vectors.

**DEFINITION 1.2.9: Successive Minima for a lattice**

For a lattice $\Lambda = \mathcal{L}(\mathbf{B})$ be a lattice of rank $n$. For $i \in \{1, 2, \ldots, n\}$, we define successive

minima $\lambda_i$ as follows:

$$\lambda_i(\Lambda) = \inf\left\{r : \dim\left(\text{span}\left(\bar{\mathcal{B}}(0,r) \cap \mathcal{L}(\mathbf{B})\right)\right) \geq i\right\}, \tag{1.5}$$

where $\bar{\mathcal{B}}(0,r) = \{\mathbf{a} \in \mathbb{R}^n : \|\mathbf{a}\| \leq r\}$ is a closed ball of radius $r$ around origin.

The quantity $\lambda_1(\Lambda)$ represents the length of shortest lattice vector, say $\mathbf{v}_1$, $\lambda_2(\Lambda)$ represents the length of second shortest vector linearly independent to $\mathbf{v}_1$ and so on. It can be proved that all the successive minima are achieved, i.e., there exist lattice vectors $\mathbf{v}_i \in \Lambda$ such that $\lambda_i = \|\mathbf{v}_i\| \ \ \forall i$.

Though there exists a lattice vector $\mathbf{v}_1 \in \Lambda$ of the smallest norm (length), it is difficult to compute that vector or the value of $\lambda_1(\Lambda)$ when the rank of the lattice is large. We, instead, have bounds for the value of $\lambda_1(\Lambda)$.

**THEOREM 1.2.10**

Let $\mathbf{B}$ be a basis for the lattice $\Lambda$ of rank $n$ and $\tilde{\mathbf{B}}$ be the GSO of the set $\mathbf{B}$, we have

$$0 \ < \ \min_i \|\tilde{\mathbf{b}}_i\| \leq \lambda_1\left(\mathcal{L}(\mathbf{B})\right) \leq \sqrt{n} \ \left(\det \Lambda\right)^{1/n} \tag{1.6}$$

In the cryptographic application of lattices, we often encounter a dual and orthogonal of a given lattice $\Lambda$. The following defines a dual and an orthogonal lattice.

**DEFINITION 1.2.11: Dual Lattice**

Let $\Lambda \subset \mathbb{R}^n$ be a lattice of rank $m$, we define a set

$$\Lambda^\vee = \{\mathbf{u} \in \text{span}(\Lambda) \mid \forall \mathbf{a} \in \Lambda : \langle \mathbf{u}, \mathbf{a} \rangle \in \mathbb{Z}\}. \tag{1.7}$$

It can be easily proved that the set $\Lambda^\vee$ is a discrete subgroup of $\mathbb{R}^n$ and hence is a lattice called a dual of the lattice $\Lambda$. For a full-rank lattice $\Lambda = \mathcal{L}(\mathbf{B})$, a basis of $\Lambda^\vee$ can be obtained by transposing the inverse matrix of $\mathbf{B}$. For the proof, we refer to the book [66].

**DEFINITION 1.2.12: Orthogonal Lattice**

Let $\Lambda \subset \mathbb{Z}^n$ be an integer lattice of rank $m$ in $\mathbb{R}^n$, we define a set

$$\Lambda^\perp = \{\mathbf{u} \in \mathbb{Z}^n \mid \forall \mathbf{a} \in \Lambda : \langle \mathbf{u}, \mathbf{a} \rangle = 0\}. \tag{1.8}$$

The set $\Lambda^\perp$ being a subgroup of $\mathbb{Z}^n$ is a lattice. It is called the orthogonal of the lattice $\Lambda$. For a full rank lattice $\Lambda = \mathcal{L}(\mathbf{B})$, a basis of $\Lambda^\perp$ can be obtained by the transpose of the inverse matrix of $\mathbf{B}$. For the proof, we refer to the book [66].

## 1.3 Hard Lattice Problem

**DEFINITION 1.3.13: Shortest Vector Problem (SVP)**

Given a lattice $\Lambda$ generated by the basis $\mathbf{B}$ i.e., $\Lambda = \mathcal{L}(\mathbf{B})$. The shortest vector problem is to

find a non-zero lattice vector $\mathbf{b} \in \mathcal{L}(\mathbf{B})$ such that $\|\mathbf{v}\| = \lambda_1$ i.e., $\mathbf{v}$ is one of the shortest lattice vectors.

Since there are infinitely many lattice vectors, it is not possible to use a straight forward brute force technique to find the smallest lattice vector. We need to, somehow (using the properties of the lattice), bring down our search to a set of finitely many lattice vectors. This is the main idea behind enumeration algorithms [48] for lattice problems. Kannan, in his paper [48], showed that it is possible to find out a finite set $S \subset \mathbb{Z}^n$ such that $\{\mathbf{xB} : \mathbf{x} \in S\}$ contains the shortest lattice vector. The enumeration algorithms for SVP have also been studied by Fincke and Pohst [39] and by Schnorr and Euchner [84]. In 2001, Ajtai, Kumar and Sivakumar [5] proposed a different asymptotic approach for finding short vectors. However, for smaller dimensions, the enumeration algorithms are more promising. For example, the HKZ-reduction algorithm [68] for SVP runs in time $2^{O(n \log n)}$ and space requirement is polynomial in $n$. Since the state-of-the-art algorithms for solving SVP are all exponential in the dimension $n$ of the lattice, it is practically impossible to solve SVP for a decent size of $n$. There is no quantum algorithm for solving the SVP whose time complexity is better than exponential in $n$. More precisely, there are three variants of the SVP. They are listed below:

- **Search SVP:** Given a lattice basis $\mathbf{B}$, find $\mathbf{v} \in \mathcal{L}(\mathbf{B})$ such that $\|\mathbf{v}\| = \lambda_1(\mathcal{L}(\mathbf{B}))$.
- **Optimisation SVP:** Given a lattice basis $\mathbf{B}$, find the value of $\lambda_1(\mathcal{L}(\mathbf{B}))$.
- **Decision SVP:** Given a lattice basis $\mathbf{B}$ and a rational number $r$, decide whether $\lambda_1(\mathcal{L}(\mathbf{B})) \leq r$ or not.

However, instead of solving the exact SVP, if we allow some scope of errors, i.e., if we are interested in a lattice vector whose length is at most some approximation factor $\gamma(n) \geq 1$ times the length of the smallest vector, there are efficient algorithms available. This version of the SVP is called the approximation variant of SVP. Corresponding to each SVP variant, we have its approximation variant as well.

- **Search SVP$_\gamma$**: Given a lattice basis $\mathbf{B}$, find $\mathbf{0} \neq \mathbf{v} \in \mathcal{L}(\mathbf{B})$ such that $\|\mathbf{v}\| \leq \gamma\lambda_1(\mathcal{L}(\mathbf{B}))$.
- **Optimisation SVP$_\gamma$**: Given a lattice basis $\mathbf{B}$, find $d$ such that the value of $d \leq \lambda_1(\mathcal{L}(\mathbf{B})) \leq \gamma d$.
- **Promise SVP$_\gamma$**: Given a lattice basis $\mathbf{B}$ and a rational number $r$, return YES if $\lambda_1(\mathcal{L}(\mathbf{B})) \leq r$; return NO if $\lambda_1(\mathcal{L}(\mathbf{B})) > \gamma \cdot r$

Another fundamental lattice problem is the closest vector problem. (CVP).

**DEFINITION 1.3.14: Closest Vector Problem (CVP)**

Let $\Lambda$ be a lattice generated by the basis $\mathbf{B}$ i.e., $\Lambda = \mathcal{L}(\mathbf{B})$. Given a vector $\mathbf{t} \in \mathbb{R}^n$, the closest vector problem is to find a lattice vector $\mathbf{b} \in \mathcal{L}(\mathbf{B})$ closest to $\mathbf{t}$ i.e., $\|\mathbf{b} - \mathbf{t}\| \leq \|\mathbf{v} - \mathbf{t}\|$ for all $\mathbf{v} \in \Lambda$.

There are three approximation variants of CVP similar to that of SVP.

- **Search CVP$_\gamma$**: Given a lattice basis $\mathbf{B}$ and a vector $\mathbf{t} \in \mathbb{R}^n$, find $\mathbf{b} \in \mathcal{L}(\mathbf{B})$ such that $\|\mathbf{b} - \mathbf{t}\| \leq \gamma \cdot \mathrm{dist}\,(\mathbf{t}, \mathcal{L}(\mathbf{B}))$.

- **Optimisation CVP$_\gamma$**: Given a lattice basis $\mathbf{B}$ and a vector $\mathbf{t} \in \mathbb{R}^n$, find $d \in \mathbb{R}$ such that $\text{dist}(\mathbf{t}, \mathcal{L}(\mathbf{B})) \leq \gamma \cdot d$.
- **Promise CVP$_\gamma$**: An instance of the problem is a triplet $(\mathbf{B}, \mathbf{t}, r)$. In YES instance $\text{dist}(\mathbf{t}, \mathcal{L}(\mathbf{B})) \leq r$; in NO instance $\text{dist}(\mathbf{t}, \mathcal{L}(\mathbf{B})) > \gamma \cdot r$.

## 1.4 (Discrete) Gaussians

The recent lattice-based public key crypto primitives rely on the Gaussian-like discrete probability distributions over lattices, called discrete Gaussians. We recall these notions in this section.

**DEFINITION 1.4.15: Gaussian Distribution on Lattice**

Let $n > 0$ be a positive integer and $s > 0$ be a real number. Define the Gaussian function $\rho_s : \mathbb{R}^n \to \mathbb{R}^+$ with the width $s$, as

$$\rho_s(\mathbf{x}) := \exp\left(-\pi \|\mathbf{x}\|^2 / s^2\right). \tag{1.9}$$

For a lattice coset $c + \Lambda \subset \mathbb{R}^n$ and parameter $s > 0$, the discrete Gaussian probability distribution $D_{c+L,s}(\mathbf{x})$ is simply the Gaussian distribution restricted to the coset:

$$D_{c+L,s}(\mathbf{x}) = \begin{cases} \rho_s(\mathbf{x}) & \text{if } \mathbf{x} \in c + \Lambda \\ 0 & \text{otherwise.} \end{cases} \tag{1.10}$$

## 1.5 Algebraic Number Theory

In this section, we review the prerequisite for studying a variant of a hard lattice problem, namely the Ring Learning with Error (RLWE) Problem.

**DEFINITION 1.5.16**

A number $\alpha \in \mathbb{C}$ is called an algebraic number if there exists a polynomial $f(X) \in \mathbb{Q}[X]$ such that $f(\alpha) = 0$. For an algebraic number, there exists a monic irreducible polynomial of minimum degree $f_\alpha(X) \in \mathbb{Q}[X]$, of which $\alpha$ is a root. This polynomial $f_\alpha(X)$ is called the minimum polynomial of $\alpha$. All other complex roots of $f_\alpha(X)$ are the conjugates of $\alpha$.

An algebraic number is called an algebraic integer if its minimum polynomial is in $\mathbb{Z}[X]$, i.e., its coefficients are all integers.

**DEFINITION 1.5.17**

An algebraic number field is an extension field $K = \mathbb{Q}(\alpha)$ obtained by adjoining an algebraic number $\alpha$ to the field $\mathbb{Q}$. The field $\mathbb{Q}(\alpha)$ is an algebraic extension of the field $\mathbb{Q}$ with the degree of extension $[\mathbb{Q}(\alpha) : \mathbb{Q}]$ equal to the the degree (say $n$) of the minimal polynomial $f_\alpha(X)$ of $\alpha$. The number field $\mathbb{Q}(\alpha)$ can be realised as $\frac{\mathbb{Q}[X]}{\langle f_\alpha(X) \rangle}$ i.e., the elements of $\mathbb{Q}(\alpha)$ consist of polynomials $g(x)$, where $x = X \mod f_\alpha(X)$, of degree at most $n - 1$.

**DEFINITION 1.5.18**

A number field $K = \mathbb{Q}(\alpha)$ forms a vector space over its subfield $\mathbb{Q}$ of dimension $n$ with a vector space basis $\{1, \alpha, \alpha^2, \ldots, \alpha^{n-1}\}$. This is called a power basis of $K \,|\, \mathbb{Q}$. We can embed number field $K = \mathbb{Q}(\alpha)$ into $\mathbb{C}$ into $n$ different ways. These embeddings are $\mathbb{Q}$-isomorphisms $\sigma_i$'s given by $\sigma_i(\alpha) = \beta_i$, where $\beta_i$ is a conjugate of $\alpha$. These are classified into two classes, real embeddings, and complex embedding, based on whether $\beta_i$ is a real or complex number. Since the complex roots of $f_\alpha(X)$ are in pairs, the complex embedding is also in pairs. Thus we have total $n = (s_1 + 2s_2)$ embeddings out of which $\{\sigma_i : 1 \leq i \leq s_1\}$ are real embeddings and $\{\sigma_i : s_1 + 1 \leq i \leq s_2\} \cup \{\sigma_i = \overline{\sigma_{i-s_2}} : s_1 + s_2 + 1 \leq i \leq 2s_2\}$ are complex embeddings.

**DEFINITION 1.5.19**

For the algebraic extension $K|\mathbb{Q}$, we define $\mathrm{Tr}(\alpha)$ as the value $\sum_{i=1}^{n} \beta_i$; the norm $\mathrm{N}(\alpha)$ is defined as $\prod_{i=1}^{n} \beta_i$, where $\beta_i$'s are conjugates of $\alpha$.

**DEFINITION 1.5.20: Ring of algebraic integers**

Let $K \mid \mathbb{Q}$ be an algebraic number field. Define a set

$$\mathcal{O}_K = \{\alpha \in K \mid \alpha \text{ is an algebraic integer } \}.$$

$\mathcal{O}_K$ forms a ring with respect to addition and multiplication. The set $\mathcal{O}_K$ also be seen as $\mathbb{Z}$-module of rank $n$ i.e., there is a basis $B = \{b_1, b_2, \ldots, b_n\} \subset \mathcal{O}_K$, whose integer linear combinations form the set $\mathcal{O}_K$. Interestingly, this basis $B$ is also a basis of the vector space $K(Q)$.

**DEFINITION 1.5.21**

An integral ideal $\mathfrak{I}$ of $\mathcal{O}_K$ is defined as an additive subgroup of $\mathcal{O}_K$ which is closed under multiplication in $\mathcal{O}_K$. Let $\mathfrak{I}_1$ and $\mathfrak{I}_2$ be two integral ideal of $\mathcal{O}_K$, we define sum and product of ideal as follows:

$$\mathfrak{I}_1 + \mathfrak{I}_2 = \{a + b \mid a \in \mathfrak{I}_1 \text{ and } b \in \mathfrak{I}_2\} \tag{1.11}$$

$$\mathfrak{I}_1\mathfrak{I}_2 = \{ab \mid a \in \mathfrak{I}_1 \text{ and } b \in \mathfrak{I}_2\} \tag{1.12}$$

$\mathfrak{I}_1 + \mathfrak{I}_2$ and $\mathfrak{I}_1\mathfrak{I}_2$ are also integral ideals. An integral ideal $\mathfrak{p}$ is said to be a prime ideal iff $a \notin \mathfrak{p}$ and $b \notin \mathfrak{p}$ implies $ab \notin \mathfrak{p}$ for all $a, b \in \mathcal{O}_K$. Note that the ideal $\mathfrak{I}_1 + \mathfrak{I}_2$ contains ideals $\mathfrak{I}_1$ and $\mathfrak{I}_2$ and the ideal $\mathfrak{I}_1\mathfrak{I}_2$ is contained in the ideals $\mathfrak{I}_1$ and $\mathfrak{I}_2$ both. Furthermore, every prime ideal $\mathfrak{p}$ of $\mathcal{O}_K$ is a maximal ideal. We define the norm $\mathrm{N}(\mathfrak{I})$ of an ideal $\mathfrak{I}$ is the index of $\mathfrak{I}$ in $\mathcal{O}_K$ i.e., $\mathrm{N}(\mathfrak{I}) = |\frac{\mathcal{O}_K}{\mathfrak{I}}|$. The ideal norm is a multiplicative function. The norm $\mathrm{N}(\mathfrak{a})$ of a principal ideal $\mathfrak{a} = (a)$ is equal to $\mathrm{N}_{K/Q}(a)$.

**PROPOSITION 1.5.22**

If a prime ideal $\mathfrak{p} \subset \mathcal{O}_K$ contains a product $\prod_{j=1}^{m} \mathfrak{i}_j$ of finitely many ideals $\mathfrak{i}_1, \mathfrak{i}_2, \ldots, \mathfrak{i}_m$ then

$\mathfrak{p}$ contains $\mathfrak{i}_j$ for some $j$.

**THEOREM 1.5.23**

Every integral ideal $\mathfrak{I}$ of $\mathcal{O}_K$ can be written as a product of powers of prime ideals of $\mathcal{O}_K$, i.e.,

$$\mathfrak{I} = \mathfrak{p}_1^{e_1} \cdot \mathfrak{p}_2^{e_2} \cdot \ldots \cdot \mathfrak{p}_t^{e_t},$$

where $\mathfrak{p}_i$ are prime ideals of $\mathcal{O}_K$ and $e_i \in \mathbb{Z}^+$.

**DEFINITION 1.5.24: Fractional Ideal**

A fractional ideal $\mathfrak{J}$ is an $\mathcal{O}_K$-module for which there exists $r \in \mathcal{O}_K^\star$ such that $d\mathfrak{J}$ is an integral ideal of $\mathcal{O}_K$. The quantity $d$ is the denominator of the fractional ideal $\mathfrak{J}$. Note that the fraction ideal is not the integral ideal of $\mathcal{O}_K$; it is not even the subset of $\mathcal{O}_K$. However, an integral ideal can be treated as a fraction ideal with denominator $1$.

We can extend the concept of norm and factorisation to the fractional ideal as follows. For a fractional ideal $\mathfrak{J}$, with a denominator $d \in \mathcal{O}_K^\star$, we define $\mathrm{N}(\mathfrak{J}) = \mathrm{N}(d\mathfrak{J}) \cdot \mathrm{N}(d)^{-1}$. If we have $d\mathfrak{J} = \prod_{1 \le i \le m_1} \mathfrak{p}_i^{e_i}$ and $(d) = \prod_{1 \le i \le m_2} \mathfrak{q}_i^{g_i}$, we have the factorisation of fractional ideal $\mathfrak{J} = \prod_{1 \le i \le m_1} \mathfrak{p}_i^{e_i} \prod_{1 \le i \le m_2} \mathfrak{q}_i^{-g_i}$.

**PROPOSITION 1.5.25**

Let $\mathfrak{I}$ be a non-zero prime ideal of a $\mathcal{O}_K$. Define $\mathfrak{J} = \{\alpha \in K \mid \alpha\mathfrak{I} \subset R\}$. Then $\mathfrak{J}$ is fractional ideal of $\mathcal{O}_K$. Furthermore $\mathcal{O}_K \subsetneq \mathfrak{J}$ and $\mathfrak{I}\mathfrak{J} = \mathcal{O}_K$.

**THEOREM 1.5.26**

If $\mathfrak{I}$ is a nonzero fractional ideal of $\mathcal{O}_K$ , then $\mathfrak{I}$ can be factored uniquely as $\prod_{i \le m} \mathfrak{p}_i^{n_i}$ a product of prime ideals, where the $n_i \in \mathbb{Z}$ are integers.

## 1.6 Ideal Lattice

**DEFINITION 1.6.27**

Let $K = \mathbb{Q}(\alpha)$ be a number field with embedding degree $n = s_1 + 2s_2$. The Minkowski or canonical embedding is an injective ring homomorphism $\sigma : K \to \mathbb{R}^{s_1} \times \mathbb{C}^{s_2} = \mathbb{R}^n$ given by $\sigma(a) = (\sigma_1(a), \sigma_2(a), \ldots \sigma_{s_1+s_2}(a))$, where $\sigma_i$'s are the embeddings of number field $\mathbb{Q}(\alpha)$ (see the Definition 1.5.18) out of which $\sigma_1, \sigma_2, \ldots \sigma_{s_1}$ are real embeddings and $\sigma_{s_1+1}, \sigma_{s_1+2}, \ldots \sigma_{s_1+s_2}$ are complex embeddings, and the rest are the complex conjugates of the complex embeddings.

Let $b_1, b_2, b_3, \ldots, b_n \in L$ be linearly dependent over $\mathbb{Z}$ (hence the $b_i$'s form a basis for $K$ over $Q$). Let $C$ be the matrix whose $k^{\text{th}}$ column is

$$\sigma_1(b_k), \sigma_2(b_k), \ldots, \sigma_{s_1}(b_k), \mathbf{Re}\,\sigma_{s_1+1}(b_k), \mathbf{Im}\,\sigma_{s_1+1}(b_k), \ldots, \mathbf{Re}\,\sigma_{s_1+s_2}(b_k), \mathbf{Im}\,\sigma_{s_1+S_2}(b_k).$$

It can be shown that

$$\det(C) = (2i)^{-s_2} \det(\sigma_j(b_k)). \tag{1.13}$$

For the proof, we refer to the book [10]. Now let $M$ be the free $\mathbb{Z}$-module generated by the $b_i$, so that $\sigma(M)$ is a free $\mathbb{Z}$-module with basis $\sigma(b_i)$ for $i = 1, \ldots, n$, hence a lattice in $\mathbb{R}^n$.

**DEFINITION 1.6.28: Ideal Lattice**

Let $\mathfrak{J}$ be a fractional ideal of $\mathcal{O}_K$, with basis $(b_1, b_2, \ldots, b_n)$, its image under canonical embedding is a lattice $\sigma(\mathfrak{J}) \subset \mathbb{R}^n$, given by the basis $B = \{\sigma(b_1), \sigma(b_2), \ldots, \sigma(b_n)\}$. Such a lattice $\sigma(\mathfrak{J})$ is called an ideal lattice.

**DEFINITION 1.6.29: Ideal Lattice Problem**

All computational problems can be defined for a general lattice and also for the ideal lattice. A computation problem in an ideal lattice may be easy, but hard on the general lattice. We will explore more on this later.

## 1.7 Learning With Errors (LWE)

The Learning With Errors (LWE) was introduced by Regev [80] in 2005. Nowadays, it has become a Swiss army knife for public key cryptographers, as it is assumed to be quantum-safe and has been shown to be as hard as worst-case lattice problems. This makes it an ideal candidate to base the security of public key cryptographic protocols.

As its name indicates, an LWE problem asks to learn a secret $\mathbf{s} \in \mathbb{Z}_q^n$, given a sequence of approximate random linear relations on $\mathbf{s}$. The following example given in a survey article by Regev [81], adds to the explanation of LWE problem.

$$\begin{aligned}
14s_1 + 15s_2 + 5s_3 + 2s_4 &\approx 8 \pmod{17} \\
13s_1 + 14s_2 + 14s_3 + 6s_4 &\approx 16 \pmod{17} \\
6s_1 + 10s_2 + 13s_3 + 1s_4 &\approx 3 \pmod{17} \\
10s_1 + 4s_2 + 12s_3 + 16s_4 &\approx 12 \pmod{17} \\
9s_1 + 5s_2 + 9s_3 + 6s_4 &\approx 9 \pmod{17} \\
3s_1 + 6s_2 + 4s_3 + 5s_4 &\approx 16 \pmod{17} \\
&\vdots \\
6s_1 + 7s_2 + 16s_3 + 2s_4 &\approx 3 \pmod{17},
\end{aligned}$$

where each equation is correct up to some small additive error (say, $\pm 1$), and the LWE problem asks to recover the secret $\mathbf{s} = (s_1, s_2, s_3, s_4)$. (The answer in this case is $\mathbf{s} = (0, 13, 9, 11)$.) If the above equations were exact, one could very efficiently find the secret $(s_1, s_2, s_3, s_4)$ using the Gaussian Elimination technique. The introduction of small errors has made the problem much more difficult. In this section, we will explore the difficulty of the LWE problem. Before we do that, let us describe the problem more formally.

**DEFINITION 1.7.30: LWE**

Let $n \geq 1$ and $q \geq 2$ be positive integers. Let $\chi$ be a (error) probability distribution on $\mathbb{Z}_q$. Let $\mathcal{A}_{\mathbf{s},\chi}$ be a probability distribution obtained by choosing a vector $\mathbf{a} \in \mathbb{Z}_q^n$ uniformally at random, $e \in \mathbb{Z}_q$ according to the distribution $\chi$, and outputting $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e)$, where additions are performed modulo $q$. We say that an algorithm solves LWE with modulus $q$ and error distribution $\chi$ if, for any $\mathbf{s} \in \mathbb{Z}^n$, given an arbitrary number of independent samples from $\mathcal{A}_{\mathbf{s},\chi}$ it outputs $\mathbf{s}$ with high probability.

For the hardness and other variants of LWE, we refer to the survey [81]. Although the LWE problem has a hardness proof, it is inefficient for practical applications. An algebraic variant of LWE, known as ring-LWE [64] and denoted by RLWE, is defined over the ring $R_q = \mathbb{Z}_q[X]/\langle X^n + 1 \rangle$, where $q$ is prime, and is stated below. The RLWE is much better than the LWE in efficiency.

**DEFINITION 1.7.31: RLWE**

Let $n \geq 1$ and $q \geq 2$ be positive integers. Let $\chi$ be a (error) probability distribution on $R_q$. Let $\mathcal{A}_{\mathbf{s},\chi}$ be a probability distribution obtained by choosing a vector $\mathbf{a} \in R_q^n$ uniformly at random, $e \in R_q$ according to the distribution $\chi$, and outputting $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e)$, where additions are performed modulo $q$. We say that an algorithm solves LWE with modulus $q$ and error distribution $\chi$ if, for any $\mathbf{s} \in R_q^n$, given an arbitrary number of independent samples from $\mathcal{A}_{\mathbf{s},\chi}$ it outputs $\mathbf{s}$ with high probability.

**Advantages of RLWE**

1. There is a quantum reduction from worst case approximate $\alpha$-**SVP** on ideal lattices in ring $R$ to the search version of ring-LWE [64], where $R = \mathbb{Z}[X]/\langle X^n + 1 \rangle$.

   **Theorem:**There is a polynomial-time quantum reduction from $O(\sqrt{n} \log n/\alpha)$-approximate SVP for cyclotomic number field ideal lattices to RLWE with $\mathrm{poly}(n)$ samples when $\alpha q > \omega(\sqrt{n})$.

   The $\alpha$-**SVP** in regular lattices is known to be NP-hard due to work by Micciancio [69].

2. A Major advantage of cryptography based on RLWE over LWE-based cryptography is the size of the public and private keys. RLWE keys are roughly the square root of keys in LWE-based cryptography[64].

3. Each element $r \in R_q$ is polynomial of degree $n-1$ over $\mathbb{Z}_q$,and polynomial multiplication can be done in $O(n \log n)$ time by FFT.

Now, we define a more general version of ring-LWE, called module-LWE, denoted by MLWE. The MLWE was discovered by Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan [21], and it was further studied by Adeline Langlois and Damien Stehle[58]. Let $K$ be the number field of degree $n$ over the field $\mathbb{Q}$, and assume $O_k$ be the ring of integers of the number field $K$. Define $K_{\mathbb{R}} = K \otimes_{\mathbb{Q}} \mathbb{R}$. Let $\chi$ be a distribution on $K_{\mathbb{R}}$. Now, we define the search and decision versions of MLWE, denoted by $\mathrm{MLWE}_{qkmn\chi}$.

**DEFINITION 1.7.32: MLWE**

**Search Version:** $\mathrm{MLWE}_{qkmn\chi}$ is to find $\mathbf{s} \in R_q^k$ for given $(\mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e})$, where $\mathbf{A} \leftarrow R_q^{m\times k}$ and $\mathbf{s} \leftarrow R_q^k$ (but, it is unknown) and small error $\mathbf{e} \in \chi^m$.

**Decision Version:** The decision variant of $\mathrm{MLWE}_{qkmn\chi}$ is to decide whether $\mathbf{s} \in R_q^k$ is the solution of the given problem $(\mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e})$, where $\mathbf{A} \leftarrow R_q^{m\times k}$ and $\mathbf{s} \leftarrow R_q^k$ (but, it is kept secret) and small error $\mathbf{e} \in \chi^m$.(it is also kept secret)

**DEFINITION 1.7.33: Hardness of MLWE**

**Hardness of Search Version:** Let $q \geq 2$ be a modulus, $m = \mathrm{poly}(\lambda)$ be the number of samples, $k > 0$ be the rank of MLWE, and $n = \mathrm{poly}(\lambda)$ be the degree of the modular polynomial. Let $\chi$ be a distribution on $K_{\mathbb{R}}$. The search problem $\mathrm{MLWE}_{qkmn\chi}$ is hard if for any PPT adversary $\mathcal{A}$, there exists a negligible function $\mathrm{negl}(n)$ such that

$$\mathrm{Adv}_{qkmn\chi}^{\mathrm{MLWE}}(\mathcal{A}) := \Pr\left[\mathcal{A}\left(\chi, \mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e}\right) = \mathbf{s}\right] < \mathrm{negl}(n)$$

for given $\mathbf{A} \leftarrow R_q^{m\times k}$ and $\mathbf{s} \leftarrow R_q^k$ (but, it is kept secret) and small error $\mathbf{e} \in \chi^m$ (it is also kept secret).

**Hardness of Decision Version:** Decision MLWE is called hard if for any PPT distinguisher $\mathcal{D}$ as adversary if there exists a negligible function $\mathrm{negl}(n)$ such that

$$\mathrm{Adv}_{qkmn\chi}^{\mathrm{DMLWE}}(\mathcal{D}) := \left|\Pr\left[\mathcal{D}\left(\chi, \mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e}\right) = 1\right] - \Pr\left[\mathcal{D}\left(\chi, \mathbf{A}, \mathbf{u}\right) = 1\right]\right| < \mathrm{negl}(n),$$

for given $\mathbf{A} \leftarrow R_q^{m\times k}$ and $\mathbf{s} \leftarrow R_q^k$ (but, it is kept secret) and small error $\mathbf{e} \in \chi^m$ (it is also kept secret).

# Milestone 2

# Lattice-based digital signature schemes

Lattices are used in cryptography to design public-key cryptographic protocols and for cryptanalysis. This report will mainly focus on its constructive applications in cryptography. In 1997, Ajtai and Dwork [4] proposed a public key cryptosystem based on the lattice. They have shown that the scheme would be secure unless the worst-case instance of a variant of SVP (called unique SVP) could be solved in polynomial time. The scheme proposed by Ajtai and Dwork was secure but not very practical. Since solving approximate variants becomes easier as the approximation factor grows, the designer started thinking of basing their cryptographic schemes on approximate variants of hard lattice problems, with an approximation factor decent enough to resist cryptanalysis, and the resulting schemes are efficient enough to be used in practice.

Some constructs of public key encryption and digital signature schemes have now become more practical than the traditional RSA-based digital signature schemes. The Fiat Shamir transform (algorithm) has now become a standard paradigm for constructing digital signature schemes. Many discrete logarithm-based digital signature schemes have been designed using this paradigm. Most of the lattice-based digital signature schemes are based on this paradigm only; for example, a series of schemes proposed in the papers [60, 27, 2, 28]. We will discuss a more recent one, namely the Dilithium-Lattice-based digital signature schemes [28]. It is also one of the finalists in the third round of the NIST post-quantum crypto competition.

## 2.1 Dilithium-Lattice based digital signature schemes

Dilithium is a lattice-based digital signature scheme that is secure against the chosen message attacks, i.e., existentially unforgeable against a chosen message attack (UF-CMA Secure). Unlike the other lattice-based signature schemes [27], Dilithium uses sampling from Uniform distribution instead of the discrete Gaussian distribution as the discrete Gaussian Sampling in the digital signatures are subject to the side channel attacks [23, 37]. In the following, we will first present the Dilithium Signature scheme and then discuss it step by step.

### Notations

We use $s \leftarrow S$ to denote that $s$ is sampled uniformly from the set $S$. If instead $S$ is given as a probability distribution, it means $s$ is sampled form the distribution $S$. For an algorithm $A$, $a \leftarrow A(b)$ denotes $a$ to be the output when the probabilistic algorithm $A$ is given $b$ as an input. Similarly, if $A$ is a deterministic algorithm which, on input $b$, outputs $a$, we write $a := A(b)$. We consider $R$ and $R_q$ to denote the rings $\mathbb{Z}[X]/\langle X^n + 1\rangle$ and $\mathbb{Z}_q[X]/\langle X^n + 1\rangle$ respectively. The

designer have fixed the parameter $n = 256$ and $q = 2^{23} - 2^{13} + 1 = 8380417$. Regular font letters denote elements in $R$ or $R_q$ and bold lower-case letters represent column vectors with coefficients in $R$ or $R_q$. By default, all vectors will be column vectors. Bold upper-case letters are matrices. For a vector $\mathbf{v}$, we denote by $\mathbf{v}^T$ It's transpose.

### Modular Reduction

We use $(\bmod^{\pm} q)$ represent centered reduction modulo $q$. It means if an element $a \equiv a' \mod^{\pm} q$ for an odd integer $q$, $a$ takes an integer value from the range $-\frac{q-1}{2} \leq a \leq \frac{q-1}{2}$. For an even integer $\delta$, if $a \equiv a' \mod^{\pm} \delta$, then $a$ takes values from $-\frac{\delta}{2} < a \leq \frac{\delta}{2}$.

### Size (Norms)

For an element $w \in \mathbb{Z}_q$, we use $\|w\|_\infty$ for $|w \bmod^{\pm} q|$. For $w(X) = \sum_{i=0}^{n-1} w_i X^i \in R$ we define $\ell_2$ and $\ell_\infty$ norms as follows.

$$\|w(X)\|_\infty = \max(\|w_i\|_\infty) \text{ and } \|w(X)\|_2 = \sqrt{\|w_0\|_\infty^2 + \|w_1\|_\infty^2 + \ldots + \|w_{n-1}\|_\infty^2} \tag{2.1}$$

Further for $\mathbf{w} = (w_1, w_2, \ldots w_k) \in R^k$, we define

$$\|\mathbf{w}\|_\infty = \max_i(\|w_i\|_\infty) \text{ and } \|\mathbf{w}\|_2 = \sqrt{\|w_1\|^2 + \|w_2\|^2 + \ldots + \|w_k\|^2}. \tag{2.2}$$

We will write $S_\eta = \{w \in R : \|w\|_\infty < \eta\}$.

### Rounding Algorithms

Rounding algorithms are used in Dilithium to compress public keys and signatures. In the following section, we discuss the rounding algorithms.

### Expanding the matrix A

The function `ExpandA`() construct $\mathbf{A} \in R_q^{k \times \ell}$ using a uniform seed $\rho \in \{0,1\}^{256}$. It expresses $\mathbf{A}$ in the NTT domain representation.

### Sampling the vectors $\mathbf{s}_1$ and $\mathbf{s}_2$

The function `ExpandS`() generates secret vectors in the key generation. It constructs secret vectors $(\mathbf{s}_1, \mathbf{s}_2) \in S_\eta^\ell \times S_\eta^k$ using a seed $\rho'$.

### Sampling the vectors y

The function `ExpandMask`() deterministically generates randomness $\mathbf{y} \in S_{\gamma_1}^\ell$ of the signature scheme using a seed $\rho'$ and a nonce $\kappa$.

### Hashing

The function $H$ will be used in our signature scheme. It is the extended output function (XOF) SHAKE-256 mapping onto the specified domain.

### Hashing to a Ball

Let $B_\tau$ be the set of elements of $R$ that have $\tau$ many coefficients equal to $\pm 1$ and the rest are $0$. Thus $|B_\tau| = 2^\tau \cdot \binom{256}{\tau}$. The Dilithium signature scheme requires a cryptographic hash that hashes onto $B_\tau$ in two steps. In the first step, an element from $\{0,1\}^\star$ is mapped to $\{0,1\}^{256}$, in the second step, XOF (e.g. SHAKE-256) is used to map the output of the first step onto $B_\tau$. A high-level description of an algorithm for creating a random element in $B_\tau$ is given in the Algorithm 2 below. It is referred to as an "inside-out" version of the Fisher-Yates shuffle [55].

**Algorithm 2:** $\mathrm{SampleInBal}(\rho)$

```
Initialise
    c = c_0 c_1 ... c_255 = 000...0
for i := 256 − τ to 255 do
    j ← {0, 1, ..., i}
    s ← {0, 1}
    c_i := c_j
    c_j = (−1)^s
return c
```

### High-order and Low-order bits

An element $r$ of $\mathbb{Z}_q$ can be written into higher-order and lower-order bits in the bitwise fashion. Let $d$ be the number of bits to break into. We can write $r_0 = r \bmod^{\pm} 2^d$ and $r_1 = (r - r_0)/2^d$. We denote it by $\mathrm{Power2Round}_q(r, d)$ algorithm.

**Algorithm 3:** $\mathrm{Power2Round}_q(r, d)$

```
r := r  mod q
r_0 := r mod± 2^d
r_1 := (r − r_0)/2^d
return (r_1, r_0)
```

Another way to break $r \in \mathbb{Z}_q$ is to choose an even divisor $\delta$ of $q-1$ and write $r = r_1 \cdot \delta + r_0$ in the same way as done earlier. The possible choices of $r_1 \cdot \delta$'s are $\{0, 1 \cdot \delta, 2 \cdot \delta, \ldots, q-1\}$. As $(q-1)$ and $0$ differ by $1$, we avoid $q-1$ as a result we may end up having $r_0$ higher by $1$. This is achieved by the procedure $\mathrm{Decompose}_q$ given in the algorithm below. We additionally define two more functions, namely $\mathrm{LowBits}_q(r, \delta)$ and $\mathrm{HighBits}_q(r, \delta)$.

**Algorithm 4:** $\text{Decompose}_q(r, \delta)$

$r := r \mod q$
$r_0 := r \bmod^{\pm} \delta$
**if** $(r - r_0) > 0$ **then**
  $r_1 := 0, r_0 := r_0 - 1$
**else**
  $r_1 := \frac{r - r_0}{\delta}$
return $(r_1, r_0)$

**Algorithm 5:** $\text{HighBits}_q(r, \delta)$

$(r_1, r_0) := \text{Decompose}_q(r, \delta)$
return $r_1$

**Algorithm 6:** $\text{LowBits}_q(r, \delta)$

$(r_1, r_0) := \text{Decompose}_q(r, \delta)$
return $r_0$

This division of HighBits and LowBits of an element of $\mathbb{Z}_q$ is done to reduce the size of the public key. We want to recover higher order bits of $r + z \in \mathbb{Z}_q$ without storing a small $z \in \mathbb{Z}_q$. For this, we use a 1-bit of hint $h$. The lead is generated and used in the following ways.

**Algorithm 7:** $\text{MakeHint}_q(z, r, \delta)$

$r_1 := \text{HighBits}_q(r, \delta)$
$v_1 := \text{HighBits}_q(r+z, \delta)$
return $r_1 \neq v_1$

**Algorithm 8:** $\text{UseHint}_q(h, r, \delta)$

$m := (q - 1)/\delta$
$(r_1, r_0) := \text{Decompose}_q(r, \delta)$
**if** $h = 1$ *and* $r_0 > 0$ **then**
  return $(r_1 + 1) \mod m$
**if** $h = 1$ *and* $r_0 \leq 0$ **then**
  return $(r_1 - 1) \mod m$
return $r_1$

For proof of the correctness of the above algorithms, we refer to the paper [28]. We will require the following lemmas in the security proof of the scheme.

**Lemma 1.** *Suppose that $q$ and $\alpha$ are positive integers satisfying $q > 2\alpha$, $q \equiv 1 (mod\ \alpha)$ and $\alpha$ is even . Let $\boldsymbol{r}$ and $\boldsymbol{z}$ be vectors of elements in $R_q$ where $\|\boldsymbol{z}\|_\infty \leq \alpha/2$ , and let $\boldsymbol{h}$, $\boldsymbol{h}'$ be two vectors of bits. Then The $\text{HighBits}_q$ , $\text{MakeHint}_q$ and $\text{UseHint}_q$ algorithms satisfy the following properties:*

1. $\text{UseHint}_q(\text{MakeHint}_q(\boldsymbol{z}, \boldsymbol{r}, \alpha), \boldsymbol{r}, \alpha) = \text{HighBits}_q(\boldsymbol{r} + \boldsymbol{z}, \alpha)$.
2. *Let $\boldsymbol{v}_1 = \text{UseHint}_q(\boldsymbol{h}, \boldsymbol{r}, \alpha)$, then $\|\boldsymbol{r} - \boldsymbol{v}_1\alpha\|_\infty \leq \alpha + 1$. Furthermore, if the number of 1's in $\boldsymbol{h}$ is $\omega$, then all except at most $\omega$ coefficients of $\boldsymbol{r} - \boldsymbol{v}_1\alpha$ will have magnitude at most $\alpha/2$ after centred reduction modulo $q$.*
3. *For any $\boldsymbol{h}$ and $\boldsymbol{h}'$, if $\text{UseHint}_q(\boldsymbol{h}, \boldsymbol{r}, \alpha) = \text{UseHint}_q(\boldsymbol{h}', \boldsymbol{r}, \alpha)$ , then $\boldsymbol{h} = \boldsymbol{h}'$.*

**Lemma 2.** *If $\|\boldsymbol{s}\|_\infty \leq \beta$ and $\|\text{LowBits}_q(\boldsymbol{r}, \alpha)\|_\infty \leq \alpha/2 - \beta$, then $\text{HighBits}_q(\boldsymbol{r} + \boldsymbol{z}, \alpha)$.*

**Lemma 3.** *Let $(\boldsymbol{r}_1, \boldsymbol{r}_0) = \text{Decompose}_q(\boldsymbol{r}, \alpha)$ , $(\boldsymbol{w}_1, \boldsymbol{w}_0) = \text{Decompose}_q(\boldsymbol{r} + \boldsymbol{s}, \alpha)$ and $\|\boldsymbol{s}\|_\infty \leq \beta$, then $\|\boldsymbol{s} + \boldsymbol{r}_0\|_\infty \leq \alpha/2 - \beta \iff \boldsymbol{w}_1 = \boldsymbol{r}_1 \wedge \|\boldsymbol{w}_0\|_\infty \leq \alpha/2 - \beta$.*

The Dilithium signature scheme is based on the Fiat-Shamir transform with an abort strategy. We will first Recap this framework below.

**ALGORITHM 2.1.1: Fiat Shamir with Aborts Framework**

**Algorithm 9:** $\texttt{GEN}(k, \ell)$

1 $\mathbf{A} \leftarrow R_q^{k \times \ell}$, $\mathbf{s}_1 \leftarrow S^{\ell}$, $\mathbf{s}_2 \leftarrow S^k$
2 $\mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 \pmod q$
3 $pk := (\mathbf{A}.\mathbf{t})$ ; $sk := (\mathbf{s}_1, \mathbf{s}_2)$

**Algorithm 10:** $\texttt{SIGN}(\mu, \rho, sk, pk)$

1 $\mathbf{y}_1 \leftarrow Y^{\ell}$, $\mathbf{y}_2 \leftarrow Y^k$ and $\mathbf{w} := \mathbf{A}\mathbf{y}_1 + \mathbf{y}_2$
2 $c := H(\mathbf{A}, \mathbf{t}, \mathbf{w}, \mu)$, $\mathbf{z}_1 := \mathbf{y}_1 + c\mathbf{s}_1$, $\mathbf{z}_2 := \mathbf{y}_2 + c\mathbf{s}_2$
3 **if** RejectSample$(\boldsymbol{z}_1, \boldsymbol{z}_2, c\boldsymbol{s}_1, c\boldsymbol{s}_2)$ **then**
4 **go to** the step 1.
5 Output $(\mathbf{z}_1, \mathbf{z}_2, c)$

**Algorithm 11:** $\texttt{VRFY}(\mu, (\mathbf{z}_1, \mathbf{z}_2, c), \mathbf{A}, \mathbf{t})$

1 $\mathbf{w} := \mathbf{A}\mathbf{z}_1 + \mathbf{z}_2 - c\mathbf{t} \pmod q$
2 Accept if $c = H(\mathbf{A}, \mathbf{t}, \mathbf{w}, \mu)$ and $\|(\mathbf{z}_1, \mathbf{z}_2)\|$ is small.

The set $S$ and $Y$ are subsets of $\mathbb{R}$ with small coefficients. $H$ is a cryptographic hash function outputting to a low-norm subset of $R$ of size at least $2^{256}$. The Rejection Sample procedure in the signing algorithm makes the signature part $(\mathbf{z}_1, \mathbf{z}_2)$ independent of the secret key.

**ALGORITHM 2.1.2: Dilithium DSA**

**Algorithm 12:** $\texttt{GEN}()$

1 $\xi \in \{0,1\}^{256}$
2 $(\rho, \rho', K) \in \{0,1\}^{256} \times \{0,1\}^{512} \times \{0,1\}^{256} := H(\xi)$ /* $H$ is instantiated as SHAKE-256 */
3 $\mathbf{s}_1, \mathbf{s}_2 \leftarrow S_\eta^l \times S_\eta^k := \text{ExpandS}(\rho')$
4 $\mathbf{A} \in R_q^{k \times l} := \text{ExpandA}(\rho)$ /* $A$ is stored in NTT Domain Representation */
5 $\mathbf{t} := \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$
6 $(\mathbf{t}_1, \mathbf{t}_0) := \text{Power2Round}_q(\mathbf{t}, d)$
7 $tr \in \{0,1\}^{256} := H(\rho \| \mathbf{t}_1)$
8 return $sk := (\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$ and $pk := (\rho, \mathbf{t}_1)$

**Algorithm 13:** $\texttt{SIGN}(sk, M)$

$\mathbf{A} \in R_q^{k \times l} := \text{ExpandA}(\rho)$
$\mu \in \{0,1\}^{512} := H(tr \| M)$
$\kappa = 0$, $(\mathbf{z}, \mathbf{h}) := \perp$
$\rho' \in \{0,1\}^{512} := H(\rho \| \mu)$ (or $\rho' \leftarrow \{0,1\}^{512}$ for randomized signing)
**while** $(\boldsymbol{z}, \boldsymbol{h}) := \perp$ **do**
    $\mathbf{y} \in S_{\gamma_1}^{\ell} := \text{ExpandMask}\,(\rho', \kappa)$
    $\mathbf{w} := \mathbf{Ay}$
    $\mathbf{w}_1 := \text{HighBits}\,(\mathbf{w}, 2\gamma_2)$
    $\tilde{c} = \{0,1\}^{256} := H(\mu, \mathbf{w}_1)$
    $c \in B_\tau := \text{SimpleInBall}(\tilde{c})$
    $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$
    $\mathbf{r}_0 := \text{LowBits}(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$
    **if** $\|\boldsymbol{z}\|_\infty \geq \gamma_1 - \beta$ *or* $\|\boldsymbol{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
        $(\mathbf{z}, \mathbf{h}) := \perp$
    **else**
        $\mathbf{h} := \text{MakeHint}_q(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$
        **if** $\|c\boldsymbol{t}_0\|_\infty \geq \gamma_2$ *or* $\#1$ *in* $\boldsymbol{h}$ *is greater than* $\omega$ **then**
            $(\mathbf{z}, \mathbf{h}) := \perp$
$\kappa := \kappa + \ell$
return $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$

**Algorithm 14:** $\texttt{VRFY}\,(pk, M, \sigma = (\tilde{c}, \mathbf{z}, \mathbf{h}))$

$\mathbf{A} \in R_q^{k \times l} := \text{ExpandA}(\rho)$
$\mu \in \{0,1\}^{512} := H(H(\rho \parallel \mathbf{t}_1) \parallel M)$
$c := \text{SimpleInBall}(\tilde{c})$
$\mathbf{w}_1' = \text{UseHint}_q\left(\mathbf{h}, \mathbf{Az} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2\right)$
return $(\|\mathbf{z}\|_\infty < \gamma_1 - \beta) \wedge (\tilde{c} = H(\mu \parallel \mathbf{w}_1')) \wedge (\text{ no of 1's in } \mathbf{h} \ \leq \omega)$

## Correctness:

Assume that the signature $\sigma := (\tilde{c}, \mathbf{z}, \mathbf{h})$ of a message $M$ is generated successfully by the signer. Then, any verifier must be able to verify the signature $\sigma$ by the signer's public key $pk := (\rho, \mathbf{t}_1)$. Due to not having full $\mathbf{t}$, the Verifier only has the option to compute

$$\begin{aligned} \mathbf{w}' &:= \text{UseHint}_q(\mathbf{h}, \mathbf{Az} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2) \\ &= \text{UseHint}_q(\mathbf{h}, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2). \end{aligned}$$

Since $\|c\mathbf{t}_0\| < \gamma_2$, using Lemma 5 we have

$$\text{UseHint}_q(\mathbf{h}, \mathbf{Az} - c\mathbf{t} + c\mathbf{t}_0, 2\gamma_2) = \text{HighBits}_q(\mathbf{Az} - c\mathbf{t}, 2\gamma_2),$$

Using $\mathbf{Az} - c\mathbf{t} = \mathbf{w} - c\mathbf{s}_2$, we get

$$\text{HighBits}_q(\mathbf{Az} - c\mathbf{t}, 2\gamma_2) = \text{HighBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2).$$

Since the parameter $\beta$ is taken as $\|c\mathbf{s}_2\| < \beta$ and $\mathrm{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) < \gamma_2 - \beta$, by Lemma 2

$$\begin{aligned}\mathrm{HighBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) &= \mathrm{HighBits}_q(\mathbf{w} - c\mathbf{s}_2 + c\mathbf{s}_2, 2\gamma_2)\\ &= \mathrm{HighBits}_q(\mathbf{w}, 2\gamma_2)\\ &= \mathbf{w}_1,\end{aligned}$$

Hence the correctness follows.

## Number of Repetitions:

In this section, we compute the probability that the prover does not send $\perp$. That is, we want to show that the algorithm comes up with a valid signature. More formally, the following lemma follows.

**Lemma 4.** *If $\beta \geq \max_{\boldsymbol{s}\in\boldsymbol{S}_\eta,\ c\in\boldsymbol{B}_\tau} \|c\boldsymbol{s}\|_\infty$ then $pr[(\boldsymbol{z},\boldsymbol{h}) \neq \perp] \approx \exp(-\beta n(k/\gamma_1 + l/\gamma_2))$ in the line 13 of Algorithm 13.*

*Proof.* In line 13 of Algorithm 13, $(\mathbf{z}, \mathbf{h}) \neq \perp$ if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and $\|\mathbf{r}_0 = \mathrm{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)\|_\infty < \gamma_2 - \beta$. And $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ implies that, for each coefficient $\sigma$ of $c\mathbf{s}_1$, the corresponding coefficient of polynomials in vector $\mathbf{z}$ inclusively lies between $-(\gamma_1 - \beta - 1)$ and $(\gamma_1 - \beta - 1)$. The corresponding coefficient of $\mathbf{y}_i$ is between $-\gamma_1 + \beta + 1 - \sigma$ and $\gamma_1 - \beta - 1 - \sigma$. The size of this range is $2(\gamma_1 - \beta) - 1$ and the coefficient of $\mathbf{y}$ have total $2\gamma_1 - 1$ choices. Therefore, the probability that each coefficient of $\mathbf{y}$ lies in the good region is

$$\left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{n.l} = \left(1 - \frac{\beta}{\gamma_1 - 1/2}\right)^{n.l} \approx \exp^{-n.\beta l/\gamma_1}.$$

Now we compute the probability that $\|\mathbf{r}_0 = \mathrm{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)\|_\infty < \gamma_2 - \beta$. If we assume that the low-order bits are uniformly distributed modulo $2\gamma_2$, then the probability that all coefficients are in good range is

$$\left(\frac{2(\gamma_2 - \beta) - 1}{2\gamma_2}\right)^{n.k} \approxeq \exp^{-n.\beta k/\gamma_2}.$$

Thus the probability $\Pr[(\mathbf{z}, \mathbf{h}) \neq \perp] \approx \left(\exp^{-n.\beta l/\gamma_1}\right)\left(\exp^{-n.\beta k/\gamma_2}\right) = \exp\left(-n.\beta(l/\gamma_1 + k/\gamma_2)\right)$ □

### Size of Public Key

The public key is $p_k := (\rho, \mathbf{t}_1)$, where $\rho \leftarrow \{0,1\}^{256}$ and $\mathbf{t}_1$ is high bits of $\mathbf{t}$ is derived by $\mathrm{Power2Round}_q(\mathbf{t}, d)$ method. In this scheme $q = 2^{23} - 2^{13} + 1 \implies q - 1 = 2^{13}(2^{10} - 1)$, which implies that the coefficients of $k$ polynomials of $\mathbf{t}_1$ lie in the set $\{0, 1, \cdots, 2^{10} - 1\}$, because according to NIST recommendation $d = 2^{13}$. So, there are 10 bits for each coefficient. Therefore, the size of $\mathbf{t}_1$ is $k \cdot 256 \cdot 10/8 = 320k$ bytes. Thus, the total size of the public key $p_k$ as a concatenation of the bit-packed representation of $\rho$ and $\mathbf{t}_1$ is $32 + 320k$ bytes.

### Size of Signature

The signature is the tuple $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$. The signature string is the concatenation of the bit-packed representation of $\tilde{c}$, encoding strings of $\mathbf{z}$ and $\mathbf{h}$, respectively. Now, the size of $\tilde{c}$ is 32 bytes. The

hint vector **h** has at most $\omega$ non-zero coefficients. So, it is enough to store the locations of non-zero coefficients. Thus, each of the first $\omega$ bytes of the string of vector **h**is the index $i$ of the next non-zero coefficient in its polynomials, that is $0 \leq i \leq 255$, or zero if there exist no more non-zero coefficients. The number $\omega$ up to $\omega + k - 1$ records the $k$ positions $j$ of the polynomial boundaries in the string of $\omega$ coefficient indices, where $0 \leq j \leq \omega$.

Now, we discuss the bit-packing of **z**. The vector **z** is an $l \times 1$ size matrix with polynomial entries.Assume there are $l$ polynomials, namely $p_1, \cdots, p_l$. Now, assume that $\gamma_1 - c_1, \cdots, \gamma_1 - c_{256}$ be the coefficients of the $p_1$. $\gamma_1 - c_{257}, \cdots, \gamma_1 - c_{512}$ be the coefficients of $p_2$, etc. Each coefficient of **z** uses 20 bits. Thus, the total size of **z** is $32l(1+\log \gamma_1)$bytes. Therefore, the size of the signature $\sigma$ is $32l(1 + \log \gamma_1) + \omega + k + 32$.

### 2.1.1 Security Proof

The Dilithium Signature scheme is the Fiat-Shamir Transformation of an identification scheme. The Fiat-Shamir transformation combines the result of a canonical identification scheme ID and a hash function H to obtain a digital signature scheme $\mathrm{FS} = [\mathrm{ID}, \mathrm{H}]$.

**Security of Fiat-Shamir Signatures in the ROM:**

The security proof in ROM of the signature scheme $\mathrm{FS} = [\mathrm{ID}, \mathrm{H}]$ can be proved in two ways in two steps.

1. Firstly, if the underlying identification scheme ID has statistical Honest Verifier Zero Knowledge (HVZK) property, then unforgeability against Chosen Message Attack (UF-CMA) and Unforgeability against No Message Attack (UF-NMA) are tightly equivalent (UF-NMA security means that the adversary cannot make any signing queries).

2. Secondly, the Forking Lemma [75],[16] (based on a technique called "rewinding") is used to prove UF-NMA security in the random-oracle model (ROM) [17] from computational Special Soundness(SS). The latter part of the security reduction is non-tight, and the loss in tightness is known to be inherent (e.g., [73],[52]).

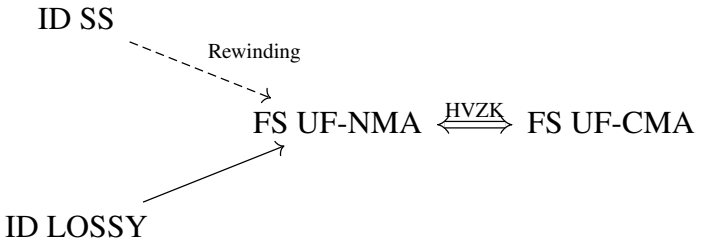



Figure 2.1: Security result of Fiat-Shamir $\mathrm{FS} = [\mathrm{ID}, \mathrm{H}]$ signature in classical ROM.The solid line represents tight reduction, and the dashed line represents non-tight reduction

We notice that the Rewinding technique by forking lemma [75],[16] does not provide tight security reduction. ID LOSSY provides a tight reduction. The informal definition of the Lossy Identification Scheme is given below.

**Lossy Identification Schemes**

To achieve signature schemes with a tight security reduction by generalizing a signature scheme by Katz and Wang [79], AFLT [1] introduced the new concept of lossy identification schemes and proved that Fiat-Shamir transformed signatures have a tight security reduction in the classical ROM. A lossy identification scheme has an additional key generator that produces a lossy public key, which is computationally indistinguishable from an honestly generated public key. Further, relative to a lossy public key, the identification scheme has *statistical soundness*, i.e., not even an unbounded adversary can successfully impersonate a prover. Figure 2.1 summarizes the security results of Fiat-Shamir signatures in the classical ROM.

### 2.1.2 Security reduction of Fiat-Shamir Signatures in Quantum Random Oracle Model(QROM)

It is necessary to analyze the security of cryptographic protocols because we are at the edge of the transition to an era of superior computation power called Quantum Computation. Assume that the Adversary has a quantum computer that provides an extra edge in queries to model with exponential computational power. Quantum computers may execute all "offline primitives", such as the hash function on arbitrary superpositions, which motivated the introduction of the quantum (accessible) random-oracle model (QROM) [20]. In the UF-CMA security experiment for signatures in the QROM, an adversary has quantum access to a perfect hash function $H$ and classical access to the signing oracle. Security Reduction of the Fiat-Shamir Signature $\mathrm{FS} = [\mathrm{ID}, \mathrm{H}]$ in QROM is the main motivation in our study.

**Quantum oracles and quantum Adversaries**

A classical oracle function $\mathrm{O} : \{0,1\}^n \longrightarrow \{0,1\}^m$, that we follow the standard approach as in [14, 20] to execution of the classical function **O**. The Model Quantum Access to **O** is defined as $\mathrm{U}_O : |x\rangle\,|y\rangle \longrightarrow |x\rangle\,|y \bigoplus \mathrm{O}(x)\rangle$, where $x \in \{0,1\}^n$ and $y \in \{0,1\}^m$.

A Quantum adversary denoted by $\mathcal{A}^{|\mathrm{O}\rangle}$ has access O in superposition by applying $\mathrm{U}_O$. The quantum time it takes to apply $\mathrm{U}_O$ is linear in the time it takes to evaluate O classically. We write $\mathcal{A}^{|\mathrm{O}\rangle}$ to represent that an oracle is quantum-accessible, contrary to oracles which can only be accessed classically, which are denoted by $\mathcal{A}^{\mathrm{O}}$. Sometimes, we abuse notation and use $|\mathrm{O}\rangle$ to denote the quantum accessible oracle.

*Main Security Statement*: The following is our main security statement for $\mathrm{SIG} := \mathrm{FS}[\mathrm{ID}, \mathrm{H}, \kappa_m]$ in the QROM.[51]

**THEOREM 2.1.3**

Assume the identification scheme ID is lossy, $\epsilon_{zk}$-perfect naHVZK has $\alpha$ bits of min-entropy, and is $\epsilon_{ls}$-lossy sound. For any quantum adversary $\mathcal{A}$ against $\mathrm{UF\text{-}CMA}_1$ ($\mathrm{sUF\text{-}CMA}_1$) security that issues at most $Q_H$ queries to the quantum random oracle $|H\rangle$and $Q_S$ classical queries to the signing oracle $\mathrm{SIGN}_1$, there exists a quantum adversary $\mathcal{B}$ (and a quantum adversary $\mathcal{C}$ against CUR) such that

$$\mathbf{Adv}^{\mathrm{UF-CMA}_1}_{\mathrm{SIG}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{LOSS}}_{\mathrm{ID}}(\mathcal{B}) + 8(Q_H+1)^2.\epsilon_{ls} + \kappa_m Q_S.\epsilon_{zk} + 2^{-\alpha+1}$$

$$\mathbf{Adv}^{\mathrm{UF-CMA}_1}_{\mathrm{SIG}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathrm{LOSS}}_{\mathrm{ID}}(\mathcal{B}) + 8(Q_H+1)^2.\epsilon_{ls} + \kappa_m Q_S.\epsilon_{zk} + 2^{-\alpha+1} + \mathbf{Adv}^{\mathrm{CUR}}_{\mathrm{ID}}(\mathcal{C})$$

and

$$\mathrm{Time}(\mathcal{B}) = \mathrm{Time}(\mathcal{C}) = \mathrm{Time}(\mathcal{A}) + \kappa_m + Q_H.$$

## 2.2 Falcon Digital Signature Algorithm

The Falcon digital signature scheme is an acronym for "Fast Fourier lattice-based compact signature over NTRU". It instantiates the signature framework of the paper [43] using NTRU lattices and fast Fourier trapdoor sampling. We will denote this framework as the GPV framework. Falcon is a post-quantum signature algorithm submitted to NIST's Post-Quantum Cryptography project: `https://csrc.nist.gov/Projects/Post-Quantum-Cryptography`. In the following section, we will discuss the prerequisites for explaining the workings of FALCON digital signature schemes.

### 2.2.1 Notations and Prerequisites

Let $\phi(X) = X^n + 1$ where $n$ is 512 or 1024 depending on the security levels we want to set up. Let $R = \frac{\mathbb{Z}[X]}{\langle \phi(X) \rangle}$, the elements of $R$ are represented as polynomials $f(X) = \sum_{i=0}^{n-1} f_i X^i$ of degree at most $n-1$ or as a vector $\mathbf{f} = (f_0, f_1, \ldots, f_{n-1})$. Let $g \in R$, the linear map $M_g : R \to R$ is defined by multiplication of polynomial $g$ i.e., $M_g(f) = f \cdot g$. The Fourier Transform of $f$ is denoted by $\hat{f}$. We denote the coefficient vector of $\hat{f}$ by $\hat{\mathbf{f}}$. We denote a lattice generated by the basis $\mathbf{B} = (\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{m-1})$ by $\Lambda(\mathbf{B})$ and the corresponding orthogonal lattice $\Lambda^{\perp}(\mathbf{B}) = \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{x}\mathbf{B}^t = 0\}$, where $\mathbf{B}^t$ is the transpose of matrix $\mathbf{B}$. We define a $q$-ary lattice $\Lambda_q(B) = \{\mathbf{y} \in \mathbb{Z}^n \mid \exists \mathbf{x} \in \mathbb{Z}^n : \mathbf{y} = \mathbf{B}\mathbf{x} \pmod q\}$ and its orthogonal $q$-ary lattice $\Lambda^{\perp}(B)_q = \{\mathbf{x} \in \mathbb{Z}^n : \mathbf{x}\mathbf{B}^t = 0 \pmod q\}$.

Let $a = \sum_{i=0}^{n-1} a_i X^i$ and $b = \sum_{i=0}^{n-1} b_i X^i$ be the elements of $R = \mathbb{Q}[X]/\langle \phi(X) \rangle$. Denote by $a^\star$ the Hermitian adjoint of $a$, which is the unique element of $R$ such that for any root $\zeta$ of $\phi$ $a^\star(\zeta) = \overline{a(\zeta)}$, overline representing the complex conjugation. For $\phi = X^n + 1$, the Hermitian adjoint $a^\star$ can be expressed as:

$$a^\star = a_0 - \sum_{i=1}^{n-1} a_i X^i \tag{2.3}$$

This definition can be extended to vectors and matrices, the adjoint $\mathbf{B}^\star$ of the matrix $\mathbf{B} \in R^{n \times m}$ is the component-wise adjoint of $\mathbf{B}^t$. The norm and the inner product can be defined naturally on $R$ as follows.

$$\langle a, b \rangle = \sum_{\phi(\zeta)=0} a(\zeta) \cdot \overline{b(\zeta)} \quad \text{and } \|a\| = \sqrt{\langle a, a \rangle} \tag{2.4}$$

We extend this definition to the inner product of the vectors coordinate-wise; for $\mathbf{u} = (u_i)_i$ and $\mathbf{v} = (v_i)_i$, $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i \langle u_i, v_i \rangle$. For the choice of $\phi$, this inner product coincides with the usual inner product $\sum_i u_i v_i$.

If the polynomials are in the FFT representation, the Equation 2.4 is used for inner product/norm computation. If they are in the coefficient representation, the usual norm is used.

For $0 < \sigma \in \mathbb{R}$, the Gaussian function on $\mathbb{R}^n$, with center at $\mathbf{c}$ and standard deviation $\sigma$ is denoted by $\rho_{\sigma,\mathbf{c}}(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}\|}{2\sigma^2}\right)$.

The discrete Gaussian distribution over $\Lambda$ of center $\mathbf{c}$ and standard deviation $\sigma$ is denoted by $D_{\Lambda,\sigma,\mathbf{c}}(\mathbf{z}) = \rho_{\sigma,\mathbf{c}}(\mathbf{z}) / \sum_{\mathbf{x} \in \Lambda} \rho_{\sigma,\mathbf{c}}(\mathbf{x})$.

### Gram-Schmidt orthogonalization (GSO)

A matrix $\mathbf{B}$ can be decomposed (see Algorithm 15) as follows:

$$\mathbf{B} = \mathbf{L} \cdot \tilde{\mathbf{B}}, \tag{2.5}$$

where $\mathbf{L}$ is a lower triangular matrix with 1's on the diagonals and rows $\tilde{\mathbf{b}_i}$ of $\tilde{\mathbf{B}}$ satisfy $\langle \mathbf{b}_i, \mathbf{b}_j \rangle = 0$ for $i \neq j$. For a full-rank lattice, this decomposition is unique and is called Gram-Schmidt orthogonalization. We define the Gram-Schmidt norm of $\mathbf{B}$ by the following.

$$\|\mathbf{B}\|_{\mathtt{GS}} = \max \|\tilde{\mathbf{b}_i}\|. \tag{2.6}$$

**Algorithm 15:** $\mathrm{GramSchmidt}_{\mathcal{R}}(\mathbf{B})$

**Input:** $\mathbf{B} = (\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{m-1})$
**Output:** $\tilde{\mathbf{B}} = \left(\tilde{\mathbf{b}}_0, \tilde{\mathbf{b}}_1, \ldots, \tilde{\mathbf{b}}_{m-1}\right)$ and $\mathbf{L} = (L_{i,j})$

```
for i = 0 to m − 1 do
    b̃_i = b_i
    for j = 0 to i − 1 do
        L_{i,j} = ⟨b_i, b̃_j⟩ / ⟨b̃_j, b̃_j⟩
        b_i = b_i − L_{i,j} b̃_j
    end
end
return B̃ = (b̃_0, b̃_1, ..., b̃_{m−1}),  L = (L_{i,j})_{0≤i,j≤m−1}
```

Here, we give you another version of **Babai's Nearest Plane Algorithm**. Here, if we have lattice basis $\mathbf{B}$ and a target vector $\mathbf{c}$, then all the calculations will be done on $\mathbf{t} \leftarrow \mathbf{cB^{-1}}$.

**Algorithm 16:** $\mathrm{NearestPlane}_{\mathcal{R}}(\mathbf{B}, \mathbf{L}, \mathbf{c})$

**Input:** $\mathbf{B} = \mathbf{L}.\tilde{\mathbf{B}}$ over $\mathcal{R}$, a vector $\mathbf{c} \in \mathrm{Span}_{\mathcal{R}}$ **(B)**
**Output:** A vector $\mathbf{v} \in \mathcal{L}$**(B)** such that $\mathbf{c} - \mathbf{v} \in \mathcal{P}(\tilde{\mathbf{B}})$

```
t ← c.B^{-1}
for j=n to 1 do
    t'_j ← t_j + Σ_{i>j} (t_i − z_i) L_{ij}
    z_j ← ⌊t'_j⌉
return v ← z.B
```

### $\mathbf{LDL}^\star$ Decomposition

In the $\mathbf{LDL}^\star$-Decomposition, we express a full rank matrix as a product $\mathbf{LDL}^\star$, where $\mathbf{L} \in R$ is a lower triangular and with 1's on the diagonal and $\mathbf{D} \in R^{n \times n}$ is the diagonal matrix. $\mathbf{LDL}^\star$-Decomposition and GSO are closely related as

$$\left[\mathbf{L} \cdot \tilde{\mathbf{B}} \text{ is GSO of } \mathbf{B}\right] \iff \left[\mathbf{L} \cdot (\tilde{\mathbf{B}}\tilde{\mathbf{B}}^\star) \cdot \mathbf{L}^\star \text{ is the } \mathbf{LDL}^\star \text{ Decomposition of } (\mathbf{BB}^\star)\right]. \tag{2.7}$$

The Algorithm 23 can be used to compute the $\mathbf{LDL}^\star$-decomposition of a $2 \times 2$ matrix.

### Cyclotomic Polynomial

The $n^{\text{th}}$ cyclotomic polynomial, $\Phi_n(X) \in \mathbb{Q}[X]$ is the polynomial whose roots are the primitive $n^{\text{th}}$ roots of unity over $\mathbb{Q}$.

$$\Phi_n(X) = \prod_{\zeta \text{ primitive root } \in \mu_n} (X - \zeta) = \prod_{1 \le a < n \text{ s.t. } (a,n)=1} (X - \zeta_n^a), \tag{2.8}$$

where $\mu_n \subset \mathbb{C}$ is the set of roots of unity and $\zeta_n = \exp(2\pi i/n)$. The roots of the polynomial $X^n - 1$ are precisely the $n^{\text{th}}$ roots of unity, so we have the factorization

$$X^n - 1 = \prod_{\zeta^n = 1} (X - \zeta). \tag{2.9}$$

If we group together the factor $(X - \zeta)$ where $\zeta$ is an element of order $d$ in $\mu_n$, we obtain

$$X^n - 1 = \prod_{d|n} \left( \prod_{\zeta \in \mu_d, \zeta \text{ primitive}} (X - \zeta) \right) \tag{2.10}$$

$$X^n - 1 = \prod_{d|n} \Phi_d(X). \tag{2.11}$$

For some small values of $n$, the polynomials are

$$\begin{aligned}
\Phi_1(X) &= X - 1 \\
\Phi_2(X) &= X + 1 \\
\Phi_3(X) &= X^2 + X + 1 \\
\Phi_4(X) &= X^2 + 1 \\
\Phi_5(X) &= X^4 + X^3 + X^2 + X + 1 \\
\Phi_6(X) &= X^2 - X + 1 \\
\Phi_7(X) &= X^6 + X^5 + X^4 + X^3 + X^2 + X + 1 \\
\Phi_8(X) &= X^4 + 1 \\
\Phi_9(X) &= X^6 + X^3 + 1 \\
\Phi_{10}(X) &= X^4 - X^3 + X^2 - X + 1 \\
\Phi_p(X) &= X^{p-1} + X^{p-2} + ... + X + 1, \text{ for any prime } p
\end{aligned}$$

### FFT and NTT

Let $f(X) \in \mathcal{R} := \frac{\mathbb{Q}[X]}{\langle \phi(X) \rangle}$. Denote by $\Omega_\phi$ the set of all (complex) roots of $\phi(X)$. Assume that $\phi(X)$ is monic $\mathbb{Q}$ and has distinct roots over $\mathbb{C}$. Let $\phi(X) = \prod_{\zeta \in \Omega_\phi}(X - \zeta)$. We denote the fast Fourier transform of $f$ with respect to $\phi$ as $\mathrm{FFT}_\phi(f) := (f(\zeta))_{\zeta \in \Omega_\phi}$. If $\phi$ is clear from the context, we can ignore the subscript $\phi$. We use the notation $\hat{f}$ to represent $\mathrm{FFT}(f)$. We note that $\mathrm{FFT}_\phi$ is a ring isomorphism and let $\mathrm{invFFT}_\phi$ be its inverse. The symbol $\odot$ denotes the multiplication in the FFT domain. The FFT and its inverse operation can be easily extended to vectors and matrices in $\mathcal{R}$ by applying these operations component-wise. All basic operations, i.e., additions, subtractions, multiplications, and divisions of polynomials in $R$ can be done in FFT representation coordinatewise, and this makes multiplications and divisions very efficient.

For $\phi = X^n + 1$, the set of complex roots $\zeta$ of $\phi$ is : $\Omega_\phi = \{\exp(\frac{i(2k+1)\pi}{n}) : 0 \leq k \leq n\}$.

The NTT (Number Theoretic Transform) is analogous to FFT in the field $\mathbb{Z}_p$, where $p$ is a prime equal to $1 \pmod{2n}$. Thus $\phi$ has $n$ roots $\{\omega_i\}$ over $\mathbb{Z}_p$ and any polynomial $f$ can be represented by $(f(\omega_i))$. Even in NTT representation, additions, subtractions, multiplications, and divisions of polynomials (modulo $\phi$ and $p$) can be performed coordinate-wise in $\mathbb{Z}_p$.

In Falcon, NTT is used in faster implementation of public key operations through $\mathbb{Z}_q$, for some prime $q$, and key pair generation. Nevertheless, the secret key implementations use fast Fourier sampling, which uses FFT.

### 2.2.2 GPV Framework

Here, we provide a very high-level description of the framework proposed by Gentry, Peikert, and Vaikuntanathan in [43] for constructing a lattice-based signature. The public key is a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, with $n > m$, which generate the lattice $\Lambda = \mathcal{L}(A)$ and the secret key is a matrix $\mathbf{B} \in \mathbb{Z}_q^{m \times m}$ generating the lattice $\Lambda_q^\perp = \mathcal{L}(B)$. The lattice $\Lambda_q^\perp$ is orthogonal to the lattice $\Lambda$ modulo $q$ i.e., for for any $\mathbf{x} \in \Lambda$ and $\mathbf{y} \in \Lambda_q^\perp$ we have $\langle \mathbf{x}, \mathbf{y} \rangle = 0 \pmod{q}$. Furthermore $\mathbf{B}\mathbf{A}^t = \mathbf{0}$.

A signature for a given message $\mathtt{msg} \in \{0,1\}^\star$ is a short vector $\mathbf{s} \in \mathbb{Z}_q^m$ such that $\mathbf{s}\mathbf{A}^t = H(\mathtt{msg})$, for some hash function $H : \{0,1\}^\star \to \mathbb{Z}_q^n$. Given the public-key $\mathbf{A}$, it is straightforward to verify that $\mathbf{s}$ is a valid signature of the message $\mu$; one requires to check if $\mathbf{s}$ is indeed short and $\mathbf{s}\mathbf{A}^t = H(\mathtt{msg})$. However, generating a signature is not that straightforward. A preimage $\mathbf{c}_0 \in \mathbb{Z}_q^m$ such that $\mathbf{c}_0\mathbf{A}^t = H(\mu)$ is first computed via standard linear algebra. $\mathbf{B}$ is then used to compute a vector $\mathbf{v} \in \Lambda_q^\perp$ close to $\mathbf{c}_0$. Finally the difference $\mathbf{s} = \mathbf{c}_0 - \mathbf{v}$ is reported as a valid signature which satisfies $\mathbf{s}\mathbf{A}^t = \mathbf{c}_0\mathbf{A}^t - \mathbf{v}\mathbf{A}^t = \mathbf{c} - \mathbf{0} = H(\mu)$ and $\mathbf{s}$ is short as $\mathbf{c}_0$ and $\mathbf{v}$ are close.

The GPV framework has been proven secure in the (quantum) random oracle model, assuming the hardness of the SIS problem. Note that the similar framework has already been used in GGH [44] and NTRUsign [45] signature schemes that were broken badly. However, the GPV framework is provably secure in the (quantum) random oracle model if we assume SIS to be hard. This security is due to how $\mathbf{v}$ is computed in the signing procedure of the GPV framework. The $\mathbf{v}$ is computed in the GGH and NTRUSign using the rounding off The algorithm proposed by the Babai [12, 11] first expresses $\mathbf{c}_0$ as a real linear combination of the rows of $\mathbf{B}$ and these real coefficients are rounded to the nearest integer and the resulting coefficient vector is then multiplied by $\mathbf{B}$ again. In the end the $\mathbf{s} = \mathbf{v} - \mathbf{c}_0$ lies in the parallelepiped $[-1, 1] \times \mathbf{B}$ which leaks information about $\mathbf{B}$. This is exploited in many key recovery attacks [70, 30].

#### Computation of v in the GPV Framework

In the computation of $\mathbf{v}$, the GPV framework relies on a randomized variant of Babai's nearest plane algorithm, proposed by Klein [53]. However, Babai's nearest plane algorithm also leaks information about $\mathbf{B}$ like the rounding-off algorithm. Thanks to the Klein randomized variant, which is shown to leak no information about $\mathbf{B}$, and hence the GPV framework is secure.

#### Hash Randomisation

The GPV framework becomes insecure if we publish two different signatures of the same hash $H(\mathtt{msg})$, as discussed in the Section 6.1 of the paper [43]. This problem is solved by prepending a salt $r \in \{0,1\}^k$ to the message before hashing it. When $k$ is large enough, it prevents hash

collisions and guarantees the resulting scheme's security. Falcon uses hash randomization and considers a salt $r \in \{0,1\}^{320}$ for prepending to the message before hashing it.

### 2.2.3 NTRU Lattices

Falcon bases its design on the class of NTRU lattices introduced by Hoffstein, Pipher and Silverman [46]. These lattices come with an additional ring structure, which reduces the size of the public keys and speeds up several other computations. On the other hand, there is no security concern if we are using NTRU lattices in the way suggested by Stehlé and Steinfeld in their paper [86] and Falcon, adopt this idea.

Consider $\phi = X^n + 1$, $n = 2^\kappa$ and $q \in \mathbb{Z}^+$. A set of four polynomials $f, g, F, G \in \frac{\mathbb{Z}[X]}{\langle \phi(X) \rangle}$ satisfying $fG - gF = q \pmod{\phi}$ and $f$ is invertible modulo $q$, is termed as NTRU secret polynomials. A public polynomial $h$ is defined as $h := gf^{-1} \mod (q, \phi)$. Define two matrices $A$ and $B$ as follows.

$$\mathbf{A} = \left[\begin{array}{c|c} 1 & h \\ \hline 0 & q \end{array}\right] \qquad \mathbf{B} = \left[\begin{array}{c|c} f & g \\ \hline F & G \end{array}\right] \tag{2.12}$$

Note that these two matrices generate the same lattice. However $\mathbf{A}$ contains the large polynomials ($h$ and $q$) and $\mathbf{B}$ contains small polynomials. It has been shown in [86] that if $f, g$ are generated with enough entropy, then $h$ will look pseudorandom. In practice, it remains hard to find out small polynomials $f', g'$ such that $h = g'(f')^{-1} \mod (q, \phi)$, even when $f, g$ are quite small. Computing secret polynomials from the public key $h$ corresponds to breaking the NTRU problem; hence, this is termed the NTRU assumption.

### 2.2.4 Instantiation with GPV framework in FALCON

We now instantiate the GPV framework described in Section 2.2.2 over the NTRU lattice. The public basis is

$$\mathbf{A} = \left[\begin{array}{c|c} 1 & h^* \end{array}\right] \tag{2.13}$$

and the secret basis is

$$\mathbf{B} = \left[\begin{array}{c|c} g & -f \\ \hline G & -F \end{array}\right]. \tag{2.14}$$

One can check that the matrices $\mathbf{A}$ and $\mathbf{B}$ are indeed orthogonal: $\mathbf{B} \times \mathbf{A}^* = 0 \mod q$. The signature of a message $m$ consists of a salt $r$ plus a pair of polynomials $(s_1, s_2)$ such that $s_1 + s_2 \times h = H(r||m)$. The value of $s_1$ is completely determined by $m, r, s_2$, so there is no need to send it. The signature of the message $m$ is reported as $(r, s_2)$. The core of the Falcon digital signature scheme is the way $(s_1, s_2)$ is generated. This is done through the ffSampling Algorithm 27. Since the complete process involves the manipulation of polynomials, **FFT** representation of the polynomial is used. In the FFT representation, the (convolution) multiplication of two degrees $(n-1)$ polynomials can be done in $O(n \log(n))$ multiplications of complex numbers, in contrast to the usual $O(n^2 \log(n))$ real multiplications. As explained earlier, given a polynomial $f(X) \in \frac{\mathbb{R}[X]}{\phi(X)}$, its FFT representation is a sequence of $n$ complex numbers obtained by evaluating the polynomial at the $n$ complex roots of $\phi(X)$. The FFT representation of a polynomial can be obtained in $O(n \log n)$ multiplications using the divide and conquer techniques. For the mathematical details of FFT, we refer to the book [26].

In a version of Falcon, $\phi(X) = X^n + 1$, where $n = 2^k$ for some $k$, this gives us a tower of field extensions

$$\mathbb{Q}[X]/(X^n + 1) \supset \mathbb{Q}[X]/(X^{n/2} + 1) \supset \ldots \supset \mathbb{Q}[X]/(X^2 + 1) \supset \mathbb{Q}.$$

A polynomial $f(X) = \sum_{i=0}^{n-1} a_i X^i \in \mathbb{Q}[X]/(X^n + 1)$ be can be decomposed uniquely as $f(X) = f_e(X^2) + X\, f_o(X^2)$, with $f_e, f_o \in \mathbb{Q}[X]/(X^{n/2} + 1)$. In coefficient representation, such a decomposition is straightforward to write:

$$f_e = \sum_{0 \le i < n/2} a_{2i} X^i \text{ and } f_o = \sum_{0 \le i < n/2} a_{2i+1} X^i \tag{2.15}$$

In the Eq.(2.15), we split $f$ with respect to its coefficients of even or odd powers of $X$, and we denote it by

$$\text{split}(f) = (f_e, f_o). \tag{2.16}$$

Similarly, given $f_e(X)$ and $f_o(X)$ in $\mathbb{Q}[X]/(X^{n/2} + 1)$, we can merge them to form an element $f(X) \in \mathbb{Q}[X]/(X^n + 1)$, using the following merge operation given in the Equation 2.17.

$$\text{merge}(f_e, f_o) = f_e(X^2) + X f_o(X^2) \in \mathbb{Q}[X]/(X^n + 1). \tag{2.17}$$

This splitting and merging can be done efficiently even in the FFT representation $\text{FFT}(f)$ of the polynomial $f(X)$ using the Algorithm 17 and Algorithm 18 respectively.

**Algorithm 17:** splitfft($\text{FFT}(f)$)

**Input:** $\text{FFT}(f) = (f(\zeta) : \zeta^n + 1 = 0)$ for some $f \in \mathbb{Q}[X]/(X^n + 1)$
**Output:** $(\text{FFT}(f_o), \text{FFT}(f_o))$.

**for** $\zeta$ *such that* $\zeta^n + 1 = 0$ *and* $\text{Im}(\zeta) > 0$ **do**
  $\zeta' \leftarrow \zeta^2$
  $f_e(\zeta') \leftarrow \frac{1}{2}[f(\zeta) + f(-\zeta)]$
  $f_o(\zeta') \leftarrow \frac{1}{2\zeta}[f(\zeta) - f(-\zeta)]$
return $(\text{FFT}(f_e), \text{FFT}(f_o))$

**Algorithm 18:** mergefft($f_e, f_o$)

**Input:** $\text{FFT}(f_o) = (f_o(\zeta') : \zeta'^{(n/2)} + 1 = 0)$ and $\text{FFT}(f_e) = (f_e(\zeta') : \zeta'^{(n/2)} + 1 = 0)$ for some $f_e, f_o \in \mathbb{Q}[X]/(X^{(n/2)} + 1)$.
**Output:** $\text{FFT}(f)$, where $f = f_e(X^2) + X f_o(X^2) \in \mathbb{Q}[x]/(X^n + 1)$.

**for** $\zeta$ *such that* $\zeta^n + 1 = 0$ **do**
  $\zeta' \leftarrow \zeta^2$
  $f(\zeta) \leftarrow f_e(\zeta') + \zeta f_o(\zeta')$
return $\text{FFT}(f) := (f(\zeta) : \zeta^n + 1 = 0)$

Given a polynomial $f(X) \in \mathbb{Q}[X]/(X^n + 1)$ in the coefficient representation, we can write it into FFT representation using split and mergefft subroutines as per the Algorithm 19.

On the other hand, the splitting of polynomials is also used in computing the norm of a polynomial in the extension $\mathbb{L} := \mathbb{Q}[X]/(X^n + 1) \supset \mathbb{K} := \mathbb{Q}[Y]/(Y^{n/2} + 1)$. Note that we have used

**Algorithm 19:** $\mathrm{FFT}(f)$

**Input:** $f(X) = \sum_{i=0}^{n-1} a_i X^i$
**Output:** $\mathrm{FFT}(f)$

**if** $n > 2$ **then**
  $f_e, f_o = \mathrm{split}(f)$
  $\mathrm{FFT}(f) = \mathrm{mergefft}(\mathrm{FFT}(f_e), \mathrm{FFT}(f_o))$.
**else if** $(n == 2)$ **then**
  $\mathrm{FFT}(f) = (a_0 + j\, a_1, a_0 - j\, a_1)$
return $\mathrm{FFT}(f)$

different indeterminates for clarity for the two extensions. The inclusion is given by the mapping $Y \mapsto X^2$. The norm of $f(X)$ i.e., $N_{\mathbb{L}/\mathbb{K}}(f)$ is the determinant of the $\mathbb{K}$-linear map $\psi_f$ sending $g \to fg$. Observe that $\{1, X\}$ is a basis of the vector space $\mathbb{L}(\mathbb{K})$. So the matrix representation (written row-wise) of $\psi_f$ with respect to the basis $\{1, X\}$ is

$$\mathbf{A} = \begin{bmatrix} f_e(Y) & f_o(Y) \\ Y f_o(Y) & f_e(Y) \end{bmatrix},$$

where $\psi_f(1) = f = f_e(Y) + X f_o(Y)$ and $\psi_f(X) = X f_e(Y) + X^2 f_o(Y) = X f_e(Y) + Y f_o(Y)$ and so $\det(A) = f_e^2(Y) - Y f_o^2(Y)$. The concept of field norm will be used in solving the NTRU equation for the polynomial $F$ and $G$ and this is achieved using the algorithm $\mathrm{NTRUSolve}$ presented in the Algorithm 20.

**Algorithm 20:** $\mathbf{NTRUSolve}_{n,q}(f, g)$ [77]

**Input:** $f, g \in \mathbb{Z}[X]/\langle X^n + 1\rangle$ with $n$, a power of 2.
**Output:** The polynomials $F, G$ satisfying NTRU equations.

**if** $n = 1$ **then**
  Compute $u, v$ such that $uf + vg = \gcd(f, g) = d$ **if** $d \nmid q$ **then**
    Abort and return $\perp$
  **else**
    $(F, G) \leftarrow (-\frac{vq}{d}, \frac{uq}{d})$
  return $(F, G)$
**else**
  $f' = N(f)$ // $N$ is norm in field extension
  $g' = N(f)$
  $(F', G') \leftarrow \mathbf{NTRUSolve}_{n/2,q}(f', g')$
  $F \leftarrow F'(x^2) g(-x)$
  $G \leftarrow G'(x^2) f(-x)$
  $\mathbf{Reduce}(f, g, F, G)$ // Refer to Algorithm 21
return $(F, G)$

**Algorithm 21:** $\mathbf{Reduce}(f, g, F, G)$

**Input:** Polynomials $f, g, F, G \in \mathbb{Z}[X]/\langle X^n + 1\rangle$
**Output:** $(F, G)$ is reduced with respect to $(f, g)$.

**repeat**
  $k \leftarrow \left\lfloor \frac{Ff^\star + Gg^\star}{ff^\star + gg^\star} \right\rceil$
  $F \leftarrow F - kf$
  $G \leftarrow G - kg$
**until** $k \neq 0$

### 2.2.5 Falcon DSA

#### Public Parameters

- The cyclotomic polynomial $\phi = X^n + 1$, where $n = 2^k$ is a power of 2. Note that $\phi$ is monic and irreducible.
- A modulus $q = 12289 \in \mathbb{N}$. We note that $\phi \pmod q$ splits over $\mathbb{Z}_q[X]$.
- A real bound $\lfloor \beta^2 \rfloor > 0$.
- Standard deviations $\sigma$ and $\sigma_{min} \leq \sigma_{max}$.
- A signature byte length `sbytelen`.

#### Key generation

The secret key is generated by first choosing $f, g \leftarrow \mathcal{R}$ randomly with small coefficients satisfying the following properties:

- $f$ should be invertible modulo $q$. $f$ is invertible modulo $q$ *iff* $\mathbf{NTT}(f)$ contains no zero.
- $\gamma = \max\left\{\|(g, -f)\|, \left\|\left(\frac{qf^\star}{ff^\star + gg^\star}, \frac{qg^\star}{ff^\star + gg^\star}\right)\right\|\right\} \leq 1.17\sqrt{q}$. This does ensure the generation of short signatures.

Secondly, the two short polynomials $F$ and $G$ are computed such that

$$fG - gF = q \pmod{(X^n + 1)}. \tag{2.18}$$

Recall that Equation(2.18) is the NTRU equation, and we can solve this for $F(X)$ and $G(X)$ using the Algorithm 20. The Algorithm 22 is used along with the Algorithm 20 to generate the NTRU secret polynomials $f, g, F$ and $G$.

**Algorithm 22:** NTRUGen$(\phi, q)$

**Input:** A monic polynomial $\phi \in \mathbb{Z}[X]$ of degree $n$, a modulus $q$.
**Output:** The polynomials $f, g, F, G$.

$\sigma_{\{f,g\}} \leftarrow 1.17\sqrt{q/2n}$ //
**for** $i \leftarrow 0$ **to** $n-1$ **do**
  $f_i \leftarrow D_{\mathbb{Z},\sigma_{\{f,g\}},0}$
  $g_i \leftarrow D_{\mathbb{Z},\sigma_{\{f,g\}},0}$
$f \leftarrow \sum f_i X^i$, $g \leftarrow \sum g_i X^i$ // $f, g \in \mathbb{Z}(X)/\langle\phi\rangle$
**if** $\mathbf{NTT}(f)$ *contains* $0$ *as a coefficient* **then**
  restart // Check that $f$ is invertible modulo $q$
$\gamma \leftarrow \max\left\{\|(g,-f)\|, \left\|\left(\frac{qf^\star}{ff^\star+gg^\star}, \frac{qg^\star}{ff^\star+gg^\star}\right)\right\|\right\}$
**if** $\gamma > 1.17\sqrt{q}$ **then**
  restart
$F, G \leftarrow \mathbf{NTRUSolve}_{n,q}(f, g)$ // Refer to Algorithm 20
**if** $(F, G) = \perp$ **then**
  restart
return $(f, g, F, G)$

A way to sample $z \leftarrow D_{\mathbb{Z},\sigma_{\{f,g\}},0}$ is to compute

$$z = \sum_{i=0}^{4096/n} z_i \text{ where } \begin{cases} z_i \leftarrow \mathbf{SamplerZ}(0, \sigma^\star), \\ \sigma^\star = 1.17 \cdot \sqrt{\frac{q}{8198}} \approx 1.43300980528773, \end{cases} \tag{2.19}$$

where **SamplerZ** is an algorithm for doing secure Gaussian sampling $z \leftarrow D_{\mathbb{Z},\sigma',0}$ for any $\sigma'$ in $R$. For more details on **SamplerZ**, we refer to the paper [40].

The polynomials $f, g, F, G$ constitute the secret key, but but they are not stored in this form; instead, they are saved in the FFT representation (see Equation 2.20) for efficiency.

$$\hat{\mathbf{B}} = \left[\begin{array}{c|c} \mathrm{FFT}(g) & -\mathrm{FFT}(f) \\ \hline \mathrm{FFT}(G) & -\mathrm{FFT}(F) \end{array}\right]. \tag{2.20}$$

In the GPV framework, generating a digital signature requires solving approximate closest vector problem in the lattice generated by $\hat{\mathbf{B}}$. We can do it efficiently and securely, thanks to the the randomized variant of Babai's nearest plain algorithm. The Babai algorithm requires the unit lower triangular matrix $\mathbf{L}$ in the GSO decomposition of $\hat{\mathbf{B}}$. As we know, the LDL decomposition of $G := \hat{\mathbf{B}} \times \hat{\mathbf{B}}^\star$ contains the same lower triangular matrix $\mathbf{L}$, we do the LDL decomposition of $G$ in the FFT domain and save the matrix $L$ in the form of tree $\mathbf{T}$, called the Falcon tree, along with the secret key $\hat{\mathbf{B}}$ to facilitates the faster signature generation. The Falcon tree is generated using the Algorithm 24 and final keygen algorithm is presented in the Algorithm 25.

**Algorithm 23:** $\mathrm{LDL}^*(G)$

**Input:** A full-rank self-adjoint matrix $G = (G_{ij}) \in \mathrm{FFT}(\mathbb{Q}[x]/(\phi))^{2\times 2}$.
**Output:** The $\mathrm{LDL}^*$ decomposition $G = \mathrm{LDL}^*$ over $\mathrm{FFT}(\mathbb{Q}[x]/(\phi))$.

$D_{00} \leftarrow G_{00}$
$L_{10} \leftarrow G_{10}/G_{00}$
$D_{11} \leftarrow G_{11} - L_{10} \odot L_{10}^* \odot G_{00}$
$\mathbf{L} \leftarrow \left[\begin{array}{c|c} 1 & 0 \\ \hline L_{10} & 1 \end{array}\right]$
$\mathbf{D} \leftarrow \left[\begin{array}{c|c} D_{00} & 0 \\ \hline 0 & D_{11} \end{array}\right]$
return $(\mathbf{L}, \mathbf{D})$

**Algorithm 24:** $\mathrm{ffLDL}^*(G)$ [33]

**Input:** A full-rank Gram matrix $G \in \mathrm{FFT}(\mathbb{Q}[x]/(x^n+1))^{2\times 2}$.
**Output:** A binary tree $T$.

$(L, D) \leftarrow \mathrm{LDL}^*(G)$ // $\mathbf{L} \leftarrow \left[\begin{array}{c|c} 1 & 0 \\ \hline L_{10} & 1 \end{array}\right]$ $\mathbf{D} \leftarrow \left[\begin{array}{c|c} D_{00} & 0 \\ \hline 0 & D_{11} \end{array}\right]$
T.value $\leftarrow L_{10}$
**if** $n = 2$ **then**
  T.leftchild $\leftarrow D_{00}$
  T.rightchild $\leftarrow D_{11}$
  return $T$
**else**
  $d_{00}, d_{01} \leftarrow \mathrm{splitfft}(D_{00})$ // $d_{ij} \in \mathrm{FFT}(\mathbb{Q}[x]/(x^{n/2}+1))$
  $d_{10}, d_{11} \leftarrow \mathrm{splitfft}(D_{11})$
  $\mathbf{G}_0 \leftarrow \left[\begin{array}{c|c} d_{00} & d_{01} \\ \hline d_{01}^* & d_{00} \end{array}\right]$
  $\mathbf{G}_1 \leftarrow \left[\begin{array}{c|c} d_{10} & d_{11} \\ \hline d_{11}^* & d_{10} \end{array}\right]$
  T.leftchild $\leftarrow \mathrm{ffLDL}^*(G_0)$
  T.rightchild $\leftarrow \mathrm{ffLDL}^*(G_1)$ // Recursive calls
  return $T$

**Algorithm 25:** $\mathbf{Keygen}(\phi, q)$

**Input:** A monic polynomial $\phi \in \mathbb{Z}[X]$ and a modulus $q$.
**Output:** A secret key $sk$ and a public key $pk$.

$f, g, F, G \leftarrow \mathrm{NTRUGen}(\phi, q)$ // Refer to the Algorithm 22
$\mathbf{B} \leftarrow \left[\begin{array}{c|c} g & -f \\ \hline G & -F \end{array}\right]$
$\hat{\mathbf{B}} \leftarrow \mathrm{FFT}(\mathbf{B})$
$\mathbf{G} \leftarrow \hat{\mathbf{B}} \times \hat{\mathbf{B}}^{\star}$
$\mathbf{T} \leftarrow \mathrm{ffLDL}^{*}(\mathbf{G})$ **for** *each leaf of* $\boldsymbol{T}$ **do**
  leaf.value $\leftarrow \sigma/\sqrt{\text{leaf.value}}$
$sk \leftarrow (\hat{\mathbf{B}}, \mathbf{T})$
$h \leftarrow gf^{-1} \pmod q$
$pk \leftarrow h$
return $(sk, pk)$

For more details of $\mathbf{LDL}^{\star}$, we refer to the paper [40]

### Signing Algorithm

Recall that, at a very high level, the signature is generated as follows. A hash value $c \in \mathbb{Z}_q[X]/\langle X^n + 1\rangle$ is computed from the message `msg` and a salt $r$. After that, it uses the secret key $f, g, F, G$ to compute two short values $s_1, s_2$ such that $s_1 + s_2 h = c \mod q$.

A simple way to find such a short values $(s_1, s_2)$ is to compute $\mathbf{t} = (\mathbf{c}, \mathbf{0}) \cdot \mathbf{B}^{-1}$ and round coefficient wise to a vector $\mathbf{z} = \lfloor \mathbf{t} \rceil$ and output $(s_1, s_2) \leftarrow (\mathbf{t} - \mathbf{z})\mathbf{B}$. Although it satisfies all the requirements of a legitimate signature, the technique is known to be insecure and leaks the private key. Falcon achieves this through a trapdoor sampler called a Fast Fourier Sampling (FFSampling). The crucial part of the signature algorithm is FFSampling, which uses the Falcon tree and discrete Gaussians over $\mathbb{Z}$. The rest of the signing algorithm is very straightforward.

**Algorithm 26:** Sign(msg,sk,$\lfloor\beta^2\rfloor$)

**Input:** A message **msg**, a secret key sk, a bound $\lfloor\beta^2\rfloor$
**Output:** A signature sig of **msg**

$r \leftarrow \{0,1\}^{320}$ // Random nonce/salt
$c := \mathrm{HashToPoint}(r \| \mathtt{msg}, q, n)$ // coeffs poly $c$ mod $\phi$ are uni mapped to $\mathbb{Z}_q$
$\hat{\mathbf{t}} \leftarrow (-\frac{1}{q} FFT(\mathrm{c}) \odot FFT(\mathrm{F}), \frac{1}{q} FFT(\mathrm{c}) \odot FFT(\mathrm{f}))$ // $\hat{\mathbf{t}} := (\hat{c}, 0)\hat{\mathbf{B}}^{-1}$
**do**
  $\hat{\mathbf{z}} \leftarrow \mathrm{FFSampling}(\hat{\mathbf{t}}, T)$ // trapdoor sampler
  $\hat{\mathbf{s}} \leftarrow (\hat{\mathbf{t}} - \hat{\mathbf{z}})\hat{\mathbf{B}}$
**while** $\|\boldsymbol{s}\|^2 > \lfloor\beta^2\rfloor$
return $(r, \mathbf{s})$

**Algorithm 27:** $\text{ffSampling}_n(\hat{\mathbf{t}}, T)$

**Input:** $\hat{\mathbf{t}} = (t_0, t_1) \in \text{FFT}(\mathbb{Q}[X]/(X^n+1))^2$, a Falcon tree $T$
**Output:** $\hat{\mathbf{z}} = (z_0, z_1) \in \text{FFT}(\mathbb{Z}[X]/(X^n+1))^2$

**if** $(n == 1)$ **then**
  $\sigma' \leftarrow$ T.value  // It is always the case that $\sigma' \in [\sigma_{\min}, \sigma_{\max}]$
  $z_0 \leftarrow \text{SamplerZ}(t_0, \sigma')$
  $z_1 \leftarrow \text{SamplerZ}(t_1, \sigma')$
  return $\hat{\mathbf{z}} = (z_0, z_1)$
$(l, T_0, T_1) \leftarrow$ (T.value, T.leftchild, T.rightchild)
$\hat{\mathbf{t}}_1 \leftarrow \text{splitfft}(t_1)$  // $\hat{\mathbf{t}}_0, \hat{\mathbf{t}}_1 \in \text{FFT}(\mathbb{Q}[X]/(X^{n/2}+1))^2$
$\hat{\mathbf{z}}_1 \leftarrow \text{ffSampling}_{n/2}(\hat{\mathbf{t}}_1, T_1)$
$z_1 \leftarrow \text{mergefft}(\hat{\mathbf{z}}_1)$  // $\hat{\mathbf{z}}_0, \hat{\mathbf{z}}_1 \in \text{FFT}(\mathbb{Z}[X]/(X^{n/2}+1))^2$
$t'_0 \leftarrow t_0 + (t_1 - z_1) \odot l$
$\hat{\mathbf{t}}_0 \leftarrow \text{splitfft}(t'_0)$
$\hat{\mathbf{z}}_0 \leftarrow \text{ffSampling}_{n/2}(\hat{\mathbf{t}}_0, T_0)$
$z_0 \leftarrow \text{mergefft}(\hat{\mathbf{z}}_0)$
return $\hat{\mathbf{z}} = (z_0, z_1)$

### Verification

**Algorithm 28:** $\text{Verify}(m, sig, pk, \lfloor\beta^2\rfloor)$

**Input:** A message m, a signature $sig = (r, s)$, a public key $pk = h \in \mathbb{Z}_q[x]/(\phi)$, a bound $\lfloor\beta^2\rfloor$
**Output:** Accept or reject

$c \leftarrow \text{HashToPoint}(r||m, n, q)$
$s_2 \leftarrow \text{Decompress}(s)$
**if** $s_2 = \perp$ **then**
  reject
$s_1 \leftarrow c - s_2.h \pmod q$  // $s_1$ should be normalized between $\lceil\frac{-q}{2}\rceil$ to $\lfloor\frac{q}{2}\rfloor$
**if** $\|(s_1, s_2)\|^2 \leq \lfloor\beta^2\rfloor$ **then**
  accept
**else**
  reject
// Reject signatures that are too long

The computation of $s_2$ can be performed in $\mathbb{Z}_q[x]/(\phi)$; the resulting values should then be normalized to the range $\lceil\frac{-q}{2}\rceil$ to $\lfloor\frac{q}{2}\rfloor$. To avoid computing a square root, the squared norm can be computed, using only integer operations, and then compared to $\lfloor\beta^2\rfloor$. Given a message $\texttt{msg}$, a signature $(r, \mathbf{s})$ and a public key $h$, the verifier first computes the hash of the message $c := \text{HashToPoint}(r\|\texttt{msg}, q, n)$ and $\mathbf{s}_2 \leftarrow \text{Decompress}(\mathbf{s})$, then computes the first signature component $s_1 = c - hs_2$, and finally verifies that $(s_1, s_2)$ is sufficiently short i.e., $\|(\mathbf{s}_1, \mathbf{s}_2)\|^2 < \lfloor\beta^2\rfloor$, where $\beta$ is a fixed chosen bound. If so, the signature is accepted. Otherwise, the verifier rejects the signature.

### Sampler over the Integers

Let $1 \leq \sigma_{\min} < \sigma_{\max}$. Here we will show how to sample securely Gaussian samples $z$ from $D_{\mathbb{Z},\sigma',\mu}$ for any $\sigma' \in [\sigma_{\min}, \sigma_{\max}]$ and $\mu \in \mathcal{R}$. This is done by $\mathrm{SamplerZ}$, which calls $\mathrm{BaseSampler}$ and $\mathrm{BerExp}$ as subroutines. We use the notations $(\gg)$ and $(\&)$ to denote the bit wise right-shift and AND, respectively. We also introduce the notations $[\![\cdot]\!]$ and UniformBits.

For any logical proposition $P$

$$[\![P]\!] = \begin{cases} 1 & \text{if } P \text{ is true,} \\ 0 & \text{if } P \text{ is false.} \end{cases} \tag{2.21}$$

Note that $[\![\cdot]\!]$ needs to be realised constantly for our algorithms to resist timing attacks. All $k \in \mathbb{Z}^+$, $\mathrm{UniformBits}(k)$ samples $z$ uniformly in $\{0, 1, \ldots, 2^k - 1\}$.

### BaseSampler

It samples an integer $z_0 \in \mathbb{Z}^+$ according to the distribution $\chi$ of support $\{0, \ldots, 18\}$ uniquely defined as:

$$\forall i \in \{0, \ldots, 18\}, \quad \chi(i) = 2^{-72} \cdot \mathrm{pdt}[i]. \tag{2.22}$$

The distribution $\chi$ is extremely close to the "half-Gaussian" $D_{\mathbb{Z}^+,\sigma_{\max}}$ in the sense that $R_{513}(\chi \| D_{\mathbb{Z}^+,\sigma_{max}}) \leq 1 + 2^{-78}$. We define the following quantities.

- the (scaled) probability distribution table $\mathrm{pdt}[i]$.
- the (scaled) cumulative distribution table $\mathrm{cdt}[i] = \sum_{j \leq i} \mathrm{pdt}[j]$.
- the (scaled) reverse cumulative distribution table $\mathrm{RCDT}[i] = \sum_{j > i} \mathrm{pdt}[j] = 2^{72} - \mathrm{cdt}[i]$.

**Algorithm 29:** $\mathrm{BaseSampler}()$

**Input:**
**Output:** An integer $z_0 \in \{0, 1, \ldots, 18\}$, z from $\chi$

$u \leftarrow \mathrm{UniformBits}(72)$
$z_0 \leftarrow 0$
**for** $i = 0, \ldots, 17$ **do**
  $z_0 \leftarrow z_0 + [\![u < \mathrm{RCDT}[i]]\!]$
return $z_0$

Given inputs $x$ and $\mathtt{ccs} \geq 0$, BerExp returns a single bit 1 with probability equal to $\mathtt{ccs} \cdot \exp(-x)$.

In the above Algorithm 27, we remind you that the $\mathrm{SamplerZ}$, is an algorithm for doing secure Gaussian sampling $z \leftarrow D_{\mathbb{Z},\sigma',0}$ for any $\sigma'$ in $R$. For more details on $\mathrm{SamplerZ}$, we refer to the paper [40].

**Algorithm 30:** ApproxExp$(x, \texttt{ccs})$

**Input:** Floating-point values $x \in [0, \ln(2)]$ and $\texttt{ccs} \in [0, 1]$
**Output:** An integral approximation of $2^{63} \cdot \texttt{ccs} \cdot \exp(-x)$

$C = [\texttt{0x00000004741183A3}, \texttt{0x00000036548CFC06}, \texttt{0x0000024FDCBF140A},$
$\texttt{0x0000171D939DE045}, \texttt{0x0000D00CF58F6F84}, \texttt{0x000680681CF796E3},$
$\texttt{0x002D82D8305B0FEA}, \texttt{0x011111110E066FD0}, \texttt{0x0555555555070F00},$
$\texttt{0x155555555581FF00}, \texttt{0x400000000002B400}, \texttt{0x7FFFFFFFFFFF4800},$
$\texttt{0x8000000000000000}]$

$y \leftarrow C[0]$
$z \leftarrow \lfloor 2^{63} \cdot x \rfloor$
**for** $u = 1$ **to** $12$ **do**
  $y \leftarrow C[u] - (z \cdot y) \gg 63$
$z \leftarrow \lfloor 2^{63} \cdot \texttt{ccs} \rfloor$
$y \leftarrow (z \cdot y) \gg 63$
**return** $y$

**Algorithm 31:** BerExp$(x, \texttt{ccs})$

**Input:** Floating point values $x, \texttt{ccs} \geq 0$
**Output:** A single bit, equal to 1 with probability approximately $\texttt{ccs} \cdot \exp(-x)$

/* Compute the unique decomposition $x = 2^s \cdot r$ with $(r, s) \in [0, \ln(2)] \times \mathbb{Z}^+$ */
$s \leftarrow \lfloor x / \ln(2) \rfloor$, $r \leftarrow x - s \cdot \ln(2)$
$s \leftarrow \min(s, 63)$
$z \leftarrow (2 \cdot \text{ApproxExp}(r, \texttt{ccs}) - 1) \gg s$ // $z \approx 2^{64-s} \cdot \texttt{ccs} \cdot \exp(-r) = 2^{64} \cdot \texttt{ccs} \cdot exp(-x)$
$i \leftarrow 64$
**do**
  $i \leftarrow i - 8$
  $w \leftarrow \text{UniformBits}(8) - ((z \gg i) \,\&\, \texttt{0xFF})$
**while** $(w = 0$ and $i > 0)$
**return** $[\![w < 0]\!]$ // Return 1 with probability $2^{-64} \cdot z \approx \texttt{ccs} \cdot \exp(-x)$

**Algorithm 32:** SamplerZ$(\mu, \sigma')$ [47]

**Input:** Floating-point values $\mu, \sigma' \in \mathcal{R}$ such that $\sigma \in [\sigma_{\min}, \sigma_{\max}]$
**Output:** An integer $z \in \mathbb{Z}$ sampled from a distribution very close to $D_{\mathbb{Z},\mu,\sigma'}$

$r \leftarrow \mu - \lfloor \mu \rfloor$ // $r \in [0, 1)$
$\texttt{ccs} \leftarrow \sigma_{\min} / \sigma'$ // the algo runs in time independent of $\sigma'$
**while** $(1)$ **do**
  $z_0 \leftarrow \text{BaseSampler}()$
  $b \leftarrow \text{UniformBits}(8) \,\&\, \texttt{0x1}$
  $z \leftarrow b + (2 \cdot b - 1) z_0$, $x \leftarrow \frac{(z-r)^2}{2\sigma'^2} - \frac{z_0^2}{2\sigma_{\max}^2}$
  **if** $\text{BerExp}(x, \texttt{ccs}) = 1$ **then**
    **return** $z + \lfloor \mu \rfloor$

# Milestone 3

# Fixing of mathematical basis of the new DSA

Most currently proposed post-quantum digital signature schemes fall into two broad categories. The **SIS**, **LWE** or its variants based Fiat Shamir with abort framework 2.1.1 and the GPV framework 2.2.2. Both frameworks base their security on the computational hardness of lattice problems. The lattice problems are known to be computationally hard, even for quantum computers. The Fiat Shamir transform is a well-known strategy for converting a secure Identification protocol into a secure digital signature algorithm using a cryptographic hash function. It is a very successful strategy. The ECDSA, the elliptic curve digital signature algorithm used currently in practice, is also based on the Fiat Shamir transform. Due to the imminent threat of quantum computers, the old mathematical foundation, namely the factorisation problem and the discrete logarithm, is no longer trustworthy. The quest for a new mathematical basis started, and the variants of hard lattice problems turned out to be quantum-safe and suitable to our cryptographic requirements. The classical lattice problems, namely **SVP**, **SIVP** and **CVP** are already known to be computationally hard, even to the quantum computers. The shortest vector problem **SVP** was shown to be NP-hard by Ajtai [3] and NP-hard to approximate to within any constant factor by Khot [50]. The best-known algorithm to find the exact shortest vector, or even some polynomial in $n$ factor approximation of it, takes time $2^{O(n)}$ [5, 56]. We also have hardness results for **SIVP**. It is known that **SIVP** is NP-hard to approximate for any constant factor [19], and finding the exact solution takes time approximately $n!$ [65] (although finding a $(1+\epsilon)$ approximation takes time $2^{O(n)}$ for any constant $\epsilon$ [18]). All these hardness results give us enough assurance for these lattice problems, but the main difficulty is that they can't be used for cryptographic purposes, even if somehow they are used in building crypto protocols, the resulting schemes is to slow to be used. The new lattice problem namely **SIS** and **LWE** came as a saviour.

The **SIS** problem was due to the work of Ajtai in $1996$, and the **LWE** problem was introduced by Regev in $2005$. The **SIS** and **LWE** problems are both hard on average (worst-case lattice problems quantumly reduce to it), and flexible enough to allow for the design of cryptographic protocols. In the following, we will revisit **SIS**, **LWE** and its variants and analyse their properties.

**DEFINITION 3.0.1: SIS Problem**

Let $n, m, q, \beta$ be positive integers with $m \gg n$, $q = \text{poly}(n)$ and $\beta \ll q$. Given $m$ uniformly random vectors $\{\mathbf{a}_i \xleftarrow{\$} \mathbb{Z}_q^n : 1 \leq i \leq m\}$, find a nonzero integer vector $\mathbf{z} = (z_1, z_2, \ldots, z_m) \in \mathbb{Z}^m$ of norm $\|\mathbf{z}\| < \beta$, such that $\sum_{i=1}^{m} z_i \mathbf{a}_i = \mathbf{0}$.

In the **SIS** problem, given in the Definition 3.0.1, $m \gg n$ and hence it is an underdetermined system of equations, and we are aiming to recover a small solution modulo $q$. There are so many solutions, and getting the desired one is an uphill task.

**DEFINITION 3.0.2: LWE Problem**

For $n, q \in \mathbb{Z}$ and the distributions $\chi$ and $\phi$ over $\mathbb{Z}$, the LWE distribution for a given vector $\mathbf{s} \in \mathbb{Z}_q^n$ is a set of pairs $(\mathbf{a}, \mathbf{t})$ where $\mathbf{t} = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod q$, where $\mathbf{a} \in \mathbb{Z}_q^n$ is sampled uniformly and where $e$ is sampled from $\phi$. The search LWE problem is to find out **s** or **e**, given $m$ many LWE samples. The decision **LWE** problem is to distinguish **t** form a uniformly random vector in $\mathbb{Z}_q^n$.

The flexibility of **SIS** and **LWE** is a boon for cryptography, but the cryptographic protocols based on them still suffer in terms of efficiency when compared to the efficiency of DLP and factorisation-based protocols. Fortunately, we have a solution: the variant of the **SIS** and **LWE** problem over the ideal and module lattices. These **SIS** variants are called ring SIS (**RSIS**) and module SIS (**MSIS**) problems. Similarly, we have the ring LWE (**RLWE**) and the module **MLWE** problems.

**DEFINITION 3.0.3: $\text{RSIS}_{q,m,\beta}$**

Let $R := \frac{\mathbb{Z}[x]}{\langle x^n+1 \rangle}$, $\quad R_q := \frac{\mathbb{Z}_q[x]}{\langle x^n+1 \rangle}$. Given $\mathbf{a}_1, \mathbf{a}_2, \ldots \mathbf{a}_m \in R_q$, chosen independently from the uniform distribution, the **RSIS** problem is to find $\mathbf{z}_1, \mathbf{z}_2, \ldots \mathbf{z}_m \in R$ such that $\sum_{i=1}^{m} \mathbf{a}_i \cdot \mathbf{z}_i = \mathbf{0} \pmod q$ and $0 < \|\vec{\mathbf{z}}\| \leq \beta$ where $\vec{\mathbf{z}} = (\mathbf{z}_1, \mathbf{z}_2, \ldots \mathbf{z}_m)^T \in R^m$.

**DEFINITION 3.0.4: $\text{MSIS}_{q,m,\beta}$**

Let $R := \frac{\mathbb{Z}[x]}{\langle x^n+1 \rangle}$, $\quad R_q := \frac{\mathbb{Z}_q[x]}{\langle x^n+1 \rangle}$. Given $\vec{\mathbf{a}}_1, \vec{\mathbf{a}}_2, \ldots \vec{\mathbf{a}}_m \in R_q^d$, chosen independently from the uniform distribution, the **MSIS** problem is to find $\mathbf{z}_1, \mathbf{z}_2, \ldots \mathbf{z}_m \in R$ such that $\sum_{i=1}^{m} \vec{\mathbf{a}}_i \cdot \mathbf{z}_i = \mathbf{0} \pmod q$ and $0 < \|\vec{\mathbf{z}}\| \leq \beta$ where $\vec{\mathbf{z}} = (\mathbf{z}_1, \mathbf{z}_2, \ldots \mathbf{z}_m)^T \in R^m$.

**DEFINITION 3.0.5: RLWE**

Let $R = \mathbb{Z}[x] / \langle x^n + 1 \rangle$ and $R_q = R \pmod q$. Let $\chi$, $\phi$ and $U$ (uniform) be distributions over $R_q$. The **RLWE** instance is $(\mathbf{A}, \mathbf{t})$, where $\mathbf{A} \leftarrow U^{1 \times m}$, $\mathbf{e} \leftarrow \phi^{1 \times m}$ and $\mathbf{s} \leftarrow \chi$.

- Search **RLWE**: Given $(\mathbf{A}, \mathbf{t})$, find **s**.
- Decision **LWE**: Distinguish **t** from uniform random element in $R_q$.

**DEFINITION 3.0.6:** MLWE

Let $R_q = \mathbb{Z}[x]/\langle x^n + 1\rangle$, $\mathbf{A} \leftarrow U^{m\times d}$ and $\mathbf{S}\leftarrow\chi^{d\times 1}$, $\mathbf{E}\leftarrow\phi^{m\times 1}$, where $\chi$, $\phi$ and $U$ (uniform) are distributions over $R_q$.

- Search $\mathbf{MLWE}$: Given $(\mathbf{A}, \mathbf{T})$, find $\mathbf{S}$, where $\mathbf{T} = \mathbf{AS} + \mathbf{E}$.
- Decision $\mathbf{MLWE}$: Distinguish $\mathbf{T}$ from uniform random element in $R_q^d$.

Note that in the above we have taken the ring $R_q = \mathbb{Z}_q[x]/\langle x^n + 1\rangle$, more generally it can be taken as $R_q = \mathbb{Z}_q[x]/\langle \varphi(x)\rangle$, where $\varphi(x)$ is irreducible over rationals, sparse with small coefficients. The ideal and module lattices have a more algebraic structure than other lattices. Because of this, the community was apprehensive about its uses in modern crypto protocols. It was thought that these additional structures may facilitate the algorithms solving **LWE** and **SIS** problems. This apprehension is still valid, but so far, no algorithm utilising these additional structures to solve **LWE** and **SIS** problems has been proposed. The best-known algorithms for ideal lattices perform essentially no better than their generic counterparts, both in theory and in practice. In particular, the asymptotically fastest known algorithms for obtaining an approximation to SVP on ideal lattices to within polynomial factors require time $2^{\Omega(n)}$, just as in the case of general lattices [67, 5]. Similarly to SIS and LWE, RSIS and RLWE admit reductions from worst-case lattice problems [63], but, however, the corresponding worst-case problem is now $\mathbf{SIVP}_\gamma$ restricted to ideal lattices. The latter problem is called $\mathbf{idSIVP}_\gamma$.

We also get some confidence in the hardness of ideal lattices because they arise naturally in algebraic number theory, a deep and well-studied branch of mathematics investigated reasonably thoroughly from a computational point of view. On the other hand, module lattice generalises both arbitrary lattices and ideal lattices. It further provides us with a lot more flexibility. A reduction from Mod-SIVP (i.e., SIVP restricted to module lattices) to M-SIS and a (quantum) reduction from Mod-SIVP to MLWE in both its search and decision versions is also presented in [58]. As Mod-SIVP is known to be a harder problem than Id-SIVP, so MLWE and MSIS is supposed to be safer to work with than RSIS and RLWE problems.

The assumption that standard lattice problems remain hard for ideal lattices is extensively used in modern cryptographic constructions via the RSIS and RLWE problems. However, ideal lattices are very structured objects, and the existence of algorithms exploiting their specific properties should not be discarded too quickly. Another approach would be to base the hardness of RSIS and RLWE on the worst-case hardness of standard lattice problems for more general classes of lattices. Note that a result in that direction is obtained in [22], but only in the case of an exponential modulus $q$, which is of limited practical interest. No weakness is currently known for ideal lattices and module lattices of module rank greater than 1, but if we are too conservative and security is our ultimate priority, we should base our protocols on the classical **LWE** and **SIS** only.

> **Remark.** The Fiat Shamir with abort framework-based digital signature schemes relies on the SIS and LWE problems and their variants for their security. They further employ the rejection sampling technique to hide the secret information.

## 3.1 GPV Framework on NTRU lattices

The GPV framework is another very successful framework for designing digital signature schemes. Its security is based on the computational hardness of the SIS Problem on the NTRU lattices (alge-

braic lattices having rich structure). We recap a very high-level description of the GPV framework here. The public key is a long (bad) basis of a $q$-ary lattice, and the private key is a short (good) basis of the same lattice. The message is hashed to a random-looking point $\mathbf{c} = H(m)$ in the span of the lattice. Using the short basis, a lattice point $\mathbf{v}$ is computed, which is very close to $\mathbf{c}$, and the signature is $\mathbf{s} = \mathbf{c} - \mathbf{v}$. The verification is done by checking if the vector $H(m) - \mathbf{s}$ is in the lattice using the bad basis.

As discussed in section 2.2.2, the GPV framework has been proven secure in the quantum random oracle model, assuming the hardness of the SIS problem. Note that the NTRUsign signature scheme based on a similar framework was broken badly [31, 71, 42]. This is because the algorithm computed the closest lattice vector leaked information about the secret basis. Crucial to the GVP frame security is how we compute a close lattice given a target. It should not leak any information about the secret key. This strategy sampling of a close lattice vector and the resulting algorithm is called the sampler.

### 3.1.1 Samplers for computing a close lattice vector

The security and quality of the sampling algorithm are the basis of the resulting digital signature scheme. There are two aspects of it: one is efficiency, and the other is quality and safety of sampling. Depending on the sampling algorithms, we have different digital signature schemes. Note that in all these schemes, the public and secret keys are generated in the same way as in the NTRU signature. We have already seen Falcon in the great details where FFSampler is used in the section 2.2.4. The basic idea was introduced in the late $90$'s with the $\mathrm{GGH}$ [44] and $\mathrm{NTRUSign}$ [45]. In the year 2008, Gentry, Peikert and Vaikuntanathan (GPV)] showed how to use Gaussian sampling in the lattice so that the signatures do not reveal any information about the secret key or trapdoor. Next, Damien Stehlé and Ron Steinfeld used the GPV framework and the NTRU lattice together and made NTRUSign and NTRUEncrypt as secure as standard worst-case problems over ideal lattices [87].

Leo Ducas, Vadim Lyubashevsky and Thomas Prest have created a GVP-style digital signature scheme DLP [29], which is instantiated over the power-2-cyclotomic tower of subfields. However, the signing algorithm is slow and quadratic in lattice dimension $2d$. This is because the randomized Babai's nearest plane algorithm or Klein's algorithm can not take advantage of the ring structure of the lattice. Then, FALCON comes into the picture. The FALCON, the DLP scheme's direct successor, has undergone many changes, especially in key and signature generation algorithms. It replaces the quadratic complexity Klein's sampler in signature generation with an efficient, quasilinear complexity lattice Gaussian sampler, which is called **Fast Fourier Orthogonalization** (FFO), [33] that does take advantage of the underlying ring structure. This FFO sampler makes FALCON attractive. All the calculations in FALCON are done in the Fast Fourier domain.

> **Remark.** The FFO makes FALCON super efficient, but it is also a main drawback of FALCON. The FFO is difficult to implement correctly, parallelize or protect against side-channel attacks. Also, FFO restricts FALCON over the power-2-cyclotomic tower of subfields, which limits FALCON's versatility in parameter selection.

#### Hybrid Sampler

In [32], Leo Ducas and Thomas Prest have developed a new Gaussian sampler, named "Hybrid Sampler", which samples lattice vectors of quality not as good as Klein's sampler [43] but is

as fast as Peikert's sampler [74] over NTRU lattice. In the hybrid sampler, they used Peikert's sampler as a subroutine, but the structure remains the same as that of Klein's. The algorithm is in quasilinear time complexity. In the year 2021, Thomas Espitau, Akira Takahashi, Mehdi Tibouchi and Alexandre Wallet have proposed a digital signature named MITAKA [36], a variant of FALCON. They replaced the FFO sampler with the Hybrid sampler over the NTRU lattice. Specifically, the security of Klein's sampler and Hybrid Sampler to make NTRU trapdoors over the power-2-cyclotomic ring of dimension $512$ (resp. $1024$) achieves over $120$ (resp. $280$) bits and $80$ (resp. $200$) bits of security, respectively.

**Remark** (Hybrid Sampler vs. FFO)**.** The substantive security loss is the main reason that led to the hybrid sampler being expelled in favour of the FFO sampler (which samples the same quality lattice vector as Klein's sampler but with quasilinear time complexity) in FALCON. Apart from security, the hybrid sampler has several advantages over the FFO sampler.

- The hybrid sampler is simpler to implement, but for FFO, it is an open question of how to implement it practically.
- The hybrid sampler is easily parallelizable, but FFO is not.
- The hybrid sampler is less difficult to protect against side-channel attacks, while in FFO, sampler$\mathbb{Z}$ is isochronous, which means FFO is free from side-channel attack [47].
- Like Peikert's sampler, the hybrid sampler also has an online-offline structure, while FFO has no offline-online structure.

Thus, the hybrid sampler loses to FFO due to the above reasons.

### MITAKA Digital Signature Scheme

Thomas Espitau *et al.* have proposed the MITAKA digital signature scheme in which they came up with an optimized way to create trapdoors for the hybrid sampler that reduces security loss by possibly constructing better quality trapdoors in a reasonable time. But MITAKA remains less secure than FALCON in equal dimensions. It loses over $20$ bits of classical CoreSVP security over rings of dimension $512$ and $50$ bits over rings of dimension $1024$. In particular, MITAKA falls short of NIST security level I in dimension $512$ and of level V in dimension $1024$, making it less than ideal from the standpoint of parameter selection. A key recovery attack on MITAKA can be found in the paper [78] by Thomas Preset.

### The ANTRAG trapdoor generation

In 2023, Thomas Espitau, Thi Thu Quyen Nguyen, Chao Sun, Mehdi Tibouchi and Alexandre Wallet, keeping all the drawbacks of MITAKA in mind, have developed an optimized way to generate NTRU trapdoors of the hybrid sampler, called ANTRAG [38].

Over the ring $R = \mathbb{Z}[x]/\langle x^n + 1\rangle, n = 2^k, k \in \mathbb{Z}_{>0}$, an NTRU lattice is just a submodule of $R^2$ generated by the rows of a matrix of the form:

$$\mathbf{B}_h = \begin{bmatrix} 1 & h \\ 0 & q \end{bmatrix}$$

for some prime $q$ and $h \in R$. Precisely, the lattice is $M_{h,q} = \{(u, v) \in R : v \equiv u \cdot h \pmod q\}$. A trapdoor for the lattice $M_{h,q}$ is a relatively short basis $\mathbf{B}_{f,g} = \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$, where $f$ is invertible

and $h \equiv g \cdot f^{-1} \pmod{q}$. Using $\mathbf{B}_{f,g}$, lattice Gaussian samplers output lattice vectors following a Gaussian distribution on the lattice $M_{h,q}$ of the standard deviation a small multiple $\alpha\sqrt{q}$ The $\alpha$ is said to be quality, and its value depends on the trapdoor and the sampler. The lower the quality, the better the trapdoor, and the higher the security level of the resulting signature scheme. For Klein's sampler, $\alpha = 1.17$ and $\alpha = 2.04$ in dimension $512$ for the Hybrid sampler. The quality is computed for the NTRU basis $\mathbf{B}_{f,g}$ as follows:

$$\alpha^2 = \max_{1 \le i \le \frac{d}{2}} \left( \max \left( \frac{|\phi_i(f)|^2 + |\phi_i(g)|^2}{q}, \frac{q}{|\phi_i(f)|^2 + |\phi_i(g)|^2} \right) \right), \tag{3.1}$$

where $\phi_i : R \mapsto \mathbb{C}, i = 1, \ldots, d$ are homomorphisms and are defined as $\phi_i(u) = u(\zeta_i)$, where the $\zeta_i$'s are the $d$ primitive $2d^{\text{th}}$ roots of unity in $\mathbb{C}$.

In both the FALCON and MITAKA, the strategy is to generate random-looking $f$ and $g$ and continue the *key generation* algorithm until the target quality is reached. Now, in ANTRAG, a different strategy is being followed: pick the pair $(f, g)$ uniformly randomly in the set of vectors that specify the desired quality level. The quality testing is done as follows:

$$\frac{q}{\alpha^2} \le |\phi_i(f)|^2 + |\phi_i(g)|^2 \le \alpha^2 q. \tag{3.2}$$

That means, for each embeddings, the pair $(|\phi_i(f)|, |\phi_i(g)|)$ lies in the annulus $A(\frac{\sqrt{q}}{\alpha}, \alpha\sqrt{q})$ bounded by the circles of radii $\frac{\sqrt{q}}{\alpha}$ and $\alpha\sqrt{q}$. More precisely, in the arc $A^+_\alpha = A^+(\frac{\sqrt{q}}{\alpha}, \alpha\sqrt{q})$ of that annulus located in the upper-right quadrant of the plane, since those absolute values are non-negative numbers. Here, the approach is to sample $f$ and $g$ using their embeddings (i.e., directly in the Fourier domain) and randomly select those embeddings uniformly and independently in the desired space. We sample $d/2$ pairs $(x_i, y_i)$ in the arc of the annulus and set the $i$-th embeddings of $f$ and $g$ to a uniformly random complex number of absolute value $x_i$ and $y_i$ respectively. Here, we have two issues:

1. The $f$ and $g$ are constructed may not be in the ring. After mapping back to the coefficient domain by Fourier inversion, their coefficients may be real numbers instead of integers. We can address this by rounding off the coefficients.

2. The rounding may not preserve the quality property, so instead of picking the pairs $(x_i, y_i)$ in $A^+_\alpha$, they are sampled from uniformly in some $A^+(r, r_1)$ with $r$ slightly larger than $\frac{\sqrt{q}}{\alpha}$ and $r_1$ is slightly smaller than $\alpha\sqrt{q}$. This increases the probability that, after rounding, all the pairs $(|\phi_i(f)|, |\phi_i(g)|)$ will end up in $A^+_\alpha$.

The description above is essentially a complete trapdoor generation algorithm for the hybrid sampler that easily reaches the same NIST security level as FALCON. Specifically, we target $\alpha = 1.15$ in dimension 512 and $\alpha = 1.23$ in dimension 1024.

**SOLMAE**

The Solmae is a successor of FALCON, MITAKA. It was introduced on April 7, 2024. It is a faster and quantum-safe digital signature following the traditional hash-and-sign paradigm in the style of GPV signatures, instantiated over the NTRU lattice. It is better to say that it offers the "best of three worlds" between FALCON, MITAKA and ANTRAG. Its design relies on the much simpler hybrid sampler [32]. The simple reuse of FALCON's key-generation algorithm is not sufficient. In [38], a new key generation algorithm brings the hybrid sampler back and describes a better way to improve the quality of the trapdoor.

**$\mathbb{Z}$alcon**

It is a floating-point free tweak version of FALCON, submitted to the $3^{\text{rd}}$ NIST PQC workshop. The key-gen algorithm is slightly modified, and FFO was replaced by the trapdoor sampler introduced in [35]. In Peikert's sampler for a covariance matrix $\Sigma \in \mathbb{R}^{n\times n}$ we have the decomposition $\Sigma = AA^t$ where $A \in \mathbb{R}^{n\times n}$. The decomposition can be found in Cholesky's decomposition. But if we want a matrix $A$ with integer entries, then $A$ will no longer be a square matrix, and a decomposition can be found in [35]. The sampler follows Peikert's approach with the integer decomposition of the covariance matrix $\Sigma \in \mathbb{R}^{n\times n}$.

In the above, we have analysed all the GPV framework-based signature schemes and their advantages and shortcomings.

## 3.2 The choice of the framework and the mathematical basis for the new proposal

We have thoroughly analysed the two frameworks: the Fiat Shamir transform with abort strategy and the GPV framework. The underlined mathematical object in both frameworks is the lattice. However, the functioning of both of these frameworks is completely different. The GPV framework relies on the class of NTRU lattices and the sampling algorithms, which sample a very close lattice vector given a target. We have already mentioned all the existing approaches for sampling and the resulting digital signature schemes based on them in section 3.1. Most existing samplers either depend on the floating-point arithmetic and have complex implementations or leak information about the trapdoor secret basis (key). These samplers are further susceptible to side-channel attacks. Moreover, the NTRU class of lattices, being highly structured in nature, may turn out to be vulnerable to algebraic cryptanalysis. The reliance on floating-point arithmetic is another cause of concern. Furthermore, most of the existing signatures based on the GPV framework have no proof of security, and there is also a risk of side-channel attacks. Due to the above-mentioned facts, we feel less confident about basing the new design on the GPV framework.

On the other hand, Fiat Shamir's transformation with an abort strategy seems promising. Moreover, the rejection sampling has much potential to make this strategy foolproof, at least in the present state of knowledge available to the community. However, careful implementation of rejection sampling is another challenge to avoid any possibility of side-channel attacks. The LWE problem and its variants, being flexible and easy to fit into our cryptographic needs, are the natural choices to depend on. The only concern is that MLWE and RLWE problems are defined over lattices with rich algebraic structures that may be vulnerable in future. Nevertheless, this is also the case with the NTRU class of lattices. So far, we have not seen any weakness in such lattices exploiting the algebraic structures present, and the community worldwide seems to trust them. One clear example is the recommendation of the Dilithium signature scheme.

Keeping these things in mind, we can rely on the Fiat Shamir with an abort framework and may use MLWE, RLWE, or classical LWE as our basis for the security of these schemes. If security is our utmost priority, we recommend basing our design on the classical LWE problem, although the resulting scheme might be slower, so there is a trade-off. If we aim for efficiency, the MLWE or the RLWE problem are the problems to work with.

# Milestone 4

# Proposal for new Digital Signature Algorithm

In the Milestone 3, we have fixed the basic framework for the new digital signature scheme. We decided to base our design on the "Fiat-Shamir with Abort" strategy [60, 61] and base the security of the scheme on the Module Learning with Errors (MLWE) problem. In the following, we will present the new quantum-safe digital signature scheme. We will first describe the basic framework of the new signature algorithm, followed by a more detailed description and analysis.

## 4.1 Basic Framework for the new Signature Scheme

Since the Dilithium [28] digital signature scheme is also based on the Fiat-Shamir with Abort technique and its security is further based on the MLWE problem, we have taken the liberty of using many notations and functions as it is used in Dilithium. We have borrowed the notations from the Dilithium so that the explanations become easier. We recall that we use $R$ to represent the ring $\mathbb{Z}[X]/\langle x^n+1\rangle$ and $R_q$ for $R \pmod q$. We present a simplified version of the new scheme in the Algorithm 4.1.1 below. The simplified version of our scheme bears a resemblance to the Dilithium signature scheme; however, the full-fledged version of the new scheme (Algorithm 5.0.1) is significantly different from it.

**ALGORITHM 4.1.1: Basic Framework for the new Signature Scheme**

**Algorithm 33:** $\texttt{GEN}(k, \ell)$

1 $\mathbf{A} \xleftarrow{\$} R_q^{k\times l}$

2 $(\mathbf{s}, \mathbf{e}) \xleftarrow{\$} S_\eta^\ell \times S_\eta^k$

3 $\mathbf{t} = \mathbf{As} + \mathbf{e} \pmod q$

4 Note that $(\mathbf{A}|\mathbf{I}_k|\text{-}t)\begin{pmatrix}\mathbf{s}\\ \mathbf{e}\\ 1\end{pmatrix} = 0.$

5 $pk := \mathbf{A}' = (\mathbf{A}|\mathbf{I}_k|\text{-}t)\,, sk := \mathbf{S} = \begin{pmatrix}\mathbf{s}\\ \mathbf{e}\\ 1\end{pmatrix}$

**Algorithm 34:** $\texttt{SIGN}(m, sk)$

**while** $\boldsymbol{z} := \perp$ **do**
  $\mathbf{y} \xleftarrow{\$} S_{\gamma}^{l+k+1}$
  $\mathbf{w} := \mathbf{A}'\mathbf{y} \bmod q$
  $c \in B_{\tau} := H(m||\mathbf{w})$ `// Here H is SHAKE-256`
  $\mathbf{z} = \mathbf{y} + (-1)^b c\,\mathbf{S}$ for some $b \xleftarrow{\$} \{0, 1\}$
  **if** $\|\boldsymbol{z}\|_{\infty} < \gamma - \beta$ **then**
    **return** Signature $\sigma := (\mathbf{z}, c)$
  **else**
    $\mathbf{z} := \perp$

**Algorithm 35:** $\texttt{VRFY}(m, pk, \sigma)$

$\mathbf{w}' := \mathbf{A}'\mathbf{z} \bmod q$
**if** $\|\boldsymbol{z}\|_{\infty} < \gamma - \beta$ ***and*** $H\left(m|\boldsymbol{w}'\right) = c$ **then**
  **return** True
**else**
  **return** False

The key generation algorithm generates a $k \times l$ matrix $\mathbf{A}$, each of whose entries is a polynomial in the ring $R_q = \mathbb{Z}_q[X]/\langle x^n + 1\rangle$. The $n = 256$ and $q$ is a suitable NTT-friendly prime. The algorithm samples random secret key vectors $\mathbf{s}$ and $\mathbf{e}$. Each coefficient of these vectors is an element of $R_q$ with small coefficients of size at most $\eta$. Finally, the second part of the public key is computed as $\mathbf{t} = \mathbf{As} + \mathbf{e} \pmod q$.

The signing algorithm generates two masking vectors of polynomials $\mathbf{y}_1$, $\mathbf{y}_2$ and a polynomial $y_3$ with coefficients less than $\gamma$. The parameter $\gamma$ is set strategically - it should be large enough that the eventual signatures do not reveal any information about the secret key, yet small enough so that the signature is not easily forged. The signer then computes the commitment $\mathbf{w} = \mathbf{A}\mathbf{y}_1 + \mathbf{y}_2 - y_3\mathbf{t}$. Later, we will fix $y_3$ to 0. The challenge $c$ is then created as the hash of message $\mathbf{m}$ and commitment $\mathbf{w}$. The output $c$ is a polynomial in $R_q$ with exactly $\tau$ coefficients are $\pm 1$ and the rest of them are 0. The potential signature is then computed as $\mathbf{z} = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \\ y_3 \end{pmatrix} + (-1)^b c\,\mathbf{S}$, where $b$ is a random bit. The parameter $\beta$ is the maximum possible coefficient of $c\mathbf{S}$. Since $c$ has $\tau \pm 1's$ and the maximum coefficient in $\mathbf{S}$ is $\eta$, it's easy to see that $\beta \leq \tau \cdot \eta$. If any coefficient of $\mathbf{z}$ is less than $\gamma - \beta$, we publish the signature $(\mathbf{z}, \mathbf{c}, b)$; otherwise, we reject and restart the signing procedure. The check is necessary for security, and the loop keeps being repeated until the check is held. The parameters are set so that the expected number of rejections for signature output is not too high.

The verification is pretty straightforward. The verifier first computes $\mathbf{w}'$ to be $\mathbf{A}'\mathbf{z}$. The verification is successful if all the coefficients of $\mathbf{z}$ are less than $\gamma - \beta$ and $c \stackrel{?}{=} H(m|\mathbf{w}')$. Below, we

justify why the verification works,

$$\begin{aligned}\mathbf{w}' &= \mathbf{Az} \bmod q\\ &= (\mathbf{A}|\mathbf{I}_k| - \mathbf{t})\,(\mathbf{y} + bc\mathbf{S})\\ &= \mathbf{w} + bc\,(\mathbf{As} + \mathbf{e} - \mathbf{t})\\ &= \mathbf{w} + bc\,(\mathbf{t} - \mathbf{t})\\ &= \mathbf{w}\end{aligned}$$

Thus, the verification works since $c = H(m|\mathbf{w}) = H(m|\mathbf{w}')$ by the above result.

### 4.1.1 Recap of notations and basic mathematical pre-requisites

Although we are following the consistent notations, since the beginning, we will recall notations and basic mathematics to make this chapter self-content.

The value of $n$ will always be $256$. The regular font letters denote elements in $R$ or $R_q$ (which includes elements in $\mathbb{Z}$ and $\mathbb{Z}_q$) and bold lower-case letters represent column vectors with coefficients in $R$ or $R_q$. By default, all vectors will be column vectors. Bold upper-case letters are matrices. For a vector $\mathbf{v}$, we denote $\mathbf{v}^T$ its transpose. For an even (resp. odd) positive integer $\alpha$, we define $r' = r \bmod^{\pm} \alpha$ to be the unique element $r'$ in the range $-\frac{\alpha}{2} < r' \leq \frac{\alpha}{2}$ (resp. $-\frac{\alpha-1}{2} < r' \leq \frac{\alpha-1}{2}$) such that $r' \equiv r \pmod{\alpha}$. We sometimes refer to this as a *centered* reduction modulo $\alpha$. For any positive integer $\alpha$, we define $r' \equiv r \bmod^{+} \alpha$ to be the unique element $r'$ in the range $0 \leq r' < \alpha$ such that $r' \equiv r \pmod{\alpha}$. When the exact representation is unimportant, we simply write $r \pmod{\alpha}$.

For an element $w \in \mathbb{Z}_q$, we write $\|w\|_\infty$ to mean $|w \bmod^{\pm} q|$. For $w(x) = \sum_{i=0}^{n-1} w_i x^i \in R$ we define $\ell_2$ and $\ell_\infty$ norms as follows.

$$\|w(X)\|_\infty = \max(\|w_i\|_\infty),\ \|w(X)\|_2 = \sqrt{\|w_0\|_\infty^2 + \|w_1\|_\infty^2 + \ldots + \|w_{n-1}\|_\infty^2} \tag{4.1}$$

Similarly, for $\mathbf{w} = (w_1, w_2, \ldots w_k) \in R^k$, we define

$$\|\mathbf{w}\|_\infty = \max_i(\|w_i\|_\infty),\ \|\mathbf{w}\|_2 = \sqrt{\|w_1\|^2 + \|w_2\|^2 + \ldots + \|w_k\|^2}. \tag{4.2}$$

We will write $S_\eta = \{w \in R : \|w\|_\infty < \eta\}$. We will write $\tilde{S}_\eta$, to denote the set $\{w \bmod^{\pm} 2\eta : w \in R\}$. Note that $S_\eta$ and $\tilde{S}_\eta$ are very similar except that $\tilde{S}_\eta$ does not contain polynomials with $-\eta$ coefficients.

## Hashing

Our scheme uses several different algorithms that hash strings in $\{0,1\}^*$ onto domains of various forms. We have used the same hashing algorithms used in Dilithium. We recall them briefly in the following. The function $\mathrm{H}$ will be used in our signature scheme. It is the extended output function (XOF) SHAKE -256 mapping onto the specified domain.

#### Hashing to a Ball

This is exactly the same as the one used in Dilithium. We briefly recall it below. Let $B_\tau$ be the set of elements of $R$ that have $\tau$ many coefficients equal to $\pm 1$ and the rest are $0$. Thus $|B_\tau| = 2^\tau \cdot \binom{256}{\tau}$.

This signature scheme requires a cryptographic hash that hashes onto $B_\tau$ in two steps. In the first step, an element from $\{0,1\}^\star$ is mapped to $\{0,1\}^{256}$ using a $2^{nd}$ pre-image resistant hash function, in the second step, XOF (e.g. SHAKE-256) is used to map the output of first step onto an element of $B_\tau$. The reason for this two-step process is that the output of the first stage allows for a very simple, compact description of an element in $B_\tau$ (under the assumption that the XOF is indistinguishable from a purely-random function). The algorithm we will use to create a random element in $B_\tau$ is sometimes referred to as an "inside-out" version of the Fisher-Yates shuffle[54] . A high-level description of the algorithm for creating a random 256-element array in $B_\tau$ with $\tau \pm 1's$ and $256 - \tau$ $0's$ using the input seed $\rho$ (and an XOF to generate the randomness needed in Steps 3 and 4 is given in the algorithm SampleInBall below.

**Algorithm 36:** SampleInBall($\rho$)

1 Initialise $\mathbf{c} = c_0 c_1 \dots c_{255} = 000 \dots 0$
2 **for** $i := 256 - \tau$ *to* $255$ **do**
3 $\quad j \leftarrow \{0, 1, \dots, i\}$
4 $\quad s \leftarrow \{0, 1\}$
5 $\quad c_i := c_j$
6 $\quad c_j = (-1)^s$
7 return $\mathbf{c}$

Normally, the algorithm should begin at $i = 0$, but since there are $256 - \tau$ 0's, the first $256 - \tau - 1$ iterations would be setting components of $c$ to 0.

### Generating the matrix A

The function $\texttt{GenA}()$ construct $\mathbf{A} \in R_q^{k \times \ell}$ in NTT domain representation using a uniform seed $\rho \in \{0,1\}^{256}$. This is the same as the $\texttt{ExpandA}()$ function in the Dilithium. The matrix $\mathbf{A}$ is only needed for multiplication, hence for faster implementations, the function $\texttt{GenA}()$ outputs $\hat{\mathbf{A}} \in \mathbb{Z}^{256}$, instead of outputting $\mathbf{A} \in R_q^{k \times \ell}$. $\hat{\mathbf{A}}$ is interpreted as the $\texttt{NTT}$ domain representation of $\mathbf{A}$.

### Sampling the vectors s and e

The function $\texttt{GenS}()$ generates secret vectors in the key generation. It constructs secret vectors $(\mathbf{s}, \mathbf{e}) \in S_\eta^\ell \times S_\eta^k$ using a seed $\rho'$. This is the same as the $\texttt{ExpandS}()$ function of the Dilithium.

### Sampling the vectors $(\mathbf{y}_1, \mathbf{y}_2)$

The function $\texttt{GenMask}()$ deterministically generates randomness $(\mathbf{y}_1, \mathbf{y}_2) \in S_\gamma^\ell \times S_\gamma^\ell$ of the signature scheme using a seed $\rho'$ and a nonce $\kappa$.

## High-order and Low-order bits

An element $r$ of $\mathbb{Z}_q$ can be decomposed into higher-order and lower-order bits as shown in Power2Round$_q$. Suppose we want to drop $d$ low-order bits of $r$, then we can write $r$ as $r = r_1.2^d + r_0$, where $r_1$ and $r_0$ give the high and low-order bits of $r$, respectively.

**Algorithm 37:** Power2Round$_q$(r,d)

$r := r \mod q$
$r_0 := r \bmod^{\pm} 2^d$
$r_1 := (r - r_0)/2^d$
return $(r_1, r_0)$

### Hint Bits

Another way to break $r \in \mathbb{Z}_q$ is to choose an even divisor $\delta$ of $q-1$ and write $r = r_1 \cdot \delta + r_0$ in the same way as done earlier. The possible choices of $r_1 \cdot \delta$'s are $\{0, 1 \cdot \delta, 2 \cdot \delta, \ldots, q-1\}$. As $(q-1)$ and $0$ differ by $1$, we avoid $q-1$ as a result we may end up having $r_0$ higher by $1$. This is achieved by the procedure $\text{Decompose}_q$ given in the algorithm below. We additionally define two more functions, namely $\text{LowBits}_q(r, \delta)$ and $\text{HighBits}_q(r, \delta)$.

**Algorithm 38:** Decompose$_q(r, \delta)$

$r := r \mod q$
$r_0 := r \bmod^{\pm} \delta$
**if** $(r - r_0) > 0$ **then**
  $r_1 := 0, r_0 := r_0 - 1$
**else**
  $r_1 := \frac{r - r_0}{\delta}$
**return** $(r_1, r_0)$

**Algorithm 39:** HighBits$_q(r, \delta)$

$(r_1, r_0) :=$ Decompose$_q(r, \delta)$
**return** $r_1$

**Algorithm 40:** LowBits$_q(r, \delta)$

$(r_1, r_0) :=$ Decompose$_q(r, \delta)$
**return** $r_0$

This decomposition of an element of $\mathbb{Z}_q$ into high and low bits is done to reduce the size of the public key. We want to recover higher order bits of $r + z \in \mathbb{Z}_q$ without storing a small $z \in \mathbb{Z}_q$. For this, we use a 1-bit of hint $h$. The details are explained in the following algorithms.

**Algorithm 41:** MakeHint$_q(z, r, \delta)$

$r_1 :=$ HighBits$_q(r, \delta)$
$v_1 :=$ HighBits$_q(r + z, \delta)$
**if** $r_1 \neq v_1$ **then**
  **return** 1

**Algorithm 42:** UseHint$_q(h, r, \delta)$

$m := (q-1)/\delta$
$(r_1, r_0) :=$ Decompose$_q(r, \delta)$
**if** $h = 1$ *and* $r_0 > 0$ **then**
  **return** $(r_1 + 1) \mod m$
**if** $h = 1$ *and* $r_0 \leq 0$ **then**
  **return** $(r_1 - 1) \mod m$
**return** $r_1$

We will require the following lemmas in the security proof of the scheme.

**Lemma 5.** *Suppose that $q$ and $\alpha$ are positive integers satisfying $q > 2\alpha$, $q \equiv 1 (mod\ \alpha)$ and $\alpha$ is even . Let $\boldsymbol{r}$ and $\boldsymbol{z}$ be vectors of elements in $R_q$ where $\|\boldsymbol{z}\|_\infty \leq \alpha/2$ , and let $\boldsymbol{h}$, $\boldsymbol{h}'$ be two vectors of bits. Then The* HighBits$_q$ *,* MakeHint$_q$ *and* UseHint$_q$ *algorithms satisfy the following properties:*

1. $\text{UseHint}_q(\text{MakeHint}_q(\boldsymbol{z}, \boldsymbol{r}, \alpha), \boldsymbol{r}, \alpha) = \text{HighBits}_q(\boldsymbol{r} + \boldsymbol{z}, \alpha)$.

2. *Let* $\boldsymbol{v}_1 = \mathrm{UseHint}_q(\boldsymbol{h}, \boldsymbol{r}, \alpha)$*, then* $\|\boldsymbol{r} - \boldsymbol{v}_1\alpha\|_\infty \leq \alpha + 1$*. Furthermore, if the number of* $1$*'s in* $\boldsymbol{h}$ *is* $\omega$*, then all except at most* $\omega$ *coefficients of* $\boldsymbol{r} - \boldsymbol{v}_1\alpha$ *will have magnitude at most* $\alpha/2$ *after centred reduction modulo* $q$*.*

3. *For any* $\boldsymbol{h}$ *and* $\boldsymbol{h}'$*, if* $\mathrm{UseHint}_q(\boldsymbol{h}, \boldsymbol{r}, \alpha) = \mathrm{UseHint}_q(\boldsymbol{h}', \boldsymbol{r}, \alpha)$ *, then* $\boldsymbol{h} = \boldsymbol{h}'$*.*

**Lemma 6.** *Let* $(\boldsymbol{r}_1, \boldsymbol{r}_0) = \mathrm{Decompose}_q(\boldsymbol{r}, \alpha)$ *,* $(\boldsymbol{w}_1, \boldsymbol{w}_0) = \mathrm{Decompose}_q(\boldsymbol{r} + \boldsymbol{s}, \alpha)$ *and* $\|\boldsymbol{s}\|_\infty \leq \beta$*, then* $\|\boldsymbol{s} + \boldsymbol{r}_0\|_\infty \leq \alpha/2 - \beta \iff \boldsymbol{w}_1 = \boldsymbol{r}_1 \wedge \|\boldsymbol{w}_0\|_\infty \leq \alpha/2 - \beta$*.*

## 4.2 The New Digital Signature Scheme

The Key Generation, Signing and Verification algorithms for our signature scheme are presented in Algorithm 5.0.1. The main design improvement over the scheme in Algorithm 4.1.1 is public key compression. Due to this, the public key size is approximately halved at the expense of fewer than a hundred extra bytes in the signature (in the form of **h**). The main complications arising from this size reduction and the measures taken to combat those are further discussed in the following sections.

### 4.2.1 Key Generation

We compress the public key by dropping $d$ low-bits of $\mathbf{t}$ by using $(\mathbf{t}_1, \mathbf{t}_0) := \mathrm{Power2Round}_q(\mathbf{t}, d)$ and output $(\mathbf{t}_1, d)$, as the public key instead of the $\mathbf{t} \in \mathbb{Z}_q$ in the template. This means instead of $\lceil \log q \rceil$ bits per coefficient, the public key requires $\lceil \log q \rceil - d + \lceil \log d \rceil$ ( $< \lceil \log q \rceil$) bits. Also, we send a 256-bit seed $\rho$ to generate the matrix $\mathbf{A}$ here instead of sending the entire matrix all by itself. Details about the final public key size are described in Section 5.1.

### 4.2.2 Signing Procedure

Here, instead of directly using $\mathbf{w}$ as the commitment, we use the high bits of $\mathbf{w}$ to ease verification. Since each (possibly long) message may require several iterations to sign, we compute an initial digest of the message using a collision-resistant hash function, and use this digest in place of the message throughout the signing procedure. To add some randomness in the signing procedure, a seed $\rho''$ is generated (using SHAKE-256) as a deterministic function of the message $\mu$ and a small secret key $K$, which is then appended with a counter $\kappa$ in order to randomly generate $\mathbf{y}$ each time the signing procedure is repeated until a valid signature is produced. For randomized signing, $\rho''$ is chosen randomly from $\{0,1\}^{512}$. Another significant change is including a hint bit $\mathbf{h}$ in the signature. This is necessary since in order to complete the verification, the verifier needs to compute high-order bits of $\mathbf{A}'\mathbf{z} - c\mathbf{t}_0$, which is not possible without having $\mathbf{t}_0$ (the low-bits of $\mathbf{t}$). How this hint bit helps is explained further in the next section. The final signature will be $\sigma := (\mathbf{z}, \tilde{c}, \mathbf{h})$. Detailed calculation regarding the signature size is available in Section 5.1.

**ALGORITHM 4.2.2: Signature Scheme 1**

**Algorithm 43:** $\texttt{GEN}(k, \ell)$

$\zeta \leftarrow \{0,1\}^{256}$

$(\rho, \rho', K) \in \{0,1\}^{256} \times \{0,1\}^{512} \times \{0,1\}^{256} := \mathrm{H}(\zeta)$ // Here H is SHAKE-256

$\mathbf{A} \in R_q^{k \times \ell} := \mathrm{GenA}(\rho)$

$(\mathbf{s}, \mathbf{e}) \in S_\eta^\ell \times S_\eta^k := \mathrm{GenS}(\rho')$

$\mathbf{t} = \mathbf{As} + \mathbf{e} \pmod q$

$(\mathbf{t}_1, \mathbf{t}_0) := \mathrm{Power2Round}_q(\mathbf{t}, d)$

Note that $\left(\mathbf{A}|\mathbf{I}_k|\text{-}t_1.2^d\right) \begin{pmatrix} \mathbf{s} \\ \mathbf{e} \\ 1 \end{pmatrix} = \mathbf{t}_0 \pmod q$

$\mathrm{tr} \in \{0,1\}^{256} := \mathrm{H}(\rho||\mathbf{t}_1)$

**return** $pk := (\rho, \mathbf{t}_1, d)$, $sk := \left(\mathbf{S} = \begin{pmatrix} \mathbf{s} \\ \mathbf{e} \\ 1 \end{pmatrix}, \mathbf{t}_0, \rho', K, \mathrm{tr}\right)$

**Algorithm 44:** $\texttt{SIGN}(m, sk)$

$\mathbf{A} \in R_q^{k \times \ell} := \mathrm{GenA}(\rho)$

$\mathbf{A}' := \left(\mathbf{A}|\mathbf{I}_k| - \mathbf{t}_1.2^d\right)$

$\mu \in \{0,1\}^{512} := \mathrm{H}(\mathrm{tr}\,||m)$

$\kappa := 0, (\mathbf{z}, \mathbf{h}) := \perp$

$\rho'' \in \{0,1\}^{512} := \mathrm{H}(K||\mu)$ (or $\rho'' \leftarrow \{0,1\}^{512}$ for randomized signing)

**while** $\boldsymbol{z} = \perp$ **do**

  $\mathbf{y} \in \tilde{S}_{\gamma_1}^{l+k+1} := \mathrm{GenMask}(\rho'', \kappa)$ // $\mathbf{w} = (\mathbf{y}_1, \mathbf{y}_2, 0)$

  $\mathbf{w} := \mathbf{A}'\mathbf{y} \bmod q$ // $\mathbf{y} = \mathbf{A}\mathbf{y}_1 + \mathbf{y}_2 \bmod q$

  $\tilde{\mathbf{w}} = \mathrm{HighBits}_q(\mathbf{w}, 2\gamma_2)$

  $\tilde{c} \in \{0,1\}^{256} := \mathrm{H}(\mu||\tilde{\mathbf{w}})$

  $c \in B_\tau := \mathrm{SampleInBall}(\tilde{\mathbf{c}})$

  $\mathbf{z} = \mathbf{y} + (-1)^b c\mathbf{S}, b \xleftarrow{\$} \{0,1\}$

  **if** $\|\boldsymbol{z}\|_\infty \geq \gamma_1 - \beta$ **then**

    $(\mathbf{z}, h) := \perp$

  **else**

    $h = \mathrm{MakeHint}_q(-(-1)^b c\mathbf{t}_0, \mathbf{w} + (-1)^b c\mathbf{t}_0, 2\gamma_2)$

    **if** $\|(-1)^b c\boldsymbol{t}_0\|_\infty \geq \gamma_2$ ***or*** the number of *1's in* ***h*** is greater than $\omega$ **then**

      $(\mathbf{z}, h) := \perp$

  $\kappa := \kappa + \ell$

**return** Signature $:= \sigma = (\mathbf{z}, \tilde{c}, \mathbf{h})$

**Algorithm 45:** $\texttt{VRFY}(m, pk, \sigma)$

$\mathbf{A} \in R_q^{k \times \ell} := \text{GenA}(\rho)$
$\mathbf{A}' := (\mathbf{A}|\mathbf{I}_k| - \mathbf{t}_1.2^d)$
$\mu \in \{0,1\}^{512} := \text{H}(\text{H}(\rho||\text{t}_1)|\text{m})$
$c := \text{SampleInBall}(\tilde{c})$
$\mathbf{w}' := \text{UseHint}_q(\mathbf{h}, \mathbf{A}'\mathbf{z} \bmod q, 2\gamma_2)$
**if** $\|z\|_\infty < \gamma_1 - \beta$ ***and*** the number of 1's in $\boldsymbol{h}$ is lesser than $\omega$ **then**
    $c' := H(\mu|\mathbf{w}')$
    **if** $c = c'$ **then**
        True

### 4.2.3 Verification

For verification, the verifier needs to compute high bits of $\mathbf{A}'\mathbf{z} - c\mathbf{t}_0 = \mathbf{w}$. But $\mathbf{t}_0$ is not a part of the public key. To overcome this issue , we need $\text{UseHint}_q$, which returns the high bits of $\mathbf{w}$ (by lemma 5, since $\|bc\mathbf{t}_0\|_\infty < \gamma_2$ with high probability, by selection of parameters). Thus, the verifier can complete the verification. The detailed correctness proof follows.

#### Correctness

We have, $\mathbf{A}'\mathbf{z} \bmod q = \left(\mathbf{A}|\mathbf{I}_k| - \mathbf{t}_1 \cdot 2^d\right)\left(\mathbf{y} + (-1)^b c\mathbf{S}\right) = \mathbf{w} + (-1)^b c\left(\mathbf{As} + \mathbf{e} - \mathbf{t}_1.2^d\right) = \mathbf{w} + (-1)^b c\mathbf{t}_0$. Since, the parameters are set such that, $\|c\mathbf{t}_0\|_\infty < \gamma_2$ with high probability, by Lemma 5, we have

$$\mathbf{w}' = \text{UseHint}(\text{MakeHint}(\text{-(-1)}^{\text{b}}\text{c}\mathbf{t}_0, \mathbf{w} + \text{(-1)}^{\text{b}}\text{c}\mathbf{t}_0, 2\gamma_2), \mathbf{w} + \text{(-1)}^{\text{b}}\text{c}\mathbf{t}_0, 2\gamma_2) = \text{HighBits}(\mathbf{w}, 2\gamma_2) = \tilde{\mathbf{w}} \tag{4.3}$$

From the above equation, $c = H(\mu|\tilde{\mathbf{w}}) = H(\mu|\tilde{\mathbf{w}'}) = c'$. Thus, the verification procedure will always return true.

## 4.3 Parameter specifications

### 4.3.1 Public key size:

Our public key is $(\rho, \mathbf{t}_1, d)$.We drop d bits of $\mathbf{t}$ to get $\mathbf{t}_1$; thus, each coefficient of each polynomial of $\mathbf{t}_1$ is of $(\lceil(\log q)\rceil - d)$ bits and $\mathbf{t}_1$ has k many polynomials each of degree 256. Thus $\mathbf{t}_1$ is of $256 \cdot (\lceil(\log q)\rceil - d) \cdot k$ bits. Now, considering we take a 256-bit seed $\rho$ to generate matrix $\mathbf{A}$ and the number of bits we drop $d$, our public key size will be $(256 + 256.(\lceil(\log q)\rceil - d).k + \lceil(\log d)\rceil)$ bits.

### 4.3.2 Signature size:

Our signature is $(\mathbf{z}, \tilde{c}, \mathbf{h})$. The challenge $\tilde{c}$ is of 256 bits (32 bytes). The hint vector $\mathbf{h}$ is sent as an encoded byte string of length $\omega + k$ using HintBitPack, thus $\mathbf{h}$ has $\omega + k$ bytes. To produce a valid signature, we must have $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, thus the total number of choices for each coefficient of $\mathbf{z}$ is $2\gamma_1 - 2\beta - 1 < 2\gamma_1$, so each coefficient of $\mathbf{z}$ can be represented using $\log_2(2\gamma_1) = 1 + \log_2 \gamma_1$ bits,

and there are $(k+l+1)$ 256 degree polynomials in $\mathbf{z}$. Hence bitsize of $\mathbf{z}$ is $256.(k+l+1).(1+\log_2 \gamma_1)$ bits. Thus in bytes, size of the signature $(\mathbf{z}, \tilde{c}, \mathbf{h})$ is $32(k+l+1)(\log_2 \gamma_1 + 1) + 32 + (\omega + k)$.

| NIST Security level | 2 | 3 | 5 |
|---|---|---|---|
| Parameters | | | |
| $q$ [modulus] | 8380417 | 8380417 | 8380417 |
| $d$ [dropped bits of $\mathbf{t}$] | 13 | 13 | 13 |
| challenge entropy $[\log \binom{256}{\tau} + \tau]$ | 192 | 196 | 257 |
| $\tau$ [no. of $\pm 1$'s in $c$ ] | 39 | 49 | 60 |
| $\gamma_1$ [$\mathbf{y}$ coefficient range] | $2^{17}$ | $2^{19}$ | $2^{19}$ |
| $\gamma_2$ [low-order rounding range] | $(q-1)/88$ | $(q-1)/32$ | $(q-1)/32$ |
| $(k, l)$ [dimensions of $\mathbf{A}$] | (4,4) | (6,5) | (8,7) |
| $\eta$ [secret key range] | 2 | 4 | 2 |
| $\beta$ $[\tau.\eta]$ | 78 | 196 | 120 |
| $\omega$ [max no. of 1's in the hint $\mathbf{h}$] | 80 | 55 | 75 |
| Repetitions | 3.94 (4.25) | 3.15 (5.1) | 2.55 (3.85) |
| Output Size | | | |
| public key sizes (bytes) | 1312.5 (1312) | 1952.5 (1952) | 2592.5 (2592) |
| signature sizes (bytes) | 5280 (2420) | 7773(3293) | 10355 (4595) |
| LWE Hardness | | | |
| BKZ blocksize b (GSA) | 422 | 622 | 860 |
| Classical Core-SVP (bits) | 123 | 181 | 251 |
| Quantum Core-SVP (bits) | 111 | 164 | 228 |
| SIS Hardness | | | |
| BKZ blocksize b | 417 | 602 | 868 |
| Classical Core-SVP (bits) | 121 | 176 | 253 |
| Quantum Core-SVP (bits) | 110 | 159 | 230 |

Table 4.1: Parameters, output sizes and security. The numbers in the parentheses are Dilithium parameters.

### 4.3.3 Number of Repetitions:

In this section, we compute the probability that the prover does not send $\perp$. That is, we want to show that the algorithm comes up with a valid signature. More formally, the following lemma follows.

**Lemma 7.** *If* $\beta \geq \max_{\boldsymbol{S} \in \boldsymbol{S}_\eta,\ c \in \boldsymbol{B}_\tau} \|c\boldsymbol{S}\|_\infty$ *then* $pr[(\boldsymbol{z}, \boldsymbol{h}) \neq \perp] \approx \exp(-\beta n(k+l+1)/\gamma_1)$ *in the line* 12 *of Algorithm 44.*

*Proof.* In line 12 of Algorithm 44, $(\mathbf{z}, \mathbf{h}) \neq \perp$ if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. And $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ implies that, for each coefficient $\sigma$ of $c\mathbf{S}$, the corresponding coefficient of polynomials in vector $\mathbf{z}$ inclusively lies between $-(\gamma_1 - \beta - 1)$ and $(\gamma_1 - \beta - 1)$. The corresponding coefficient of $\mathbf{y}_i$ is between

$-\gamma_1 + \beta + 1 - \sigma$ and $\gamma_1 - \beta - 1 - \sigma$. The size of this range is $2(\gamma_1 - \beta) - 1$ and the coefficient of $\mathbf{y}$ have total $2\gamma_1 - 1$ choices. Therefore, the probability that each coefficient of $\mathbf{y}$ lies in the good region is

$$\left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{n.(l+k+1)} = \left(1 - \frac{\beta}{\gamma_1 - 1/2}\right)^{n.(l+k+1)} \approx \exp^{-n.\beta(l+k+1)/\gamma_1}.$$

□

It is difficult to formally compute the probability that step 16 results in a restart. The parameters were set in Dilithium (so in this new signature scheme) such that heuristically $(\mathbf{z}, h) = \perp$ with a probability between 1 and 2%. So, most of the loop repetitions will be caused by line 12.

Brief ideas about other security estimates and parameters are given in Sections 4.5 and 4.6.

## 4.4 Zero-Knowledge Proof

Here we will show that outputting $\mathbf{z}$ such that $\mathbf{z} \in S^{k+l+1}_{\gamma_1-\beta-1}$ is not revealing any information about the secret $\mathbf{S}$. In other words, we will show that the probability distribution of such $\mathbf{z}$ is independent of the distribution of $\mathbf{S}$. We have

$$Pr[\mathbf{z} = \mathbf{y} + c\mathbf{S} \ \wedge C = c] = Pr[C = c]Pr[\mathbf{y} = \mathbf{z} - c\mathbf{S}|C = c]$$

Notice that, since $\|c\mathbf{S}\|_\infty \leq \beta$, so $\mathbf{z} - c\mathbf{S} \in S^{k+l+1}_{\gamma_1-1}$,which is valid value of $\mathbf{y}$. Thus, $Pr[\mathbf{y} = \mathbf{z} - c\mathbf{S}] = 1/|S^{k+l+1}_{\gamma_1-1}|$. Therefore, every $\mathbf{z} \in S^{k+l+1}_{\gamma_1-\beta-1}$ has an equal probability of being generated. And the probability of generating $\mathbf{z}$ and not returning $\perp$ is exactly $|S^{k+l+1}_{\gamma_1-\beta-1}|/|S^{k+l+1}_{\gamma_1-1}|$. Since $c$ comes randomly from $B_\tau$, the distribution of $(\mathbf{z}, c)$ is entirely random over $B_\tau \times S^{k+l+1}_{\gamma_1-\beta-1}$.

## 4.5 Security Reductions

Before starting with the security reductions, let us formally define the assumptions on which the security of our scheme is based.

### The MLWE Problem

For integers $m, k$, and a probability distribution $D : R_q \to [0, 1]$, we say that the advantage of algorithm A in solving the decisional $\mathbf{MLWE}_{m,k,D}$ problem over the ring $R_q$ is

$$\begin{aligned}\mathrm{Adv}^{\mathrm{MLWE}}_{m,k,D} := |\Pr[b = 1|\mathbf{A} \leftarrow R_q^{m\times k}; \mathbf{t} \leftarrow R_q^m; b \leftarrow \mathrm{A}(\mathbf{A}, \mathbf{t})] \\ - \Pr[b = 1|\mathbf{A} \leftarrow R_q^{m\times k}; s \leftarrow D^k; e \leftarrow D^m; b \leftarrow \mathrm{A}(\mathbf{A}, \mathbf{As} + \mathbf{e})]|.\end{aligned}$$

### The MSIS Problem

To an algorithm A we associate the advantage function $\mathrm{Adv}^{\mathrm{MSIS}}_{m,k,\gamma}$ to solve the $\mathbf{MSIS}_{m,k,\gamma}$ problem over the ring $R_q$ as

$$\mathrm{Adv}^{\mathrm{MSIS}}_{m,k,\gamma}(\mathrm{A}) := \Pr[0 \leq \|\mathbf{y}\|_\infty \leq \gamma \wedge [\mathbf{I}|\mathbf{A}] \cdot \mathbf{y} = \mathbf{0}|\mathbf{A} \leftarrow R_q^{m\times k}; \mathbf{y} \leftarrow \mathrm{A}(\mathbf{A})].$$

**The SelfTargetMSIS Problem**

Let $\mathrm{H} : \{0,1\}^* \to B_\tau$ be a cryptographic hash function. To an algorithm A we associate the advantage function

$$\mathrm{Adv}^{\mathrm{SelfTargetMSIS}}_{\mathrm{H},m,k,\gamma}(\mathrm{A}) := \Pr\left[0 \le \|\mathbf{y}\|_\infty \le \gamma \wedge \mathrm{H}(\mu||[\mathbf{I}|\mathbf{A}]\cdot\mathbf{y}) = c | \mathbf{A} \leftarrow R_q^{m\times k}; \left(\mathbf{y} := \begin{pmatrix}\mathbf{r}\\ c\end{pmatrix}, \mu\right) \leftarrow \mathrm{A}^{|\mathrm{H}(\cdot)\rangle}(\mathbf{A})\right]$$

### 4.5.1 Key-Recovery Attack: Solving Module-LWE

In the Key Generation algorithm 48 of our scheme, an MLWE instance $(\mathbf{A}, \mathbf{t} = \mathbf{As}+\mathbf{e})$ is generated. Then $(\mathbf{A}, \mathbf{t})$ is published as the public key and $(\mathbf{s}, \mathbf{e})$ is kept as the secret key by the signer. Now, we know that, given an MLWE instance, it is equally hard to solve both the Search (SLWE) and the Decision(DLWE) versions of the problem. Thus, recovery of the secret key $(\mathbf{s}, \mathbf{e})$ boils down to solving either of these two versions of the problem.

### 4.5.2 Forgery Attack: Solving MSIS/SelfTargetMSIS problem

The standard security notion for digital signatures is UF-CMA security, which is security under chosen message attacks. In this security model, the adversary gets the public key and accesses a signing oracle to sign messages of his/her choice. The adversary aims to develop a valid signature of a new message. A slightly stronger security requirement that is sometimes useful is SUF-CMA, which is strong unforgeability under a chosen message attack. In the latter case, the adversary has to develop a new signature of an already-seen message. In this section, we will prove that our signature scheme is SUF-CMA secure based on the hardness of the standard MLWE and MSIS lattice problem.

**Lemma 8.** *Forging a signature implies finding* $\boldsymbol{v}$ *and* $\boldsymbol{u}$ *such that* $\|\boldsymbol{v}\|_\infty \le \max(2(\gamma_1-\beta), 2)$ *and* $\|\boldsymbol{u}\|_\infty \le 4\gamma_2+2$ *such that* $\boldsymbol{A'v}+\boldsymbol{u} = \boldsymbol{0}$ *with* $(\boldsymbol{v},\boldsymbol{u}) \neq \boldsymbol{0}$*. Furthermore,* $\boldsymbol{u}$ *has at most* $2\omega$ *coefficients of absolute value greater than* $2\gamma_2$*.*

*Proof.* If the adversary creates a valid signature $(\mathbf{z}, \mathbf{h}, c)$ for a message $\mu$, then the adversary must have queried $H(\mu||\mathbf{w}_1) = c$ where $\mathbf{w}_1 = \mathrm{UseHint}_q(\mathbf{h}, \mathbf{A'z}, 2\gamma_2)$, either directly by querying H or indirectly signature oracle.

**Case 1**: If $c$ is queried during a signing query, then the forger can output another valid forgery $(\mathbf{z'}, \mathbf{h'}, c)$ with $c = H(\mu||\mathbf{w}'_1)$ where $\mathbf{w}'_1 = \mathrm{UseHint}_q(\mathbf{h'}, \mathbf{A'z}, 2\gamma_2)$. This implies $\mathbf{w}_1 = \mathbf{w}'_1$, or we found a 2nd preimage for $c$.
Since $\mathbf{w}_1 = \mathbf{w}'_1$,

$$\mathrm{UseHint}(\mathbf{h}, \mathbf{A'z}, 2\gamma_2) = \mathbf{w}_1$$
$$\mathrm{UseHint}(\mathbf{h'}, \mathbf{A'z'}, 2\gamma_2) = \mathbf{w}_1$$

If $\mathbf{z} = \mathbf{z'}$, then from lemma 5 says that $\mathbf{h} = \mathbf{h'}$, which contradicts our assumption that the adversary has found a new signature. Thus $\mathbf{z} \neq \mathbf{z'}$ and

$$\|\mathbf{A'z} - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \le 2\gamma_2 + 1$$
$$\|\mathbf{A'z'} - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \le 2\gamma_2 + 1$$

By the triangle inequality, we have:

$$\|\mathbf{A}'(\mathbf{z}-\mathbf{z}')\|_\infty \leq \|\mathbf{A}'\mathbf{z}-\mathbf{w}_1\cdot 2\gamma_2\|_\infty + \|\mathbf{A}'\mathbf{z}'-\mathbf{w}_1\cdot 2\gamma_2\|_\infty \leq 4\gamma_2+2$$

So there exists a vector $\mathbf{u}$ such that $\|\mathbf{u}\|_\infty \leq 4\gamma_2+2$ and $\mathbf{v}=(\mathbf{z}-\mathbf{z}') \neq \mathbf{0}$ such that $\|\mathbf{v}\|_\infty \leq \max(2(\gamma_1-\beta),2)$. Since $h$ and $h'$ have all expect $\omega$ coefficients equal to 0, so from lemma5 we say that all but $\omega$ coefficients of $\mathbf{A}'\mathbf{z}-\mathbf{w}_1\cdot 2\gamma_2$ and $\mathbf{A}'\mathbf{z}'-\mathbf{w}_1\cdot 2\gamma_2$ are less than $\gamma_2$. So all but $2\omega$ coefficients of $\mathbf{u}$ are less than $2\gamma_2$.

**Case 2**: We now do the proof where the query was done directly to H. By the forking lemma[76], there are two signatures $(\mathbf{z},\mathbf{h},c)$ and $(\mathbf{z}',\mathbf{h}',c')$ for $c\neq c'$ for the same commitment such that:

$$\begin{aligned}&\text{UseHint}(\mathbf{h},\mathbf{A}\mathbf{z}_1+\mathbf{z}_2-c\cdot 2^d\mathbf{t}_1,2\gamma_2)=\mathbf{w}_1\\&\text{UseHint}(\mathbf{h}',\mathbf{A}\mathbf{z}_1'+\mathbf{z}_2'-c'\cdot 2^d\mathbf{t}_1,2\gamma_2)=\mathbf{w}_1\end{aligned}$$

By lemma5,we have:

$$\begin{aligned}&\|\mathbf{A}\mathbf{z}_1+\mathbf{z}_2-c\cdot 2^d-\mathbf{w}_1\cdot 2\gamma_2\|_\infty \leq 2\gamma_2+1\\&\|\mathbf{A}\mathbf{z}_1'+\mathbf{z}_2'-c'\cdot 2^d-\mathbf{w}_1\cdot 2\gamma_2\|_\infty \leq 2\gamma_2+1\end{aligned}$$

$\|\mathbf{A}\mathbf{z}_1+\mathbf{z}_2-c\cdot 2^d-\mathbf{A}\mathbf{z}_1'-\mathbf{z}_2'+c'\cdot 2^d\|_\infty \leq \|\mathbf{A}\mathbf{z}_1+\mathbf{z}_2-c\cdot 2^d-\mathbf{w}_1\cdot 2\gamma_2\|_\infty+\|\mathbf{A}\mathbf{z}_1'+\mathbf{z}_2'-c'\cdot 2^d-\mathbf{w}_1\cdot 2\gamma_2\|_\infty \leq 4\gamma_2+2$
So,

$$\begin{aligned}&\|\mathbf{A}(\mathbf{z}_1-\mathbf{z}_1')+(\mathbf{z}-\mathbf{z}')+(c'-c)\cdot 2^d\mathbf{t}_1\|_\infty \leq 4\gamma_2+2\\&\implies (\mathbf{A}|\mathbf{I}_k|-2^d\mathbf{t}_1)\begin{pmatrix}\mathbf{z}_1-\mathbf{z}_1'\\ \mathbf{z}_2-\mathbf{z}_2'\\ c-c'\end{pmatrix}+\mathbf{u}=\mathbf{0}\\&\implies \mathbf{A}'\mathbf{v}+\mathbf{u}=\mathbf{0}\end{aligned}$$

where $\|\mathbf{v}\|_\infty \leq \max(2(\gamma_1-\beta),2)$ and $\|\mathbf{u}\|_\infty \leq 4\gamma_2+2$. For the same reason as in the first case, at most $2\omega$ coefficients of $\mathbf{u}$ can be greater than $2\gamma_2$. □

From the above lemma, the adversary has to solve a homogeneous MSIS problem for the matrix $\mathbf{A}'$ with infinity norm bounds $\gamma'=\max(2(\gamma_1-\beta),4\gamma_2+2,2)$ to create a new signature of an already-seen message. That is how we can say our signature scheme is SUF-CMA. Our signature is Zero-Knowledge Proof, which is why, during the chosen message attack, an adversary gets no information about the secret key. Since performing a key-recovery attack is reduced to solving MLWE, the adversary cannot extract any information about the secret key from the public key.

Now consider the case where an adversary has quantum access to H and classical access to a signing oracle. If it can produce a forgery of a new message, then there is also an adversary who can produce a forgery without access to the signing oracle, while the adversary only gets the public key. This latter security model is called UF-NMA − unforgeability under no-message attack. To prove UF-NMA security, we only analyze the hardness of the experiment where the adversary only gets a public key $(\mathbf{A},\mathbf{t})$. Practically, an adversary will get $\mathbf{t}_1$, in which case the attack will be more difficult.

**Lemma 9.** *Forging a signature so that the adversary has quantum access to H, and he/she has to create a valid signature for at least one message without any examples of legitimate signatures. Then, the adversary needs to output a valid message and signature pair $M,(\boldsymbol{z},h,c)$ so that:*

1. $\|\boldsymbol{z}\|_\infty \leq \gamma_1 - \beta$
2. $H(\mu||\,\mathrm{UseHint}_q(\boldsymbol{h}, \boldsymbol{A'z}, 2\gamma_2)) = c$
3. *Number of* $1's$ *in* $\boldsymbol{h}$ *is* $\leq \omega$

*and this turns into solving a* $\mathrm{SelfTargrtMSIS}$ *problem.*

*Proof.* From lemma5 we can say that

$$2\gamma_2 \cdot \mathrm{UseHint}_q(\mathbf{h}, \mathbf{A'z}, 2\gamma_2) = \mathbf{A'z} + \mathbf{u} \tag{4.4}$$

where $\|\mathbf{u}\| \leq 2\gamma_2 + 1$. Notice that only $\omega$ coefficients of $\mathbf{u}$ will have magnitude greater than $\gamma_2$. We have $\mathbf{t} = \mathbf{t}_1 2^d + \mathbf{t}_0$ with $\|\mathbf{t}_0\| \leq 2^{d-1}$, then from equation (4.4) we have,

$$\mathbf{A'z} + \mathbf{u} = (\mathbf{A}|\mathbf{I}_k| - 2^d\mathbf{t}_1)\begin{pmatrix}\mathbf{z}_1\\ \mathbf{z}_2\\ c\end{pmatrix} + \mathbf{u} = \mathbf{A}\mathbf{z}_1 + \mathbf{z}_2 - c\cdot 2^d\mathbf{t}_1 + \mathbf{u} = \mathbf{A}\mathbf{z}_1 + \mathbf{z}_2 - c\mathbf{t} + \mathbf{u'} \tag{4.5}$$

where $\mathbf{u'} = c\mathbf{t}_0 + \mathbf{u}$.

So,

$$\|\mathbf{u'}\|_\infty \leq \|c\mathbf{t}_0\|_\infty + \|\mathbf{u}\|_\infty \leq \|c\|_1 \cdot \|\mathbf{t}_0\|_\infty + \|\mathbf{u}\|_\infty \leq \tau \cdot 2^{d-1} + 2\gamma_2 + 1$$

So an adversary(quantum) who is successful in forging against a new message is able to find $\mathbf{z}, c, \mathbf{u'}, M$ such that $\|\mathbf{z}\|_\infty \leq \gamma_1 - \beta, \|c\|_\infty = 1, \|\mathbf{u'}\|_\infty \leq \tau \cdot 2^{d-1} + 2\gamma_2 + 1$ and $M \in \{0,1\}^*$ such that

$$\mathrm{H}\left(\mu||\frac{1}{2\gamma_2}\cdot[\mathbf{A}|\mathbf{I}_k| - \mathbf{t}|\mathbf{I}_k]\begin{pmatrix}\mathbf{z}_1\\ \mathbf{z}_2\\ c\\ \mathbf{u'}\end{pmatrix}\right) = c \tag{4.6}$$

So, equivalently, we can say that if we assume $\mathrm{H}(\mu||x) = \mathrm{H'}(\mu||2\gamma_2 x)$, then we have

$$\mathrm{H'}\left(\mu||[\mathbf{A}|\mathbf{I}_k| - \mathbf{t}|\mathbf{I}_k]\begin{pmatrix}\mathbf{z}_1\\ \mathbf{z}_2\\ c\\ \mathbf{u'}\end{pmatrix}\right) = c \tag{4.7}$$

This is exactly the definition of the $\mathrm{SelfTargetMSIS}$ problem. To solve $\mathrm{SelfTargetMSIS}$one has to pick a $\mathbf{w}$ and compute $\mathrm{H'}(\mu||\mathbf{w}) = c$ and then find $\mathbf{z}, \mathbf{u'}$ such that

$$[\mathbf{A}|\mathbf{I}_k]\mathbf{z} + \mathbf{u'} = c\mathbf{t} + \mathbf{w} = \mathbf{t'} \tag{4.8}$$

with the conditions $\|\mathbf{z}\|_\infty \leq \gamma_1 - \beta, \|c\|_\infty = 1, \|\mathbf{u'}\|_\infty \leq \tau \cdot 2^{d-1} + 2\gamma_2 + 1$. □

While such reductions are good for a "sanity check", they are not particularly useful for setting parameters. So, how does one set parameters? So far, the well-known way of parameter setting is by trying to solve MLWE and MSIS problems and noting the conditions for which such solutions are hard to find. In our parameter setting, we mainly consider the lattice-based way of solving. In Section 4.6, we describe the best-known lattice attacks and give a brief idea about how they are used in setting the parameters of our scheme.

## 4.6 Concrete Security Analysis via Lattice Reduction

As described in the previous section, both secret key recovery and signature forgery can be prevented by reducing them to well-known hard problems. However, such reductions do not provide enough insight into setting parameters. For that, we need to try to solve the aforementioned hard problems, that is, find short vectors in a lattice. To find short vectors, various lattice reduction methods [59, 82, 25, 24, 88] can be applied.

### 4.6.1 Lattice Reduction and Core-SVP Hardness

As of now, the best-known state-of-the-art algorithm used for finding very short non-zero vectors in Euclidean lattices is the Block-Korkine-Zolotarev algorithm (BKZ) [82], proposed by Schnorr and Euchner in 1991 and later improved by Y.Chen [24, 25]. Both the improved and original versions achieve the same result asymptotically.

The BKZ algorithm with block size $b$ makes calls to an algorithm (SVP oracle) that solves the Shortest Vector Problem (SVP) in a lattice of dimension $b$ to reduce a lattice basis. It is in the literature that the number of calls to that oracle remains polynomial, yet concretely evaluating the number of calls is rather painful. So we choose to ignore this polynomial factor and rather evaluate only the *core SVP hardness*, the cost of *one call* to an SVP oracle in dimension $b$. The security of our scheme relies on the necessity to run BKZ with a large block size $b$ and the fact that the *core SVP hardness* is exponential in $b$. The best known classical SVP solver [15] runs in time $\approx 2^{c_C \cdot b}$ with classical cost$(c_C) = \log_2 \sqrt{3/2} \approx 0.292$ and provides $0.292.b$ classical bit security. The best known quantum SVP solver [57] runs in time $\approx 2^{c_Q \cdot b}$ with quantum cost$(c_Q) = \log_2 \sqrt{13/9} \approx 0.265$ and similarly provides $0.265.b$ quantum bit security. One may hope to improve these runtimes, but going below $\approx 2^{c_P \cdot b}$ with plausible cost$(c_P) = \log_2 \sqrt{4/3} \approx 0.2075$ would require a theoretical breakthrough. Indeed, the best known SVP solvers rely on covering the $b$-dimensional spheres with a cone of centre-to-edge angle $\pi/3$: this requires $2^{c_P \cdot b}$ cones.

The strength of BKZ increases with $b$. More concretely, given as input a basis $(\mathbf{b}_1, \ldots, \mathbf{b}_n)$ of an n-dimensional lattice, BKZ repeatedly uses the $b$-dimensional SVP-solver on projected lattices generated by $B_{[i,j]} = (\pi_i(\mathbf{b}_i), \pi_i(\mathbf{b}_{i+1}),$
$\ldots, \pi_i(\mathbf{b}_j))^T$, where $i \leq n, j = \min(n, i+b)$ and where $\pi_i(\mathbf{b}_k)$ denotes the projection of $\mathbf{b}_k$ orthogonally to the vectors $(\mathbf{b}_1, \ldots, \mathbf{b}_{i-1})$. Since any lattice reduction must preserve the lattice volume, Vol$(\Lambda(B_{[i,j]})) = \prod_{k=i}^{j} \|\mathbf{b}_k^*\|$ should remain constant throughout. Thus BKZ preserves the sum of the $\ell_i's$ ,where $\ell_i = \log_2 \|b_i^*\|$, for $i = 1, \ldots, n$ throughout the execution.

Now, how does this help in estimating security? Do we apply the BKZ algorithm? No, because in order to make our scheme secure, the cost of solving SVP (core SVP hardness) must be high; that is block size should be so high that it is infeasible to solve the SVP. So, instead of running BKZ directly, we model the behaviour of BKZ using the geometric series assumption(which is known to be optimistic from the attacker's point of view) [See Appendix 4.8]. In short, BKZ decreases the $\ell_i's$ with small $i's$ and increases $\ell_i's$ with large $i's$, since $\sum \ell_i$ should remain constant, in a way flattening the curve of $\ell_i$ to coincide with the GSA plot [34].

While the MLWE and MSIS problems are defined over polynomial rings, currently, there is no way of exploiting this ring structure, and therefore, the best attacks are mounted by simply viewing these problems as LWE and SIS problems, as is done in the following sections.

### 4.6.2 Solving MLWE

Any $\text{MLWE}_{\ell,k,D}$ instance for some distribution $D$ can be viewed as a classical LWE instance of dimensions $256.\ell$ and $256.k$, where 256 is the degree of each polynomial entry of $\mathbf{A}_{k\times\ell}$. Thus, the MLWE instance $(\mathbf{A}, \mathbf{t} = \mathbf{As} + \mathbf{e})$ can be rewritten as an LWE instance as
$(rot(\mathbf{A}), vec(\mathbf{t}) = rot(\mathbf{A}).vec(\mathbf{s}) + vec(\mathbf{e}))$, where $vec(\cdot)$ maps a vector of ring elements to the vector obtained by concatenating the coefficients of its coordinates, and $rot(\mathbf{A}) \in \mathbb{Z}_q^{256.k\times 256.\ell}$ is obtained by replacing all entries $a_{ij} \in R_q$ of $\mathbf{A}$ by the $256 \times 256$ circulant matrix whose $z$-th column is $vec(x^{z-1}.a_{ij})$. Here, we are mainly concerned about the lattice reduction attacks, the Primal and Dual attacks.

#### Primal Attack

This attack gives a method of solving the search version of the $\text{LWE}_{256.\ell,256.k}$ problem, which can be restated as finding $(vec(\mathbf{s}), vec(\mathbf{e})) \in \mathbb{Z}^{256.\ell} \times \mathbb{Z}^{256.k}$ from $(rot(\mathbf{A}), vec(\mathbf{t}))$. Thus, the primal attack is finding a short non-zero vector in the lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}^d : \mathbf{Mx} = \mathbf{0} \bmod q\}$, where $\mathbf{M} = \begin{pmatrix} rot(\mathbf{A})_{[1:m]} | \mathbf{I}_m | vec(\mathbf{t})_{[1:m]} \end{pmatrix}$ is an $m \times d$ matrix where $d = 256.\ell + m + 1$ and $m \leq 256.k$. Indeed, it is sometimes not optimal to use all the given samples in lattice attacks.

To date, there are two well-known ways to reduce LWE to a lattice problem. One way is to reduce the LWE problem to a BDD problem on a lattice and further reduce it to a uSVP problem on a 1-dimensional higher lattice using Kannan's embedding technique [49]. Another way similarly reduces the LWE problem to an ISIS problem, then to a CVP problem on a lattice and further to a uSVP problem on a 1-dimensional higher lattice using the Bai-Galbraith embedding technique [13]. Thus, to establish the cost of solving an LWE instance, we may consider the cost of lattice reduction for solving uSVP. Two conflicting estimates [7] for the success of lattice reduction in solving uSVP are available in the literature. The first is going back to [41] and was developed in [6, 8] for LWE. The second estimate was recently outlined in [9] and is relied upon in [28] and many other recently developed schemes. We will use the shorthand *2008 estimate* for the former and *2016 estimate* for the latter. Following the footsteps of Dilithium, we use the Bai-Galbraith embedding technique along with the *2016 estimate* in the security analysis of our scheme.

The final lattice obtained by the second reduction approach [13] is the same as the primal attack lattice $\Lambda$ defined above. Now, the 2016 estimate [9, 7] predicts that a unique short vector $\mathbf{x}$ of the lattice $\Lambda$ can be successfully found if

$$\sqrt{b/d}\|(\mathbf{x})\| \approx \sqrt{b}\sigma \leq \delta_0^{2b-d}\text{Vol}(\Lambda(\mathbf{B}))^{1/d} \tag{4.9}$$

under the assumption that GSA holds (i.e., the basis is BKZ-$b$ reduced). Here, $\delta_0^{2b-d}\text{Vol}(\Lambda(\mathbf{B}))^{1/d} = \|\mathbf{b}^*_{d-b+1}\|$ (by def 4) and $\sqrt{b}\sigma$ is the expected length of $\pi_{d-b+1}(\mathbf{x})$. Thus, if the expected norm of $\mathbf{x}$ after projection orthogonally to the first $d - b$ vectors is less than $2^{\ell_{d-b}}$, it is likely that the MLWE solution can be easily extracted from the BKZ output. So, we tried all possible numbers $m$ of rows, and for each trial, we increased the block size $b$ and continued modelling the behaviour of BKZ (Sec. 4.6.1) until the above estimate held true.

#### Dual Attack

The dual attack is finding a short non-zero vector in the dual q-ary lattice $\Lambda' = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^d : \mathbf{M}^T\mathbf{x} = \mathbf{y} \bmod q\}$, $\mathbf{M} = \begin{pmatrix} rot(\mathbf{A})_{[1:m]} \end{pmatrix}$ is an $m \times d$ matrix where $d = 256.\ell$ and $m \leq 256.k$.

Now, let us see how finding such a short vector $(\mathbf{r}_1, \mathbf{r}_2)$ in the lattice $\Lambda'$ solves the DLWE. Since, $(\mathbf{r}_1, \mathbf{r}_2) \in \Lambda'$, we have $\mathbf{M}^T\mathbf{r}_1 = \mathbf{r}_2 \bmod q$. Compute $\mathbf{r}_1^T vec(\mathbf{t})_{[1:m]}$. Here, either of the following

two cases may arise.
**Case 1:** If $vec(\mathbf{t}) \xleftarrow{\$} \mathbb{Z}^m$, then $\mathbf{r}_1^T vec(\mathbf{t})$ will also look uniformly random in $\mathbb{Z}^m$.
**Case 2:** If $vec(\mathbf{t})_{[1:m]} \leftarrow$ LWE distribution, that is, $vec(\mathbf{t})_{[1:m]} = \mathbf{M}.vec(\mathbf{s}) + vec(\mathbf{e})_{[1:m]}$, then, $\mathbf{r}_1^T.vec(\mathbf{t})_{[1:m]} = \mathbf{r}_2^T vec(\mathbf{s}) + \mathbf{r}_1^T vec(\mathbf{e})_{[1:m]}$.Since $(vec(\mathbf{s}), vec(\mathbf{e}))$ are short LWE secret vectors and $(\mathbf{r}_1, \mathbf{r}_2)$ are short vectors in $\Lambda'$, $\mathbf{r}_1^T.vec(\mathbf{t})_{[1:m]}$ must be short.
Thus, by computing $\mathbf{r}_1^T.vec(\mathbf{t})_{[1:m]}$ and observing it, we can tell if $vec(\mathbf{t})$ comes from an LWE or uniform distribution, thus solving DLWE.

Assume the short vector $(\mathbf{r}_1, \mathbf{r}_2)$ is of length $\ell$, then for short $\mathbf{s}, \mathbf{e}$ normally distributed of standard deviation $\sigma$, we have standard deviation of $\mathbf{r}_2^T vec(\mathbf{s}) + \mathbf{r}_1^T vec(\mathbf{e})_{[1:m]}$ as $\ell\sigma$. Thus, in Case 2, $vec(\mathbf{t})_{[1:m]}$ will have standard deviation $\ell\sigma$, but Decision-LWE cannot be solved only when $vec(\mathbf{t})_{[1:m]}$ appears to be uniformly distributed. It is known [62] that if we have a normally-distributed random variable with standard deviation greater than $\sqrt{3}q$ and we reduce it modulo $q$, then the result is statistically-close (within $\approx 2^{-80}$) to the uniform distribution. Therefore, our LWE will be secure from dual attack whenever,

$$\ell\sigma > \sqrt{3}q \tag{4.10}$$

As in the case of primal attack, here also, for each value of $m$, we increased the value of $b$ until the $\ell = \ell_1 = \|\mathbf{b}_1^*\|$, where $\mathbf{b}_1$ is the first vector of the BKZ reduced basis.

### 4.6.3 Solving MSIS and SelfTargetMSIS

The best-known attack against the $\mathrm{SelfTargetMSIS}$problem involves either breaking the security of $\mathrm{H}'$ or solving the $\mathbf{MSIS}_{k,k+l+1,\gamma}$ where $\gamma = \max(\gamma_1 - \beta, \tau \cdot 2^{d-1} + 2\gamma_2 + 1)$ corresponding to the matrix $[\mathbf{A}|\mathbf{I}_k| - \mathbf{t}']$. To date, there is no study where one can exploit the ring/module structure to solve **MSIS** instance. So the attacker will convert $\mathbf{MSIS}_{k,k+l+1,\gamma}$ to $\mathbf{SIS}_{256\cdot k,256\cdot(k+l+1),\gamma}$ by considering $\mathrm{rot}(\mathbf{A}|\mathbf{I}_k| - \mathbf{t}') \in \mathbb{Z}_q^{256\cdot k \times 256\cdot(k+l+1)}$. Similary the attack aginst $\mathbf{MSIS}_{k,k+l,\gamma'}$ instance in 8 can be mapped in to $\mathbf{SIS}_{256\cdot k,256\cdot(k+l),\gamma'}$ by considering the matrix $\mathrm{rot}(\mathbf{A}|\mathbf{I}_k) \in \mathbb{Z}^{256\cdot k \times 256\cdot(k+l)}$.

## 4.7 Supporting Algorithms

We output signature $(\mathbf{z}, \tilde{c}, \mathbf{h})$ when the number of 1's in $\mathbf{h} \leq \omega$. In the line 15 of our signature scheme 5.0.1, the hint vector $\mathbf{h}$ is generated using $\mathrm{MakeHint}_q$, where both $-bc\mathbf{t}_0, \mathbf{w} + bc\mathbf{t}_0 \in R_q^k$, thus $\mathbf{h} \in R_q^k$. To optimize the size of the signature, we encode the coefficients of the hint vector $\mathbf{h} \in R_q^k$ into a byte string $y$ of length $\omega + k$ in HintBitPack and reverse the process to recover the original hint vector $\mathbf{h}$ in HintBitUnpack. The last $k$ bytes of $y$ contain information about how many non-zero coefficients are present in each of the polynomials $\mathbf{h}[0], \mathbf{h}[1], \ldots, \mathbf{h}[k-1]$ respectively and the first $\omega$ bytes of $y$ contain information about the exact indices of $\mathbf{h}[i], i \in \{0, \ldots, k\}$ where those non-terms occur.

**Algorithm 46:** HintBitPack($\mathbf{h}$)

$y \in B^{\omega+k} \leftarrow 0^{\omega+k}$
$k \leftarrow 0$
**for** $i = 0$ **to** $k-1$ **do**
  **for** $j = 0$ **to** $255$ **do**
    **if** $\boldsymbol{h}[i] \neq 0$ **then**
      $y[k] \leftarrow j$
      $k \leftarrow k+1$
  $y[\omega+i] \leftarrow k$
**return** $y$

**Algorithm 47:** HintBitUnpack($y$)

$\mathbf{h} \in R_2^k \leftarrow 0^k$
$k \leftarrow 0$
**for** $i = 0$**to** $k-1$ **do**
  **if** $y[\omega+1] < k$ ***or*** $y[\omega+i] > \omega$ **then**
    **return** $\perp$
  **while** $k < y[\omega+i]$ **do**
    $\mathbf{h}[i]_{y[k]} \leftarrow 1$
    $k \leftarrow k+1$
**while** $k < \omega$ **do**
  **if** $y[k] \neq 0$ **then**
    **return** $\perp$
  $k \leftarrow k+1$
**return** $\mathbf{h}$

## 4.8 Constructing basis profile

The BKZ algorithm is a lattice reduction algorithm that takes as input a basis $\mathbf{B}$ of a lattice $\Lambda$ and reduces it to a better basis of $\Lambda$. For our cryptanalytic purpose, we are interested in the properties of the profile after lattice reduction.

**Definition 1** (Basis Profile)**.** *Given a basis* $\boldsymbol{B} \in \mathbb{R}^{d\times m}$ *its profile is the tuple* $(\ell_i)_{i=1}^m \in \mathbb{R}^m$*, where* $\ell_i = \log\|\boldsymbol{b}_i^*\|$.

Often, we consider lattices of a particular form.

**Definition 2** ($q$-ary lattice)**.** *For some* $q \in \mathbb{Z}_{>0}$*, a rank* $m$ *lattice* $\Lambda$ *is a* $q$*-ary lattice if* $q\mathbb{Z}^m \subseteq \Lambda \subseteq \mathbb{Z}^m$.

Solution to the MLWE and MSIS problems (as discussed in Sec **??** and 4.5) can be achieved by reducing to a problem of finding short vector in q-ary lattices of the form $\Lambda_q^\perp(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{Ax} = \mathbf{0} \bmod q\}$. If one can permute the columns of $\mathbf{A}$ to $(\mathbf{A}_1|\mathbf{A}_2)$ such that $\mathbf{A}_1 \in \mathrm{Gl}_n(\mathbb{Z}_q)$ then one can form the basis

$$\mathbf{B_A} = \begin{bmatrix} q\mathbf{I}_n & -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ \mathbf{0} & \mathbf{I}_{m-n} \end{bmatrix}$$

of $\Lambda_q^\perp(\mathbf{A})$. As explained in section 4.5, in order to find a short vector in some lattice, we need to BKZ-reduce a basis like $\mathbf{B_A}$ until they follow the Geometric Series Assumption, as can be seen in Figure 4.1. Before going into detail, let's define some important quantities needed to get an overview of how we model the behaviour of BKZ-$b$ for our security estimates.

**Definition 3** (Root Hermite factor)**.** *For* $b \geq 50$ *let*

$$\delta_0 = \left(\frac{b}{2\pi e}(\pi b)^{1/b}\right)^{1/2(b-1)} \tag{4.11}$$

For smaller values of $b$, the root hermite factor $\delta_b$ is determined experimentally. After BKS-$b$ reduction on a basis $\mathbf{B} \in \mathbb{R}^{d\times m}$ of a lattice $\Lambda$ , we estimate $\|\mathbf{b}_1\| \approx \delta_0^m.\text{Vol}(\Lambda)^{1/m}$. Thus $\delta_0$ can be used to convey the quality of a reduced basis. The other heuristic we use is the Geometric Series Assumption (GSA) [83]. This asserts that after lattice reduction, the Gram-Schmidt norms decrease as a geometric series.

**Definition 4** (Geometric Series Assumption(GSA))**.** *After lattice reduction on a basis* $\boldsymbol{B} \in \mathbb{R}^{d\times m}$ *there exists* $\alpha \in (0, 1)$ *such that for* $1 \leq i \leq m$*, we have* $\|\boldsymbol{b}_i^*\| = \alpha^{i-1}\|\boldsymbol{b}_1\|$.

Given a basis $\mathbf{B}^{d\times m}$ of lattice $\Lambda$, and assuming both $\|\mathbf{b}_1\| = \delta_0^m.\text{Vol}(\Lambda)^{1/m}$ and the GSA after BKZ-$b$ reduction, since $\text{Vol}(\Lambda) = \prod_i \|\mathbf{b}_i^*\| = \|\mathbf{b}_1\|^m(\alpha \dots \alpha^{m-1}) = \|\mathbf{b}_1\|^m \alpha^{\frac{m(m-1)}{2}}$, we have $\alpha \approx \delta_0^{-2}$.

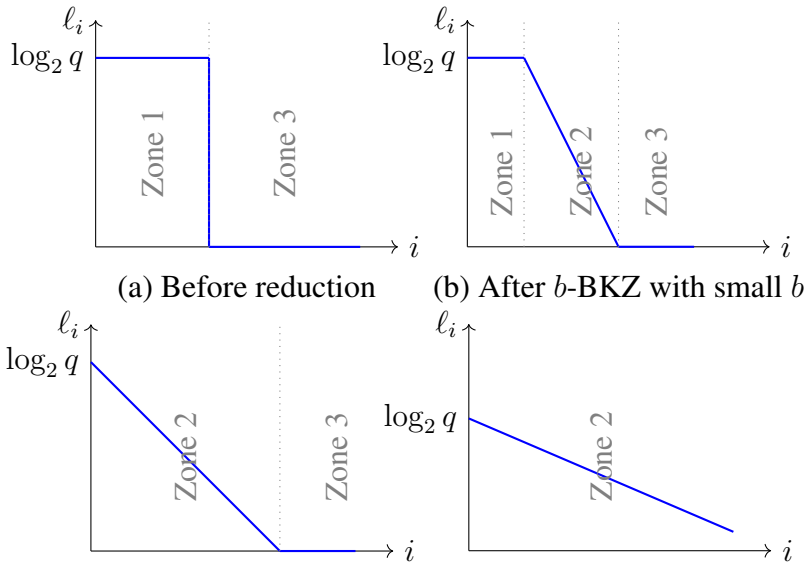



Figure 4.1: **Evolution of Gram-Schmidt length in log-scale under BKZ reduction for various block sizes.** The area under the curves remains constant, and the slope in Zone 2 decreases with the block size. Note that Zone 3 may disappear before Zone 1, depending on the shape of the input basis.

For our final assumption, we specialise to q-ary lattices, specifically those of the form $\Lambda_q^{\perp}(\mathbf{A})$ for $\mathbf{A} \in \mathbb{Z}^{n\times m}$ with basis $\mathbf{B_A} \in \mathbb{Z}^{m\times m}$, as discussed above. So, for $\mathbf{B_A} = [\mathbf{b}_1, \dots \mathbf{b}_m]$, we have $\ell_i = \log_2\|\mathbf{b}_i^*\| = \log_2\|q.\mathbf{e}_i\| = \log_2 q$ for $i \in \{1, \dots, n\}$ and $\ell_i = \log_2\|\mathbf{b}_i^*\| = \log_2\|\mathbf{e}_i\| = 0$ for $i \in \{n + 1, \dots, m\}$. Thus, the basis profile of $\mathbf{B_A}$ (Before reduction) is given by Figure 4.1(a).

Under the root Hermite factor and GSA heuristics, we assume that after BKZ-$b$ reduction $\|\mathbf{b}_i^*\| = \delta_0^m\text{Vol}(\Lambda)^{1/m}.\alpha^{i-1} = \|\mathbf{b}_1\|\delta_0^{-2(i-1)}$. From here, we have $\|\mathbf{b}_{d-b+1}^*\| = \delta_0^{2b-d}\text{Vol}(\Lambda(\mathbf{B}))^{1/d}$(4.9) (in this section $m = d$). Now, by eq. 4.11, it can be heuristically estimated that for sufficiently large $b$, the local slope of the $\ell_i's$ (basis profile plot) converges to,

$$slope(b) = 2.\log(\delta_0) = \frac{1}{b-1}\log_2\left(\frac{b}{2\pi e}(\pi.b)^{1/b}\right) \tag{4.12}$$

After lattice reduction, the basis profile of a BKZ-$b$ reduced basis is given by Figure 4.1(d). So, what is meant by modelling the behaviour of BKZ is doing whatever is necessary to reduce the

basis profile from Figure 4.1(a) to the GSA plot, keeping in mind that the volume of the lattice is preserved, that is, the area under the curve remains constant. Now, as can be predicted from the expression of the slope from eq 4.12, $slope(b)$ decreases with $b$, implying that the larger $b$, the flatter the output $\ell_i's$ (Figure 4.1). Thus, as explained in Section 4.5, we keep on increasing $b$ until the GSA plot is achieved and consider that $b$ to be the optimal BKZ blocksize to achieve the required bit security.

# Milestone 5

# Analysis of proposed Digital Signature Algorithm

In the Milestone 3, we have fixed the mathematical basis of our digital signature scheme to be the hardness of solving the module learning with error (MLWE) problem. Based on the intractability of MLWE, we have come up with a post-quantum digital signature scheme in the Milestone 4. As the Dilithium digital signature scheme, which is currently the state-of-the-art digital signature algorithm, is also based on the same mathematical principle, the way of generating keys (public and secret) and sampling elements of the modules in our proposed algorithm have similarities. We have further kept the same notations for ease of understanding. Below, we present our proposed signature scheme again for completeness. In the description of the algorithm below, we slightly tweak the notations to make its description is in line with our implementation.

**ALGORITHM 5.0.1: Signature Scheme**

**Algorithm 48:** $\texttt{GEN}(k, \ell)$

$\zeta \leftarrow \{0,1\}^{256}$

$(\rho, \rho', K) \in \{0,1\}^{256} \times \{0,1\}^{512} \times \{0,1\}^{256} := \mathrm{H}(\zeta)$ `// Here H is SHAKE-256`

$\mathbf{A} \in R_q^{k \times \ell} := \mathrm{GenA}(\rho)$

$(\mathbf{s}_1, \mathbf{s}_2) \in S_\eta^\ell \times S_\eta^k := \mathrm{GenS}(\rho')$

$\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 \pmod q$

$(\mathbf{t}_1, \mathbf{t}_0) := \text{Power2Round}_q(\mathbf{t}, d)$

Note that $\left(\mathbf{A}|\mathbf{I}_k|\text{-}\mathbf{t}_1 2^d\right) \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \\ 1 \end{pmatrix} = \mathbf{t}_0 \pmod q$ `// Set` $\mathbf{s} := \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \\ 1 \end{pmatrix}$, $\mathbf{A}' := \left(\mathbf{A}|\mathbf{I}_k|\text{-}\mathbf{t}_1 2^d\right)$

$pk := \mathrm{BytePackPk}\,(\rho, \mathbf{t}_1)$, $\mathrm{tr} \in \{0,1\}^{256} := \mathrm{H}(pk)$

$sk := \mathrm{BytePackSk}\left(\rho, K, \mathrm{tr}, \mathbf{s} := \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \\ 1 \end{pmatrix}, \mathbf{t}_0, \mathbf{t}_1\right)$

**return** $pk, sk$

**Algorithm 49:** $\texttt{SIGN}(m, sk)$

$(\rho, K, \mathrm{tr}, \mathbf{s}, \mathbf{t}_0, \mathbf{t}_1) := \mathrm{ByteUnPackSk}(sk)$ // $\mathbf{s} := \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \\ 1 \end{pmatrix}$

$\mathbf{A} \in R_q^{k \times \ell} := \mathrm{GenA}(\rho)$

$\mathbf{A}' := \left(\mathbf{A}|\mathbf{I}_k| - \mathbf{t}_1.2^d\right)$

$\mu \in \{0,1\}^{512} := \mathrm{H}(\mathrm{tr}\,||m)$

$\kappa := 0$

$\rho'' \in \{0,1\}^{512} := \mathrm{H}(K||\mu)$ (or $\rho'' \leftarrow \{0,1\}^{512}$ for randomized signing)

**while** *True* **do**

  $\mathbf{y} \in \tilde{S}_{\gamma_1}^{l+k+1} := \mathrm{GenMask}(\rho'', \kappa)$

  $\mathbf{w} := \mathbf{A}'\mathbf{y} \bmod q$

  $\mathbf{w}_1 = \mathrm{HighBits}_q(\mathbf{w}, 2\gamma_2)$

  $\tilde{c} \in \{0,1\}^{256} := \mathrm{H}(\mu||\mathbf{w}_1)$

  $c \in B_\tau := \mathrm{SampleInBall}(\tilde{\mathbf{c}})$

  $\mathbf{z} = \mathbf{y} + (-1)^b c\mathbf{s},\ b \xleftarrow{\$} \{0,1\}$

  **if** $\|\boldsymbol{z}\|_\infty \leq \gamma_1 - \beta$ **then**

    $h = \mathrm{MakeHint}_q(-(-1)^b c\mathbf{t}_0, \mathbf{w} + (-1)^b c\mathbf{t}_0, 2\gamma_2)$

    **if** $\|(-1)^b c\boldsymbol{t}_0\|_\infty \leq \gamma_2$ ***and*** the number of *1's in* $\boldsymbol{h}$ is less than or equal to $\omega$ **then**

      **return** $\sigma := \mathrm{BytePackSig}(\mathbf{z}, \tilde{c}, \mathbf{h})$

  $\kappa = \kappa + (k + \ell + 1)$

**Algorithm 50:** $\texttt{VRFY}(m, pk, \sigma)$

$(\mathbf{z}, \tilde{c}, \mathbf{h}) = \mathrm{ByteUnPackSig}(\sigma)$

**if** $\|\boldsymbol{z}\|_\infty \geq \gamma_1 - \beta$ **then**

  **return** False

**if** the number of 1's in $\boldsymbol{h}$ is less than or equals to $\omega$ **then**

  **return** False

$(\rho, \mathbf{t}_1) = \mathrm{ByteUnPackPk}(pk)$

$\mathbf{A} \in R_q^{k \times \ell} := \mathrm{GenA}(\rho)$

$\mathbf{A}' := (\mathbf{A}|\mathbf{I}_k| - \mathbf{t}_1.2^d)$

$\mu \in \{0,1\}^{512} := \mathrm{H}(\mathrm{H}(\mathrm{pk})|\mathrm{m})$

$c := \mathrm{SampleInBall}(\tilde{c})$

$\mathbf{w}_1' := \mathrm{UseHint}_q(\mathbf{h}, \mathbf{A}'\mathbf{z} \bmod q, 2\gamma_2)$

**return** $c \stackrel{?}{=} H\ (\mu|\mathbf{w}_1')$

The aim of this milestone is to come up with a draft Implementation of our proposed algorithm, analyse and test it against our theoretical analysis carried out in the Milestone 4.

## 5.1 Parameter specification

In this section, we recall the parameters suggested in the previous report and analyze the average number of repetitions needed for signature generation in both theory and practice.

### 5.1.1 Public key size:

Our public key is $(\rho, \mathbf{t}_1)$. We drop $d$ bits from $\mathbf{t}$ to get $\mathbf{t}_1$; thus, each coefficient of each polynomial of $\mathbf{t}_1$ is of $(\lceil(\log q)\rceil - d)$ bits and $\mathbf{t}_1$ has $k$ polynomials, each of degree 256. Thus, $\mathbf{t}_1$ is of $256\cdot(\lceil(\log q)\rceil - d)\cdot k$ bits. Now, considering that we take a 256-bit seed $\rho$ to generate matrix $\mathbf{A}$ and the number of bits we drop $d$, our public key size will be $(256 + 256.(\lceil(\log q)\rceil - d).k + \lceil(\log d)\rceil)$ bits.

### 5.1.2 Signature size:

Our signature is $(\mathbf{z}, \tilde{c}, \mathbf{h})$. The challenge $\tilde{c}$ is of 256 bits (32 bytes). The hint vector $\mathbf{h}$ is sent as an encoded byte string of length $\omega + k$ using HintBitPack, so $\mathbf{h}$ has $\omega + k$ bytes. To produce a valid signature, we must have $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, thus the total number of choices for each coefficient of $\mathbf{z}$ is $2\gamma_1 - 2\beta - 1 < 2\gamma_1$, so each coefficient of $\mathbf{z}$ can be represented using $\log_2(2\gamma_1) = 1 + \log_2 \gamma_1$ bits, and there are $(k + l + 1)$ 256-degree polynomials in $\mathbf{z}$. Hence, the bit size of $\mathbf{z}$ is $256 \cdot (k + l + 1) \cdot (1 + \log_2 \gamma_1)$ bits. Thus, in bytes, the size of the signature $(\mathbf{z}, \tilde{c}, \mathbf{h})$ is $32(k + l + 1)(\log_2 \gamma_1 + 1) + 32 + (\omega + k)$.

As we have seen that public and secret keys of our algorithm are tuples containing several objects, we have encoded them into consecutive bytes so that they can be sent and hashed, if needed, conveniently. We have used a similar approach for encoding as used in the Dilithium digital signature scheme. However, we note that byte encoding of keys and other parameters has nothing to do with the security of the signature algorithm.

### 5.1.3 Implementation Details

As can be seen in Table 5.1, we have considered the set of parameters as in Dilithium with a noteworthy theoretical improvement in the number of repetitions needed for a valid signature generation. Here is a practical approximation (see the Table 5.2) of the average number of repetitions after a successful implementation of the scheme. Thus, in practice, a higher efficiency in signature generation is achievable than in theory. At this point, we want to note that most of the experiments are carried out on a single core of Intel® Core™ i9-13900K Processor on the Ubuntu 22.04 operating system with our draft implementation.

## 5.2 An alternate set of parameters

### 5.2.1 Choice of parameters

Our signature scheme's security and performance are dependent on a few parameters. We have strategically selected the parameters to achieve sufficient security and better performance.

| NIST Security level | 2 | 3 | 5 |
|---|---|---|---|
| Parameters | | | |
| $q$ [modulus] | 8380417 | 8380417 | 8380417 |
| $d$ [dropped bits of **t**] | 13 | 13 | 13 |
| challenge entropy [$\log \binom{256}{\tau} + \tau$] | 192 | 196 | 257 |
| $\tau$ [no. of $\pm 1$'s in $c$ ] | 39 | 49 | 60 |
| $\gamma_1$ [**y** coefficient range] | $2^{17}$ | $2^{19}$ | $2^{19}$ |
| $\gamma_2$ [low-order rounding range] | $(q-1)/88$ | $(q-1)/32$ | $(q-1)/32$ |
| $(k, l)$ [dimensions of **A**] | (4,4) | (6,5) | (8,7) |
| $\eta$ [secret key range] | 2 | 4 | 2 |
| $\beta$ [$\tau.\eta$] | 78 | 196 | 120 |
| $\omega$ [max no. of 1's in the hint **h**] | 80 | 55 | 75 |
| Repetitions | 3.94 (4.25) | 3.15 (5.1) | 2.55 (3.85) |
| Output Size | | | |
| public key sizes (bytes) | 1312 (1312) | 1952 (1952) | 2592 (2592) |
| signature sizes (bytes) | 5300 (2420) | 7773(3293) | 10355 (4595) |
| LWE Hardness | | | |
| BKZ blocksize b (GSA) | 422 | 622 | 860 |
| Classical Core-SVP (bits) | 123 | 181 | 251 |
| Quantum Core-SVP (bits) | 111 | 164 | 228 |
| SIS Hardness | | | |
| BKZ blocksize b | 417 | 602 | 868 |
| Classical Core-SVP (bits) | 121 | 176 | 253 |
| Quantum Core-SVP (bits) | 110 | 159 | 230 |

Table 5.1: Parameters, output size and security. The numbers in parentheses are corresponding parameters for Dilithium.

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Time | Average repetitions | Time | Average repetitions | Time |
| 10 | 2.3 | 0m2.053s | 2.5 | 0m3.065s | 3.0 | 0m2.508s |
| $10^2$ | 3.56 | 0m4.692s | 2.89 | 0m7.085s | 2.53 | 0m6.173s |
| $10^3$ | 3.366 | 0m11.396s | 2.84 | 0m14.993s | 2.493 | 0m19.518s |
| $10^4$ | 3.4768 | 1m26.931s | 2.8668 | 1m54.030s | 2.4135 | 2m36.486s |
| $10^5$ | 3.44832 | 12m55.592s | 2.87829 | 18m0.062s | 2.43664 | 25m1.336s |
| $10^6$ | 3.470817 | 127m44.446s | 2.883677 | 179m35.854s | 2.44017 | 250m6.992s |
| $10^7$ | 3.4696347 | 1274m37.047s | 2.8877414 | 1796m7.173s | 2.4393666 | 2543m8.646s |
| | 3.47(3.94) | | 2.89(3.15) | | 2.44(2.55) | |

Table 5.2: Practical repetitions for signature generation. The numbers in parentheses are the theoretical values.

| NIST Security level | 2 | 3 | 5 |
|---|---|---|---|
| Parameters | | | |
| $q$ [modulus] | 8383489 | 8383489 | 8383489 |
| $d$ [dropped bits of **t**] | 13 | 13 | 13 |
| challenge entropy [$\log\binom{256}{\tau} + \tau$] | 196 | 228 | 255 |
| $\tau$ [no. of $\pm 1$'s in $c$ ] | 36 | 45 | 60 |
| $\gamma_1$ [**y** coefficient range] | $2^{17}$ | $2^{19}$ | $2^{19}$ |
| $\gamma_2$ [low-order rounding range] | $(q-1)/96$ | $(q-1)/32$ | $(q-1)/32$ |
| $(k, l)$ [dimensions of **A**] | (4,4) | (6,5) | (8,7) |
| $\eta$ [secret key range] | 2 | 4 | 2 |
| $\beta$ $[\tau.\eta]$ | 72 | 180 | 120 |
| $\omega$ [max no. of 1's in the hint **h**] | 90 | 56 | 70 |
| Repetitions | 3.5 | 2.9 | 2.4 |
| Output Size | | | |
| public key sizes (bytes) | 1312 | 1952 | 2592 |
| signature sizes (bytes) | 5310 | 7774 | 10350 |
| LWE Hardness | | | |
| BKZ blocksize b (GSA) | 422 | 622 | 860 |
| Classical Core-SVP (bits) | 123 | 181 | 251 |
| Quantum Core-SVP (bits) | 111 | 164 | 228 |
| SIS Hardness | | | |
| BKZ blocksize b | 423 | 602 | 868 |
| Classical Core-SVP (bits) | 123 | 176 | 253 |
| Quantum Core-SVP (bits) | 112 | 159 | 230 |

Table 5.3: Parameters, output sizes and security. The numbers for SIS security are for the strongly unforgeable.

**Choice of $q$ and $\gamma_1$**

Our first priority in choosing prime $q$ has to be **NTT** friendly. We have chosen a 23-bit prime $q = 8383489$ so that there exists a 512-th root of unity $r = 11998 \mod 8383489$. This implies that the cyclotomic polynomial $x^{256} + 1$ splits into linear factors $X - r^i \mod q$ with $i = 1, 3, 5, ..., 511$. On the other hand, since we are sampling **y** from $S_{\gamma_1}^{l+k+1}$, and the number of repetitions directly depends on $\gamma_1$, so as to reduce repetitions in the signature algorithm, we have to keep the lowest value of $\gamma_1$ to be $2^{17}$ and then by testing so many **NTT** friendly primes we got the selected prime, which is suitable for the design.

**Choice of $\gamma_2$**

$\gamma_2$'s choice is more strategic. It must be a divisor of $(q-1)/2$. $\gamma_1$ and $\gamma_2$ are directly involved with the hardness of the SIS problem in our scheme. So, we have to minimize those values as well. Also, the signature algorithm has a rejection in Step 17. However, it is difficult to formally count the probability that Step 17 results in a restart. So initially we aimed to set $\gamma_2$'s value greater

than $\|c\mathbf{t}_0\|_\infty$. But for NIST level 2, to achieve the necessary security, we kept $\gamma_2$'s value less than $\|c\mathbf{t}_0\|_\infty$. We have simulated our signature scheme several times and observed that due to $\gamma_2$'s value for security level 2, theoretically and practically, the repetitions of the signature algorithm are very close. But for levels 3 and 5, we get exact repetitions of what we claim for keeping the value of $\gamma_2$ greater than $\|c\mathbf{t}_0\|_\infty$. One can think that for security levels 3 and 5, we can reduce the values of $\gamma_2$ more. But then, practically, a signature generation will take more time and about $1003$ repetitions to generate one signature corresponding to one message, which is unexpected.

### Choice of $\omega$

$\omega$ is the number of 1's in the hint vector $\mathbf{h}$. Initially, we assigned some values to $\omega$ for each security level and ran the signature algorithm several times. Then we have assigned those critical values to $\omega$ for each security level so that the rejection rate is very low. $\tau$ indicates the number of $\pm$ 1 in the challenge polynomial $c$. Before going into the details of how we have chosen the values of $\tau$ for different security levels, we first discuss what challenge entropy is. Challenge entropy refers to the amount of uncertainty or randomness in the challenge sent by a verifier to a prover or by one party to another in a protocol. If the challenge is a random variable $C$, then its challenge entropy measures how hard it is to guess or predict the challenge. NIST security level 2 parameters make dilithium strong, and to break it, one must require computational resources comparable to or greater than the cost of a collision search on SHA-256/SHA3-256. Now, to find a collision, one needs to take about $2^{128}$ operations. So it is sufficient to put $\tau$'s value 23, and the resultant entropy will be 131. But we have chosen more randomness for each level.

### Choice of $\ell$,$k$,$\eta$

Choosing values for $\ell$ and $k$ for each security level is entirely dependent on lattice reduction attacks. One can check out the repository [1] and see modelBKZ.py. We independently input $\ell$, $k$, $\eta$ and with the help of $\mathrm{LWE\text{-}optimize\text{-}attack}$, one can get the best m as a number of Ring-LWE samples and best BKZ block size b for both the primal and dual attack. Similarly, with the help of $\mathrm{SIS\text{-}optimize\text{-}attack}$ and the SIS-bound, which we have seen already, one can get the best MSIS dimension and best BKZ block size.

### Implementation Details

Here is a practical approximation of the average number of repetitions (see Table 5.4) with the new alternate set of parameters (see Table 5.3).

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Time | Average repetitions | Time | Average repetitions | Time |
| 10 | 4.5 | 0m2.292s | 3.9 | 0m1.583s | 2.0 | 0m4.073s |
| $10^2$ | 3.21 | 0m5.672s | 3.13 | 0m4.113s | 2.61 | 0m5.020s |
| $10^3$ | 3.562 | 0m11.677s | 2.849 | 0m14.133s | 2.522 | 0m18.429s |
| $10^4$ | 3.5588 | 1m20.377s | 2.9179 | 1m53.098s | 2.4773 | 2m34.892s |
| $10^5$ | 3.56552 | 12m50.690s | 2.95179 | 18m3.067s | 2.47101 | 24m43.102s |
| $10^6$ | 3.546833 | 127m18.597s | 2.944847 | 178m42.723s | 2.46936 | 247m16.082s |
| $10^7$ | 3.5467555 | 1289m7.553s | 2.9478429 | 1795m0.587s | 2.4693252 | 2468m45.859s |
| | 3.55(3.5) | | 2.94(2.9) | | 2.47(2.4) | |

Table 5.4: Practical repetitions for signature generation. The numbers in parentheses are the theoretical values.

[1] `https://github.com/pq-crystals/security-estimates/tree/master`

We are continually refining our implementation and testing it for efficiency and weaknesses. We plan to demonstrate our algorithm in the technical review of Milestone 5. We want to note that most of the technical work in this milestone was to come up with a draft implementation of the project, and we have successfully achieved that. We are hopeful to come up with a robust Python implementation of our proposed signature scheme by the next (final) milestone.

# Milestone 6

# Implementation Details

In the Milestone 4, we have produced a design of our proposed digital signature scheme along with a detailed cryptanalysis of the same. Alongside, we have also specified the parameters of our scheme pertaining to the NIST specified security levels while using the same 23-bit NTT-friendly prime 8380417 as in [28]. In fact, we have kept all the same parameters as in Dilithium, to make our claim of achieving lesser rejections with same public-key size stronger. After that, moving on to Milestone 5, we gave a detailed analysis of how the parameters were selected along with a new set of parameters with a different prime 8383489. In this Milestone 6 we give two sets of implementations of our scheme both of which are in python. The first one is highly motivated from the reference implementation [1] in the NIST submission of Dilithium-3.1 [2] and follows the techniques used by [3], while the second one is a direct adaptation of the FIPS-204 [72]. We also provide `kat` vectors for smooth verification of our implementations, in the `assets` folder. We now move on to discuss in brief detail specifics of both these implementations along with a comparison of their performance in the following sections.

## 6.1 Implementation I

In the first implementation [Fig. 6.1], we have divided the `Proposal_DSA` package inside the `src` directory further into four python packages `dsa`, `modules`, `polynomials`, `utilities`. The python code with filename `dsa.py` inside the `dsa` directory contains the implementation of the main algorithms [Fig. 6.3] `keygen()`, `sign(sk,msg)` and `verify(pk,msg,sig)` along with some other helper functions. The functions in the `modules` and `polynomials` packages perform the same operations, the only difference being that the former acts on each module element, precisely each vector of polynomials, and the latter on each polynomial within it.

[1] `https://github.com/pq-crystals/dilithium/releases/tag/v3.1`
[2] `https://pq-crystals.org/dilithium/resources.shtml`
[3] `https://github.com/GiacomoPope/dilithium-py.git`

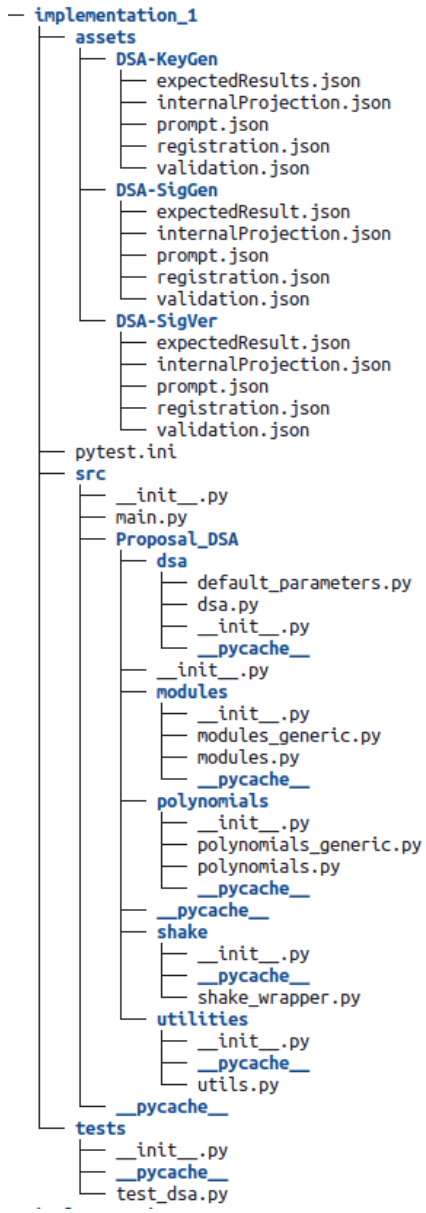



Figure 6.1: Implementation I

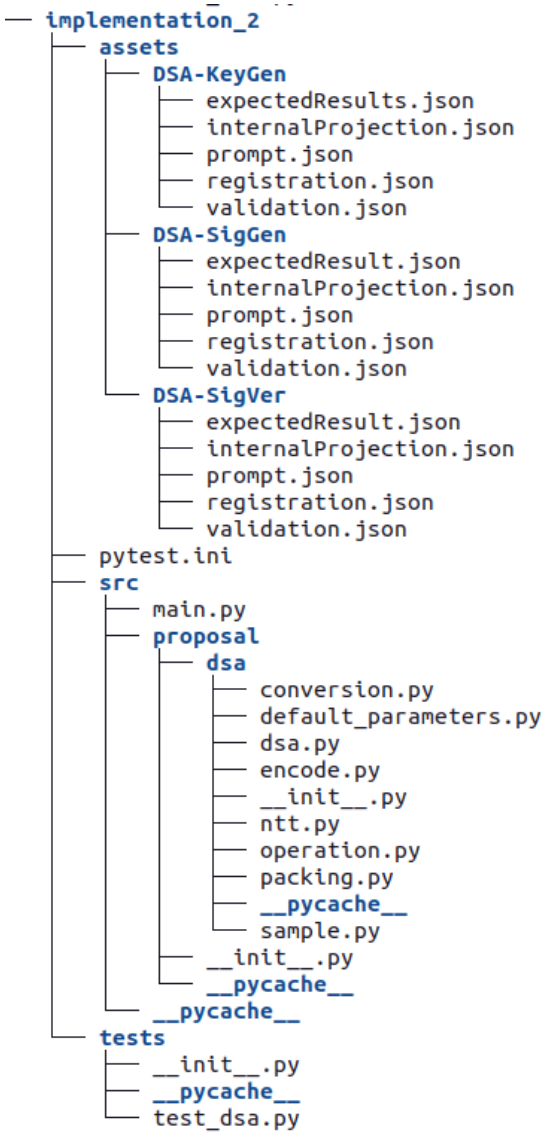



Figure 6.2: Implementation II

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Signing Time | Average repetitions | Signing Time | Average repetitions | Signing Time |
| 10 | 5.5 | 0.3742s | 2.2 | 0.2607s | 3.0 | 0.4675s |
| $10^2$ | 4.52 | 0.2990s | 1.79 | 0.2274s | 2.34 | 0.2677s |
| $10^3$ | 4.04 | 0.2153s | 1.78 | 0.1209s | 2.52 | 0.2196s |
| $10^4$ | 4.05 | 0.2031s | 1.77 | 0.1110s | 2.62 | 0.2192s |
| $10^5$ | 4.05 | 0.1623s | 1.78 | 0.1286s | 2.59 | 0.2497s |
| $10^6$ | 4.05 | 0.1482s | 1.79 | 0.1194s | 2.58 | 0.2050s |
| | 4.37 | 0.2337s | 1.85 | 0.1613s | 2.61 | 0.2714s |

Table 6.1: Practical repetitions and signature time using Implementation I and parameter set 5.1

```python
def keygen(self):
    """
        :Generates a public-private keypair
    """
    # Random seed
    zeta = self.random_bytes(32) #{32*8 = 256} bit seed

    # Expand with an XOF (SHAKE256)
    seed_bytes = self._h(zeta, 128)

    # Split bytes into suitable chunks
    rho, rho_prime, cap_k = seed_bytes[:32], seed_bytes[32:96], seed_bytes[96:]

    # Generate matrix A ∈ R^(kxl) in the NTT domain
    cap_a_hat = self._expand_cap_a_hat(rho)[1]
    # print(a_hat)

    # Generate the error vectors s1 ∈ R^l, s2 ∈ R^k
    s1_list, s2_list, s1, s2 = self._expand_s(rho_prime)

    #new secret vector
    s = self.M([[elt] for elt in s1_list + s2_list + [self.R([1])]])  # {s = [s1 | s2 | 1]^T}

    s1_hat = s1.to_ntt()

    # Matrix multiplication
    t = (cap_a_hat @ s1_hat).from_ntt() + s2
    t1, t0 = t.power_2_round(self.d)

    # Pack up the bytes
    pk = self._pack_pk(rho, t1)
    tr = self._h(pk, 32)

    sk = self._pack_sk(rho, cap_k, tr, s, t0, t1)
    return pk, sk
```

```python
def verify(self, pk_bytes, m, sig_bytes):
    """
    Verifies a signature for a message m from a
    byte-encoded public key and signature
    """
    rho, t1 = self._unpack_pk(pk_bytes)
    c_tilde, z, h = self._unpack_sig(sig_bytes)
    if h.sum_hint() > self.omega:
        return False
    if z.check_norm_bound(self.gamma_1 - self.beta):
        return False

    cap_a_prime_hat = self._cap_a_prime_hat_matrix(rho, t1)
    tr = self._h(pk_bytes, 32) #Compute trace {tr}
    mu = self._h(tr + m, 64)

    z_hat = z.to_ntt()
    cap_a_prime_z = (cap_a_prime_hat @ z_hat).from_ntt()
    w_prime = h.use_hint(cap_a_prime_z, 2 * self.gamma_2)
    w_prime_bytes = w_prime.bit_pack_w(self.gamma_2)
    return c_tilde == self._h(mu + w_prime_bytes, 32)
```

```python
def sign(self, sk_bytes, msg):

    # unpack the secret key
    rho, cap_k, tr, s, t0, t1 = self._unpack_sk(sk_bytes)
    cap_a_prime_hat = self._cap_a_prime_hat_matrix(rho, t1)

    mu = self._h(tr + msg, 64) # Set seeds and nonce (kappa)
    kappa = 0
    rho_prime_2 = self._h(cap_k + mu, 64)

    # Precompute NTT representation
    s_hat = s.to_ntt()
    t0_hat = t0.to_ntt()

    alpha = self.gamma_2 << 1
    count = 0
    while True:
        count += 1
        y = self._expand_mask_vector(rho_prime_2, kappa)
        y_hat = y.to_ntt()
        kappa += self.k + self.l + 1 #TODO: Shashank
        w = (cap_a_prime_hat @ y_hat).from_ntt()
        w1, w0 = w.decompose(alpha)

        w1_bytes = w1.bit_pack_w(self.gamma_2)
        c_tilde = self._h(mu + w1_bytes, 32)
        c = self.R.sample_in_ball(c_tilde, self.tau)

        # Store c in NTT form
        c_hat = c.to_ntt()
        bit = random.randint(0, 1)
        if bit:
            z = y + (s_hat.scale(c_hat)[1]).from_ntt()
        else:
            z = y - (s_hat.scale(c_hat)[1]).from_ntt()

        if not (z.check_norm_bound(self.gamma_1 - self.beta)):
            c_t0 = t0_hat.scale(c_hat)[1].from_ntt()
            if not c_t0.check_norm_bound(self.gamma_2):
                #makehint
                if bit:
                    h = (-c_t0).make_hint(w+c_t0, alpha)
                else:
                    h = c_t0.make_hint(w-c_t0, alpha)

                if h.sum_hint() <= self.omega:
                    return  self._pack_sig(c_tilde, z, h), count
```

Figure 6.3: Python code for `keygen()`, `sign(sk,msg)`, `verify(pk,msg,sig)`

The code can be executed in two ways, either by directly executing the `main.py` file or through python interactive shell. Figure 6.4 shows how we can generate the public key and secret key pairs and then sign a message of our choice. Here, we need to make sure that the message is in bytes. It also shows how verification will fail with a wrong message. In fact, the verification will also fail with the wrong key. One can also check the parameters that are being used for the signature scheme. This implementation does not hard-code the optimal parameters before hand, the parameters need to be provided while execution. We further give a practical approximation of the average number of repetitions and average time taken during each signature generation in Tables 6.1 and 6.2. The implementation of the proposed schemes is available at GitHub repository.

```
>>>
>>> from Proposal_DSA.dsa.dsa import *
>>> dsa2 = DSA(q=8383489,q_len=23,n=256,root_of_unity=11998,d=13,k=4,l=4,eta=2,tau=40,omega=90,gamma_1=131072,gamma_2=87328)
>>>
>>> pk, sk = dsa2.keygen()
>>>
>>> # example of signing
>>> msg = b'hello world!'
>>> sig,_ = dsa2.sign(sk,msg)
>>>
>>> # verification
>>> dsa2.verify(pk,msg,sig)
True
>>>
>>>
>>> # verification will fail with a wrong msg
>>> msg_wrong = b'hi!'
>>> dsa2.verify(pk,msg_wrong,sig)
False
>>>
>>> # parameters used in the algorithm
>>> dsa2.k
4
>>> dsa2.l
4
>>>
>>>
```

Figure 6.4: Signature generation and verification using Implementation I

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Signing Time | Average repetitions | Signing Time | Average repetitions | Signing Time |
| 10 | 5.2 | 0.3656s | 5.6 | 0.5476s | 1.9 | 0.3458s |
| $10^2$ | 4.41 | 0.2650s | 3.6 | 0.3712s | 2.66 | 0.2476s |
| $10^3$ | 4.21 | 0.1899s | 3.12 | 0.1849s | 2.66 | 0.2247s |
| $10^4$ | 4.13 | 0.1965s | 3.32 | 0.1896s | 2.61 | 0.3189s |
| $10^5$ | 4.13 | 0.1544s | 3.24 | 0.1868s | 2.64 | 0.2509s |
| $10^6$ | 4.15 | 0.1587s | 3.25 | 0.1831s | 2.60 | 0.2061s |
| | 4.37 | 0.2217s | 3.68 | 0.2772 | 2.51 | 0.2657s |

Table 6.2: Practical repetitions and signature time using Implementation I and parameter set 5.3

```python
def dsa_keygen(self, random_seed: bytes = None) -> tuple[bytes, bytes]:
    """
    Generates a public-private key pair from a seed. Can take an input seed for deterministic output.
    Args:
        random_seed (bytes): for deterministic output 32 bytes input bytestring.
    Returns:
        ( public_key, private_key ) (Tuple[ bytes, bytes]): Tuple of key pair containing public key an
    Raises:
        ValueError: If the input seed is not 32 bytes.
        TypeError: If the input seed is not a byte string.
    """
    # line 1: choose random 32-byte seed
    if random_seed is None:
        random_seed = secrets.token_bytes(32)
    elif not (isinstance(random_seed, (bytes, bytearray)) and len(random_seed) == 32):
        raise ValueError("expected a 32-bytes bytestring.")

    # line 2: expand the hashed random seed into 3 different parts.
    seed = self.convert.hash(random_seed,128)
    rho = seed[0:32]         # a. Get ρ (rho):          The first 32 bytes
    rho_prime = seed[32:96]  # b. Get ρ' (rho_prime):   The next 64 bytes
    k_seed = seed[96:]       # c. Get k_seed:           The last 32 bytes

    # line 3: generate the matrix A (k x l) and store polynomials as list of coefficients.
    matrix_a_cap = self.sample.gen_a(rho)

    # line 4: generate and store s and e polynomial vectors.
    s1, s2 = self.sample.gen_s(rho_prime)

    # line 5 compute: vector t as 'A.s + e'.
        # a. Transform s into the NTT domain for faster polynomial multiplication.
    s1_ntt = [self.ntt.ntt(p) for p in s1]
        # b. Compute the matrix-vector product Â ∘ NTT(s)
    a_s1_ntt = self.ntt.multiply_ntt_matrix_vector(matrix_a_cap, s1_ntt)
        # c. Transform the result back from the NTT domain to the standard polynomial form.
    a_s1 = [self.ntt.inv_ntt(p) for p in a_s1_ntt]
        #d. Add the second secret vector e
    t = self.ntt.add_vector_ntt(a_s1, s2)

    # line 6: decompose t into (t1, t0) such that t = (t1.2^d + t0) mod q
    t1, t0 = self.ntt.power2round_vec(t)

    # encode public_key
    public_key = self.encode.byte_pack_pk(rho, t1)

    # line 8: hash public_key to make tr
    tr = self.convert.hash(public_key,32)

    # concatenating to obtain the secret vector
    poly=[1 if i==0 else 0 for i in range(self.N)]
    s= s1 + s2 + [poly]

    # encode private_key
    private_key = self.encode.byte_pack_sk(rho, k_seed, tr, s, t0, t1)

    # line 10: return both public and private keys.
    return public_key, private_key
```

```python
def dsa_verify(self, public_key: bytes, message, signature: bytes) -> bool:
    """
    Internal function to verify a signature for a formatted message M'.
    Args:
        public_key (bytes): The public key bytestring.
        message (bytes): The message in bytes.
        signature (bytes): The signature bytestring.
    Returns:
        result (bool): True if the signature is valid, False otherwise.
    Raises:
        ValueError: If the public key or signature is invalid.
        TypeError: If the message is not a byte string.
    """
    if not isinstance(public_key, bytes):
        raise ValueError("Invalid public key format.")
    if not (isinstance(message, (bytes, bytearray))):
        raise TypeError("Invalid message format. Message should be in bytes")
    if not isinstance(signature, bytes):
        raise ValueError("Invalid signature format.")

    rho, t1 = self.encode.byte_unpack_pk(public_key)

    c_tilde, z, h = self.encode.sig_decode(signature)

    if h is None:
        return False

    a_hat = self.sample.gen_a(rho)

    pow_d_t1=[[(-(1 << self.d))*t1[i][j] for j in range (self.N)] for i in range (self.k)]

    pow_d_t1_ntt=[self.ntt.ntt(p) for p in pow_d_t1]

    # line 8: concatenating (A | I | t1_ntt) to obtain the public matrix A_prime matrix in ntt form
    id_matrix=[[[0 for _ in range (self.N)] for _ in range (self.k)]for _ in range (self.k)]
    for i in range (self.k):
        id_matrix[i][i][0]=1
    id_ntt=[[self.ntt.ntt(p) for p in id_matrix[i]] for i in range (self.k)]

    a_prime=[a_hat[i] + id_ntt[i] + [pow_d_t1_ntt[i]] for i in range (self.k)]

    tr = self.convert.hash(public_key, 32)

    mu = self.convert.hash(tr + message, 64)

    # Compute: A ∘ NTT(z)
    z_ntt = [self.ntt.ntt(p) for p in z]
    a_prime_z_ntt = self.ntt.multiply_ntt_matrix_vector(a_prime, z_ntt)

    a_prime_z = [self.ntt.inv_ntt(p) for p in a_prime_z_ntt]

    # line 11 : computing the high-bits of w using the hint vector
    w1 = [[0 for _ in range(self.N)] for _ in range(self.k)]
    for i in range(self.k):
        for j in range(self.N):
            w1[i][j] = self.operation.use_hint(h[i][j], a_prime_z[i][j])

    c_hash_encoded = self.convert.hash(mu + self.encode.w1_encode(w1), 32)

    return (self.convert.infinity_norm(z) < (self.gamma1 - self.beta)) and (c_hash_encoded == c_tilde)
```

Figure 6.5: Python code for `dsa_keygen()`,`dsa_verify(pk,msg,sig)`

## 6.2 Implementation II

In this implementation[Fig. 6.1], we only have one python package `proposal` inside `src`, which further contain the package `dsa`. The main algorithms `dsa_keygen()`, `dsa_sign(sk,msg)` and `dsa_verify(pk,msg,sig)` are implemented within the python file `dsa.py`. This is a direct implementation of the algorithms in FIPS-204 [72], so we do not use separate python packages for `modules` and `polynomials`. In fact, the entire implementation uses python lists instead of defining separate python classes for modules and polynomials, thereby making the implementation more compact. The `conversion.py` implements all kinds of conversions between bits, bytes and integers and also the hash function. `packing.py` and `operation.py` implement the bit packing and high bit, low bit, hint bit generation algorithms respectively. The encode and decode operations on the public and secret keys and signatures are implemented in `encode.py`. The `Gen_A`, `Gen_S`, `Gen_Mask` and various other sampling algorithms are implemented in `sample.py`, while `ntt.py` implements all the required NTT operations.

This code can also be executed in two ways, either by directly executing the `main.py` file or through python interactive shell. Figure 6.7 shows how we can generate the public key and secret key pairs and then sign a message of our choice. It also shows how verification will fail with a wrong message. In fact, the verification will also fail with the wrong key. One can also check the parameters that are being used for the signature scheme. Contrary to the above, the main difference of this implementation lies in the fact that the optimal parameters are hard-coded before hand in the python file `default_paramters.py` with the `dsa` package. Both the parameters 5.1 and 5.3 for each security level are specified in two separate dictionaries `DEFAULT_PARAMETERS_1` and

```python
def dsa_sign(self, private_key: bytes, message, random_bit: bool = None):
    """
    Algorithm to generate a signature for a message.
    Args:
        private_key (bytes): The private key bytestring.
        message (bytes): The message to be signed in bytes.
        random_bit: random bit used for randomizing the signature
    Returns:
        signature (bytes): The generated signature as a bytestring.
    Raises:
        TypeError: If the private key or random input seed is invalid.
        ValueError: If the input seed is not 32 bytes.
        TypeError: If the message is not a byte string.
    """
    c_tilde = None

    if not isinstance (private_key, (bytes, bytearray)):
        raise TypeError("Invalid private key format.")
    if not (isinstance(message, (bytes, bytearray))):
        raise TypeError("Invalid message format. Message should be in bytes")

    # line 1: breaking the private key into 6 subcomponents.
    rho, k_seed, tr, s, t0, t1 = self.encode.byte_unpack_sk(private_key)

    # modify the t1 as: -(2^d) * t1_vec, coefficient wise.
    pow_d_t1 = [[(-(1 << self.d))*t1[i][j] for j in range (self.N)] for i in range (self.k)]

    # convert all polynomial vectors to NTT form.
    s_ntt = [self.ntt.ntt(p) for p in s]
    t0_ntt = [self.ntt.ntt(p) for p in t0]
    pow_d_t1_ntt = [self.ntt.ntt(p) for p in pow_d_t1]

    # line 2: sample a k x l matrix from input seed rho.
    matrix_a_hat = self.sample.gen_a(rho)

    # line 3: concatenating (A | I | t1_ntt) to obtain the public matrix A_prime matrix in ntt form
    id_matrix=[[[0 for _ in range (self.N)] for _ in range (self.k)] for _ in range (self.k)]
    for i in range (self.k):
        id_matrix[i][i][0]=1
    id_ntt=[[self.ntt.ntt(p) for p in id_matrix[i]] for i in range (self.k)]

    a_prime=[matrix_a_hat[i] + id_ntt[i] + [pow_d_t1_ntt[i]] for i in range (self.k)]

    # line 4: hash the message after concatenating it after tr with shake 256 into a 64-bytes bytestring.
    mu = self.convert.hash(tr + message,64)

    # line 6: compute a private random seed by hashing (K + input random seed + mu) with shake 256 into a 64-
    rho_prime_prime = self.convert.hash(k_seed + mu, 64)
    #rho_prime_prime=self.random_bytes(64) # for randomized signing

    # initialize kappa, z and h.
    kappa = 0
    z = None
    h = None

    # line 7: rejection sampling loop.
    while z is None and h is None:

        self.total_num_of_rejections +=1 #counting average rejections

        # line 8: generate a vector of (k+l+1) polynomials using private random seed.
        y = self.sample.gen_mask(rho_prime_prime, kappa)

        # line 9: create a vector of k polynomials by multiplying A and y in NTT domain.
        y_ntt = [self.ntt.ntt(p) for p in y]
        a_prime_y = self.ntt.multiply_ntt_matrix_vector(a_prime, y_ntt)
        w = [self.ntt.inv_ntt(p) for p in a_prime_y]

        # line 10: component wise conversion to high bits.
        w_1 = [[self.operation.high_bits(w[i][j]) for j in range (self.N)] for i in range (self.k)]

        # line 11: encode the vector polynomial w_1 into a byte string
        # hash it after concatenating it after mu into a bytestring of size 32 using shake 256.
        c_tilde = self.convert.hash(mu + self.encode.w1_encode(w_1), 32)

        # line 12: takes an input seed rho of length 32 and converts it into a polynomial c.
        c = self.sample.sample_in_ball(c_tilde)

        c_ntt  = self.ntt.ntt(c)
        cs_ntt = self.ntt.multiply_ntt_polynomial_vector(c_ntt, s_ntt)
        cs = [self.ntt.inv_ntt(p) for p in cs_ntt]

        # line 13: sampling the random bit b and constructing the response vector
        if random_bit is None :
            b = random.randint(0,1)
        else:
            b = random_bit
        if not b:
            z = self.ntt.add_polynomial_vectors(y, cs)
        else:
            minus_cs = [[(-1) * cs[i][j] for j in range (self.N)] for i in range (self.k + self.l + 1)]
            z = self.ntt.add_polynomial_vectors(y, minus_cs)

        # line 14: check l infinity norm for z
        if self.convert.infinity_norm(z) >= self.gamma1 - self.beta:
            z = None
            h = None
        else:
            # line 15: calculating the hint vector componentwise
            ct0_ntt = self.ntt.multiply_ntt_polynomial_vector(c_ntt, t0_ntt)
            ct0 = [self.ntt.inv_ntt(p) for p in ct0_ntt]

            h = [[0] * self.N for _ in range(self.k)] # initialize the empty hint vector.
            if not b:
                if not self.convert.infinity_norm(ct0) >= self.gamma2:
                    w_plus_ct0 = self.ntt.add_polynomial_vectors(w, ct0)
                    for i in range(self.k):
                        for j in range(self.N):
                            h[i][j] = self.operation.make_hint(-ct0[i][j],w_plus_ct0[i][j])
            else:
                minus_ct0 = [[(-1)*ct0[i][j] for j in range (self.N)] for i in range (self.k)]
                if not self.convert.infinity_norm(minus_ct0) >= self.gamma2:
                    w_minus_ct0 = self.ntt.add_polynomial_vectors(w, minus_ct0)
                    for i in range(self.k):
                        for j in range(self.N):
                            h[i][j] = self.operation.make_hint(-minus_ct0[i][j],w_minus_ct0[i][j])

            if self.convert.calc_ones(h) > self.omega:
                z = None
                h = None

        kappa = kappa + (self.l + self.k + 1)

    # line 17: encode the signature with c_tilda, z and h.
    sigma = self.encode.sig_encode(c_tilde, self.convert.centered_modulus(z), h)

    return sigma
```

Figure 6.6: Python code for `dsa_sign(sk,msg)`

```
>>>
>>> from proposal.dsa import DSA_128_2 as dsa
>>> pk, sk = dsa.dsa_keygen()
>>>
>>> #example of signing
>>> msg = b'hello world!'
>>> sig = dsa.dsa_sign(sk,msg)
>>>
>>> #verification
>>> dsa.dsa_verify(pk,msg,sig)
True
>>>
>>> # verfication fails with wrong msg
>>> msg_wrong = b'hi!'
>>> dsa.dsa_verify(pk,msg_wrong,sig)
False
>>>
>>> # parameters used in the algorithm
>>> dsa.q
8383489
>>> dsa.k
4
>>> dsa.l
4
>>> from proposal.dsa import DSA_128_1 as dsa
>>> dsa.q
8380417
>>>
```

Figure 6.7: Signature generation and verification using Implementation II

`DEFAULT_PARAMETERS_2`, we just need to import the required dictionary during execution. In Figure 6.7, `DSA` stands for the signature algorithm, `128` stands for the NIST security level and `2` stands for the parameter set used. Note that the prime used is different for `DSA_128_1` and `DSA_128_2`. Further, we give the same practical approximation of the average number of repetitions and average time taken during each signature generation for this implementation in Tables 6.3 and 6.4.

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Signing Time | Average repetitions | Signing Time | Average repetitions | Signing Time |
| 10 | 3.1 | 0.2563s | 3.2 | 0.3642s | 3.9 | 0.5688s |
| $10^2$ | 4.5 | 0.3024s | 2.97 | 0.3394s | 2.37 | 0.4310s |
| $10^3$ | 3.86 | 0.2758s | 3.13 | 0.3712s | 2.49 | 0.4552s |
| $10^4$ | 4.09 | 0.3071s | 3.17 | 0.3838s | 2.55 | 0.5089s |
| $10^5$ | 4.03 | 0.2990s | 3.18 | 0.3703s | 2.59 | 0.5106s |
| $10^6$ | 4.04 | 0.3341s | 3.18 | 0.4026s | 2.58 | 0.4846s |
| | 3.94 | 0.2958s | 3.14 | 0.3719s | 2.75 | 0.4932s |

Table 6.3: Practical repetitions and signature time using Implementation II and parameter set 5.1

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Number of Runs | Average repetitions | Time | Average repetitions | Time | Average repetitions | Time |
| 10 | 3.0 | 0.2384s | 3.70 | 0.3939s | 2.70 | 0.4451s |
| $10^2$ | 3.77 | 0.2684s | 3.40 | 0.3905s | 2.73 | 0.4718s |
| $10^3$ | 3.50 | 0.2640s | 2.88 | 0.3670s | 2.67 | 0.4734s |
| $10^4$ | 3.56 | 0.2843s | 2.90 | 0.3937s | 2.68 | 0.5106s |
| $10^5$ | 3.58 | 0.2786s | 2.88 | 0.3529s | 2.68 | 0.5187s |
| $10^6$ | 3.58 | 0.3008s | 2.88 | 0.3836s | 2.64 | 0.4924s |
| | 3.50 | 0.2724s | 3.10 | 0.3803s | 2.68 | 0.4853s |

Table 6.4: Practical repetitions and signature time using Implementation II and parameter set 5.3

Table 6.2 gives a comparative study of how both the implementations 1 and 2 perform in terms

of the time taken for generating a valid signature, for the parameter sets I and II for all security levels. Observe that though the public and secret key sizes remain unaltered, signature sizes differ in both the parameter sets.

| Security level | 2 | | 3 | | 5 | |
|---|---|---|---|---|---|---|
| Parameter Set | I | II | I | II | I | II |
| Imp 1 | 0.2337s | 0.2217s | 0.1613s | 0.2772s | 0.2714s | 0.2657s |
| Imp 2 | 0.2958s | 0.2724s | 0.3719s | 0.3803s | 0.4932s | 0.4853s |
| Public key size(bytes) | 1312 | 1312 | 1952 | 1952 | 2592 | 2592 |
| Secret key size(bytes) | 3904 | 3904 | 6048 | 6048 | 7520 | 7520 |
| Signature size(bytes) | 5300 | 5310 | 7773 | 7774 | 10355 | 10350 |

Table 6.5: A comparitive study

# Appendix A

# NIST Security Categories

NIST has defined a separate category for each of the following security requirements:

1. Any attack that breaks the relevant security definition must require computational resources comparable to or greater than those required for key search on a block cipher with a 128-bit key (e.g. AES128)

2. Any attack that breaks the relevant security definition must require computational resources comparable to or greater than those required for collision search on a 256-bit hash function (e.g. SHA256/ SHA3-256)

3. Any attack that breaks the relevant security definition must require computational resources comparable to or greater than those required for key search on a block cipher with a 192-bit key (e.g. AES192)

4. Any attack that breaks the relevant security definition must require computational resources comparable to or greater than those required for collision search on a 384-bit hash function (e.g. SHA384/ SHA3-384)

5. Any attack that breaks the relevant security definition must require computational resources comparable to or greater than those required for key search on a block cipher with a 256-bit key (e.g. AES 256)

# Bibliography


[1] Michel Abdalla, Pierre-Alain Fouque, Vadim Lyubashevsky, and Mehdi Tibouchi. Tightly Secure Signatures From Lossy Identification Schemes. *Journal of Cryptology*, page 35, 2015.

[2] Michel Abdalla, Pierre-Alain Fouque, Vadim Lyubashevsky, and Mehdi Tibouchi. Tightly secure signatures from lossy identification schemes. *J. Cryptol.*, 29(3):597–631, 2016.

[3] Miklós Ajtai. The shortest vector problem in $L_2$ is *NP*-hard for randomized reductions (extended abstract). In Jeffrey Scott Vitter, editor, *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 10–19. ACM, 1998.

[4] Miklós Ajtai and Cynthia Dwork. A public-key cryptosystem with worst-case/average-case equivalence. In Frank Thomson Leighton and Peter W. Shor, editors, *Proceedings of the Twenty-Ninth Annual ACM Symposium on the Theory of Computing, El Paso, Texas, USA, May 4-6, 1997*, pages 284–293. ACM, 1997.

[5] Miklós Ajtai, Ravi Kumar, and D. Sivakumar. A sieve algorithm for the shortest lattice vector problem. In Jeffrey Scott Vitter, Paul G. Spirakis, and Mihalis Yannakakis, editors, *Proceedings on 33rd Annual ACM Symposium on Theory of Computing, July 6-8, 2001, Heraklion, Crete, Greece*, pages 601–610. ACM, 2001.

[6] Martin R. Albrecht, Robert Fitzpatrick, and Florian Göpfert. On the efficacy of solving LWE by reduction to unique-svp. In Hyang-Sook Lee and Dong-Guk Han, editors, *Information Security and Cryptology - ICISC 2013 - 16th International Conference, Seoul, Korea, November 27-29, 2013, Revised Selected Papers*, volume 8565 of *Lecture Notes in Computer Science*, pages 293–310. Springer, 2013.

[7] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.

[8] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *IACR Cryptol. ePrint Arch.*, page 46, 2015.

[9] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.

[10] Robert B. Ash. *A Course In Algebraic Number Theory*. Dover Publications Inc., ISBN=978-0486477541, 2003.

[11] László Babai. On lovász' lattice reduction and the nearest lattice point problem (shortened version). In Kurt Mehlhorn, editor, *STACS 85, 2nd Symposium of Theoretical Aspects of Computer Science, Saarbrücken, Germany, January 3-5, 1985, Proceedings*, volume 182 of *Lecture Notes in Computer Science*, pages 13–20. Springer, 1985.

[12] László Babai. On lovász' lattice reduction and the nearest lattice point problem. *Comb.*, 6(1):1–13, 1986.

[13] Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary LWE. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy - 19th Australasian Conference, ACISP 2014, Wollongong, NSW, Australia, July 7-9, 2014. Proceedings*, volume 8544 of *Lecture Notes in Computer Science*, pages 322–337. Springer, 2014.

[14] Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca, and Ronald de Wolf. Quantum lower bounds by polynomials, 1998.

[15] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. *IACR Cryptol. ePrint Arch.*, page 1128, 2015.

[16] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, CCS '06, page 390–399, New York, NY, USA, 2006. Association for Computing Machinery.

[17] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, Virginia, USA, November 3-5, 1993*, pages 62–73. ACM, 1993.

[18] Johannes Blömer and Stefanie Naewe. Sampling methods for shortest vectors, closest vectors and successive minima. In Lars Arge, Christian Cachin, Tomasz Jurdzinski, and Andrzej Tarlecki, editors, *Automata, Languages and Programming, 34th International Colloquium, ICALP 2007, Wroclaw, Poland, July 9-13, 2007, Proceedings*, volume 4596 of *Lecture Notes in Computer Science*, pages 65–77. Springer, 2007.

[19] Johannes Blömer and Jean-Pierre Seifert. On the complexity of computing short linearly independent vectors and short bases in a lattice. In Jeffrey Scott Vitter, Lawrence L. Larmore, and Frank Thomson Leighton, editors, *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, May 1-4, 1999, Atlanta, Georgia, USA*, pages 711–720. ACM, 1999.

[20] Dan Boneh, ¨Ozgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. *Random Oracles in a Quantum World*, page 41–69. Springer Berlin Heidelberg, 2011.

[21] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. Fully homomorphic encryption without bootstrapping. Cryptology ePrint Archive, Paper 2011/277, 2011. https://eprint.iacr.org/2011/277.

[22] Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*, pages 575–584. ACM, 2013.

[23] Leon Groot Bruinderink, Andreas Hülsing, Tanja Lange, and Yuval Yarom. Flush, gauss, and reload - A cache attack on the BLISS lattice-based signature scheme. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *Cryptographic Hardware and Embedded Systems - CHES 2016 - 18th International Conference, Santa Barbara, CA, USA, August 17-19, 2016, Proceedings*, volume 9813 of *Lecture Notes in Computer Science*, pages 323–345. Springer, 2016.

[24] Yuanmi Chen. *Réduction de réseau et sécurité concrète du chiffrement complètement homomorphe*. Phd thesis, Université Paris Diderot - Paris 7, June 2013.

[25] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[26] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, 2nd edition, 2001.

[27] Léo Ducas, Alain Durmus, Tancrède Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 40–56. Springer, 2013.

[28] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.

[29] Léo Ducas, Vadim Lyubashevsky, and Thomas Prest. Efficient identity-based encryption over ntru lattices. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 22–41. Springer, 2014.

[30] Léo Ducas and Phong Q. Nguyen. Learning a zonotope and more: Cryptanalysis of ntrusign countermeasures. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2-6, 2012. Proceedings*, volume 7658 of *Lecture Notes in Computer Science*, pages 433–450. Springer, 2012.

[31] Léo Ducas and Phong Q. Nguyen. Learning a zonotope and more: Cryptanalysis of ntrusign countermeasures. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology – ASIACRYPT 2012*, pages 433–450, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[32] Léo Ducas and Thomas Prest. A hybrid gaussian sampler for lattices over rings. *Cryptology ePrint Archive*, 2015.

[33] Léo Ducas and Thomas Prest. Fast fourier orthogonalization. In *Proceedings of the ACM on International Symposium on Symbolic and Algebraic Computation*, ISSAC '16, page 191–198, New York, NY, USA, 2016. Association for Computing Machinery.

[34] Léo Ducas, Thomas Espitau, and Eamonn W. Postlethwaite. Finding short integer solutions when the modulus is small. Cryptology ePrint Archive, Paper 2023/1125, 2023.

[35] Léo Ducas, Steven Galbraith, Thomas Prest, and Yang Yu. Integral matrix gram root and lattice gaussian sampling without floats. Cryptology ePrint Archive, Paper 2019/320, 2019. `https://eprint.iacr.org/2019/320`.

[36] Thomas Espitau. Mitaka: Faster, simpler, parallelizable and maskable hash-and-sign signatures on ntru lattices. In *Proceedings of the 8th ACM on ASIA Public-Key Cryptography Workshop*, APKC '21, page 1, New York, NY, USA, 2021. Association for Computing Machinery.

[37] Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. Side-channel attacks on BLISS lattice-based signatures: Exploiting branch tracing against strongswan and electromagnetic emanations in microcontrollers. In Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 1857–1874. ACM, 2017.

[38] Thomas Espitau, Thi Thu Quyen Nguyen, Chao Sun, Mehdi Tibouchi, and Alexandre Wallet. Antrag: Annular NTRU trapdoor generation - making mitaka as secure as falcon. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part VII*, volume 14444 of *Lecture Notes in Computer Science*, pages 3–36. Springer, 2023.

[39] U. Fincke and M. Pohst. Improved methods for calculating vectors of short length in a lattice, including a complexity analysis. *Math. Comp.*, 44(170):463–471, 1985.

[40] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over ntru. 2019.

[41] Nicolas Gama and Phong Nguyen. Predicting lattice reduction. volume 4965, pages 31–51, 04 2008.

[42] Craig Gentry, Jakob Jonsson, Jacques Stern, and Michael Szydlo. Cryptanalysis of the ntru signature scheme (nss) from eurocrypt 2001. In *Proceedings of the 7th International Conference on the Theory and Application of Cryptology and Information Security: Advances in Cryptology*, ASIACRYPT '01, page 1–20, Berlin, Heidelberg, 2001. Springer-Verlag.

[43] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. *Electron. Colloquium Comput. Complex.*, TR07-133, 2007.

[44] Oded Goldreich, Shafi Goldwasser, and Shai Halevi. Public-key cryptosystems from lattice reduction problems. In Burton S. Kaliski Jr., editor, *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, volume 1294 of *Lecture Notes in Computer Science*, pages 112–131. Springer, 1997.

[45] Jeffrey Hoffstein, Nick Howgrave-Graham, Jill Pipher, Joseph H. Silverman, and William Whyte. NTRUSIGN: digital signatures using the NTRU lattice. In Marc Joye, editor, *Topics in Cryptology - CT-RSA 2003, The Cryptographers' Track at the RSA Conference 2003, San Francisco, CA, USA, April 13-17, 2003, Proceedings*, volume 2612 of *Lecture Notes in Computer Science*, pages 122–140. Springer, 2003.

[46] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe Buhler, editor, *Algorithmic Number Theory, Third International Symposium, ANTS-III, Portland, Oregon, USA, June 21-25, 1998, Proceedings*, volume 1423 of *Lecture Notes in Computer Science*, pages 267–288. Springer, 1998.

[47] James Howe, Thomas Prest, Thomas Ricosset, and Mélissa Rossi. Isochronous gaussian sampling: From inception to implementation. Cryptology ePrint Archive, Paper 2019/1411, 2019. `https://eprint.iacr.org/2019/1411`.

[48] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In David S. Johnson, Ronald Fagin, Michael L. Fredman, David Harel, Richard M. Karp, Nancy A. Lynch, Christos H. Papadimitriou, Ronald L. Rivest, Walter L. Ruzzo, and Joel I. Seiferas, editors, *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 25-27 April, 1983, Boston, Massachusetts, USA*, pages 193–206. ACM, 1983.

[49] Ravi Kannan. Minkowski's convex body theorem and integer programming. *Mathematics of Operations Research*, 12(3):415–440, 1987.

[50] Subhash Khot. Hardness of approximating the shortest vector problem in lattices. In *45th Symposium on Foundations of Computer Science (FOCS 2004), 17-19 October 2004, Rome, Italy, Proceedings*, pages 126–135. IEEE Computer Society, 2004.

[51] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In *IACR Cryptology ePrint Archive*, 2018.

[52] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal security proofs for signatures from identification schemes. In *Proceedings, Part II, of the 36th Annual International Cryptology Conference on Advances in Cryptology — CRYPTO 2016 - Volume 9815*, page 33–61, Berlin, Heidelberg, 2016. Springer-Verlag.

[53] Philip N. Klein. Finding the closest lattice vector when it's unusually close. In David B. Shmoys, editor, *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms, January 9-11, 2000, San Francisco, CA, USA*, pages 937–941. ACM/SIAM, 2000.

[54] Donald E. Knuth. *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.

[55] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Boston, third edition, 1997.

[56] Ravi Kumar and D. Sivakumar. On polynomial-factor approximations to the shortest lattice vector length. *SIAM J. Discret. Math.*, 16(3):422–425, 2003.

[57] Thijs Laarhoven. *Search Problems in Cryptography*. Phd thesis, Eindhoven University of Technology, Eindhoven, Netherlands, 2015.

[58] Adeline Langlois and Damien Stehle. Worst-case to average-case reductions for module lattices. Cryptology ePrint Archive, Paper 2012/090, 2012. `https://eprint.iacr.org/2012/090`.

[59] Lenstra Lenstra and Lovasz. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261:515–534, 1982.

[60] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer, 2009.

[61] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755. Springer, 2012.

[62] Vadim Lyubashevsky. Basic lattice cryptography: The concepts behind kyber (ML-KEM) and dilithium (ML-DSA). *IACR Cryptol. ePrint Arch.*, page 1287, 2024.

[63] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.

[64] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *IACR Cryptol. ePrint Arch.*, page 230, 2012.

[65] Daniele Micciancio. Efficient reductions among lattice problems. In Shang-Hua Teng, editor, *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2008, San Francisco, California, USA, January 20-22, 2008*, pages 84–93. SIAM, 2008.

[66] Daniele Micciancio and Shafi Goldwasser. *Complexity of Lattice Problems: a cryptographic perspective*, volume 671 of *The Kluwer International Series in Engineering and Computer Science*. Kluwer Academic Publishers, 2002.

[67] Daniele Micciancio and Panagiotis Voulgaris. A deterministic single exponential time algorithm for most lattice problems based on voronoi cell computations. In Leonard J. Schulman, editor, *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC 2010, Cambridge, Massachusetts, USA, 5-8 June 2010*, pages 351–358. ACM, 2010.

[68] Daniele Micciancio and Michael Walter. Fast lattice point enumeration with minimal overhead. In Piotr Indyk, editor, *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2015, San Diego, CA, USA, January 4-6, 2015*, pages 276–294. SIAM, 2015.

[69] Daniele Micciancio and Bogdan Warinschi. A linear space algorithm for computing the hermite normal form. *Electron. Colloquium Comput. Complex.*, TR00-074, 2000.

[70] Phong Q. Nguyen and Oded Regev. Learning a parallelepiped: Cryptanalysis of GGH and NTRU signatures. In Serge Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings*, volume 4004 of *Lecture Notes in Computer Science*, pages 271–288. Springer, 2006.

[71] Phong Q. Nguyen and Oded Regev. Learning a parallelepiped: Cryptanalysis of ggh and ntru signatures. *J. Cryptol.*, 22(2):139–160, apr 2009.

[72] National Institute of Standards and Technology. Module-lattice-based digital signature standard (fips 204). Technical Report Federal Information Processing Standards (FIPS) 204, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, August 13 2024.

[73] Pascal Paillier and Damien Vergnaud. Discrete-log-based signatures may not be equivalent to discrete log. In *Advances in Cryptology - ASIACRYPT 2005, 11th International Conference on the Theory and Application of Cryptology and Information Security, Chennai, India, December 4-8, 2005, Proceedings*, volume 3788 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2005.

[74] Chris Peikert. An efficient and parallel gaussian sampler for lattices. In *Proceedings of the 30th Annual Conference on Advances in Cryptology*, CRYPTO'10, page 80–97, Berlin, Heidelberg, 2010. Springer-Verlag.

[75] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *J. Cryptology*, 13:361–396, 2000.

[76] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *J. Cryptol.*, 13(3):361–396, January 2000.

[77] Thomas Pornin and Thomas Prest. More efficient algorithms for the ntru key generation using the field norm. In *Public-Key Cryptography – PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, page 504–533, Berlin, Heidelberg, 2019. Springer-Verlag.

[78] Thomas Prest. A key-recovery attack against mitaka in the t-probing model. Cryptology ePrint Archive, Paper 2023/157, 2023. `https://eprint.iacr.org/2023/157`.

[79] Haifeng Qian and Zhibin Li. New signature schemes with tight security reductions. In *2006 International Conference on Computational Intelligence and Security*, volume 2, pages 1323–1326, 2006.

[80] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6):34:1–34:40, 2009.

[81] Oded Regev. The learning with errors problem (invited survey). In *Proceedings of the 25th Annual IEEE Conference on Computational Complexity, CCC 2010, Cambridge, Massachusetts, USA, June 9-12, 2010*, pages 191–204. IEEE Computer Society, 2010.

[82] Claus Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66:181–199, 08 1994.

[83] Claus Peter Schnorr. Lattice reduction by random sampling and birthday methods. In Helmut Alt and Michel Habib, editors, *STACS 2003*, pages 145–156, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[84] Claus-Peter Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Math. Program.*, 66:181–199, 1994.

[85] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Rev.*, 41(2):303–332, 1999.

[86] Damien Stehlé and Ron Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In Kenneth G. Paterson, editor, *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings*, volume 6632 of *Lecture Notes in Computer Science*, pages 27–47. Springer, 2011.

[87] Damien Stehlé and Ron Steinfeld. Making ntruencrypt and ntrusign as secure as standard worst-case problems over ideal lattices. Cryptology ePrint Archive, Paper 2013/004, 2013. `https://eprint.iacr.org/2013/004`.

[88] Masaya Yasuda. A survey of solving svp algorithms and recent strategies for solving the svp challenge. In Tsuyoshi Takagi, Masato Wakayama, Keisuke Tanaka, Noboru Kunihiro, Kazufumi Kimoto, and Yasuhiko Ikematsu, editors, *International Symposium on Mathematics, Quantum Theory, and Cryptography*, pages 189–207, Singapore, 2021. Springer Singapore.